REACT NATIVE

CHEAT SHEETS

One Codebase, Every Platform: React Native 0.87, Expo, and Each OS It Targets

Core Components · Expo Router · Native Modules · Reanimated · EAS · iOS · Android · macOS · tvOS · visionOS · Windows · Web

72 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP
01 Introduction & Architecture
02 Creating a Project
03 Project Structure
04 Running & Dev Workflow
05 TypeScript in React Native

REACT REFRESHER
06 Components, JSX & Props
07 State & Effect Hooks
08 Context, Reducers & Custom Hooks
09 React 19 Features

CORE COMPONENTS
10 View, Text & Image
11 TextInput & Keyboard
12 Pressable & Buttons
13 ScrollView
14 FlatList & SectionList
15 Modal & Feedback Components

STYLING & LAYOUT
16 StyleSheet Basics
17 Flexbox in React Native
18 Dimensions, Safe Areas & Responsive Layout
19 Theming & Dark Mode

NAVIGATION
20 Expo Router Basics
21 Expo Router Layouts
22 React Navigation
23 Deep Linking & Universal Links

PLATFORM-SPECIFIC CODE & CONFIG
24 Platform Module & File Extensions
25 App Config & Config Plugins
26 Cross-Platform Device APIs

DATA, STATE & STORAGE
27 Networking & Data Fetching
28 State Management Libraries
29 Key-Value & Secure Storage
30 Local Databases & Offline

ANIMATION & GESTURES
31 Animated API
32 Reanimated
33 Gesture Handler

NATIVE CODE & THE NEW ARCHITECTURE
34 Turbo Native Modules
35 Fabric Native Components
36 Expo Modules API

TESTING & DEBUGGING
37 Unit & Component Testing
38 End-to-End Testing
39 Debugging & Performance

BUILD, RELEASE & UPDATES
40 EAS Build
41 EAS Submit & Versioning
42 Over-the-Air Updates
43 Upgrading React Native & Expo

APPLE: IOS
44 iOS Project Anatomy
45 Xcode & Simulators
46 iOS Permissions & Entitlements
47 iOS UI Conventions & APIs
48 iOS Push Notifications
49 iOS Code Signing & Provisioning
50 App Store & TestFlight Release
51 iOS Extensions, Widgets & Live Activities

APPLE: IPADOS, MACOS, TVOS & VISIONOS
52 iPadOS
53 macOS
54 tvOS (Apple TV)
55 visionOS

ANDROID
56 Android Project Anatomy
57 Android Studio, Emulators & adb
58 Android Permissions
59 Android UI Conventions & APIs
60 Android Push Notifications
61 Build Types, Flavors & SDK Levels
62 Android App Signing
63 Google Play Release
64 Android TV, Large Screens & Other Devices

WINDOWS
65 React Native Windows Setup
66 Windows Native Modules & APIs
67 Packaging & Distributing on Windows

WEB & OTHER PLATFORMS
68 React Native for Web
69 Web-Specific Patterns & DOM Components
70 Other Out-of-Tree Platforms

QUICK REFERENCE
71 Quick Reference — Components, Styling & Navigation
72 Quick Reference — Data, Native & Release

RN

Table of Contents

72 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & Archite…
- 02 Creating a Project
- 03 Project Structure
- 04 Running & Dev Workflo…
- 05 TypeScript in React Na…

REACT REFRESHER 06 – 09

- 06 Components, JSX & Pr…
- 07 State & Effect Hooks
- 08 Context, Reducers & C…
- 09 React 19 Features

CORE COMPONENTS 10 – 15

- 10 View, Text & Image
- 11 TextInput & Keyboard
- 12 Pressable & Buttons
- 13 ScrollView
- 14 FlatList & SectionList
- 15 Modal & Feedback Co…

STYLING & LAYOUT 16 – 19

- 16 StyleSheet Basics
- 17 Flexbox in React Native
- 18 Dimensions, Safe Area…
- 19 Theming & Dark Mode

NAVIGATION 20 – 23

- 20 Expo Router Basics
- 21 Expo Router Layouts
- 22 React Navigation
- 23 Deep Linking & Univers…

PLATFORM-SPECIFIC CODE & CONFIG 24 – 26

- 24 Platform Module & File…
- 25 App Config & Config Pl…
- 26 Cross-Platform Device…

DATA, STATE & STORAGE 27 – 30

- 27 Networking & Data Fet…
- 28 State Management Lib…
- 29 Key-Value & Secure St…
- 30 Local Databases & Offl…

ANIMATION & GESTURES 31 – 33

- 31 Animated API
- 32 Reanimated
- 33 Gesture Handler

NATIVE CODE & THE NEW ARCHITECTURE 34 – 36

- 34 Turbo Native Modules
- 35 Fabric Native Compon…
- 36 Expo Modules API

TESTING & DEBUGGING 37 – 39

- 37 Unit & Component Tes…
- 38 End-to-End Testing
- 39 Debugging & Performa…

BUILD, RELEASE & UPDATES 40 – 43

- 40 EAS Build
- 41 EAS Submit & Versioni…
- 42 Over-the-Air Updates
- 43 Upgrading React Nativ…

APPLE: IOS 44 – 51

- 44 iOS Project Anatomy
- 45 Xcode & Simulators
- 46 iOS Permissions & Enti…
- 47 iOS UI Conventions & …
- 48 iOS Push Notifications
- 49 iOS Code Signing & Pr…
- 50 App Store & TestFlight …
- 51 iOS Extensions, Widget…

APPLE: IPADOS, MACOS, TVOS & VISIONOS 52 – 55

- 52 iPadOS
- 53 macOS
- 54 tvOS (Apple TV)
- 55 visionOS

ANDROID 56 – 64

- 56 Android Project Anato…
- 57 Android Studio, Emulat…
- 58 Android Permissions
- 59 Android UI Convention…
- 60 Android Push Notificat…
- 61 Build Types, Flavors & …
- 62 Android App Signing
- 63 Google Play Release
- 64 Android TV, Large Scre…

WINDOWS 65 – 67

- 65 React Native Windows …
- 66 Windows Native Modul…
- 67 Packaging & Distributi…

WEB & OTHER PLATFORMS 68 – 70

- 68 React Native for Web
- 69 Web-Specific Patterns…
- 70 Other Out-of-Tree Pla…

QUICK REFERENCE 71 – 72

- 71 Quick Reference — Co…
- 72 Quick Reference — Dat…

Introduction & Architecture

WHAT REACT NATIVE IS

React Native lets you build **native apps with React**. Your components render real platform views (`UIView` on iOS, `android.view.View` on Android), not a WebView. JavaScript runs on the **Hermes** engine and talks to native code through **JSI**, a C++ interface that replaced the old asynchronous JSON bridge. Since 0.82 the **New Architecture** is the only option.

VERSION SNAPSHOT (OCT 2026)

- React Native `0.87` stable; `0.88` due Oct 12, `0.89` Dec 7
- About one minor release every two months; latest 3 are supported
- React `19.2`, Hermes V1 default since `0.84`
- Expo SDK 57 ships RN `0.86`; SDK 58 (beta) moves to `0.88`

HELLO, WORLD

```tsx
// App.tsx — the smallest complete app
import { StyleSheet, Text, View } from 'react-native'

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello, React Native!</Text>
    </View>
  )
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center' },
  title: { fontSize: 24, fontWeight: '600' },
})
```

NOTES

- `View` — becomes `UIView` / `ViewGroup`; the basic layout box
- `Text` — all strings must live inside `<Text>`
- `StyleSheet.create` — JS objects, not CSS files
- `flex: 1` — fill the screen; layout is Flexbox via **Yoga**
- Default export is the root component the native app mounts
- Same code runs on iOS and Android unchanged

IOS

- `<View>` renders a native UIView; `<Text>` a native text view
- Building for iOS needs a Mac with Xcode
- Every native change goes through App Store review

ANDROID

- `<View>` renders a native ViewGroup; `<Text>` a TextView
- Android builds on macOS, Windows or Linux
- Play Store releases ship as `.aab` bundles

NEW ARCHITECTURE PIECES

PIECE	WHAT IT DOES
JSI	C++ API: JS holds direct references to native objects, sync calls
Fabric	New renderer; concurrent React, sync layout reads
TurboModules	Lazy-loaded, type-safe native modules
Codegen	Generates native glue from TypeScript specs
Hermes	JS engine tuned for mobile: bytecode, fast startup
Yoga	Cross-platform Flexbox layout engine (C++)
Metro	JS bundler and dev server
Interop layer	Runs older Paper components on Fabric

WHERE IT RUNS

PLATFORM	MAINTAINED BY
iOS / iPadOS	Meta (core, in-tree)
Android	Meta (core, in-tree)
macOS · Windows	Microsoft (out-of-tree)
visionOS	Callstack (out-of-tree)
tvOS · Android TV	Community (react-native-tvos)
Web	Community (react-native-web)
OpenHarmony	Huawei partners (out-of-tree)

⚠ COMMON MISTAKES

- Expecting HTML tags like `<div>` or `<p>`; only RN components (or a web-only renderer) work
- Following pre-2025 tutorials that toggle `newArchEnabled`; the flag is ignored since 0.82
- Assuming one codebase means zero native work; signing, permissions, and store setup are per OS

✓ TIPS & BEST PRACTICES

- Start with Expo (`create-expo-app`) unless you have a reason to manage native folders by hand
- Check a library on **reactnative.directory** for New Architecture and platform support before installing
- Read the release blog before upgrading; each minor can contain breaking changes

QUICK SUMMARY

- `View` / `Text` — native UI primitives
- `JSI` — direct JS ↔ native calls
- `Fabric` / `TurboModules` — renderer and modules
- `Hermes` — default JS engine

Creating a Project

TWO WAYS TO START

The React Native docs recommend a **framework**, and today that means **Expo**: it generates the native projects for you, adds routing, and handles builds. The **Community CLI** creates a bare app with `ios/` and `android/` folders you own and edit directly. Both run the same React Native; you can add native code to an Expo app at any time.

PREREQUISITES

- Node.js `22.13+` (required since RN 0.87)
- iOS builds need a Mac with Xcode; Android needs Android Studio + JDK 17
- Watchman recommended on macOS (`brew install watchman`)
- A physical device works with Expo Go or a development build

CREATE AND RUN

```
# Expo (recommended)
npx create-expo-app@latest MyApp      # default: Expo Router + TS
npx create-expo-app@latest MyApp --template blank-typescript
cd MyApp && npx expo start               # QR code, press i / a / w

# Bare React Native (Community CLI)
npx @react-native-community/cli@latest init MyApp
npx @react-native-community/cli@latest init MyApp --version 0.87.0
cd MyApp
npx react-native run-ios                 # needs pod install first time
npx react-native run-android

# Check your machine
npx react-native doctor                  # bare
npx expo-doctor                          # Expo
```

NOTES

- → `@latest` — always pull the current template
- → `--template` — `default`, `blank`, `blank-typescript`, `tabs`
- → Both paths produce a TypeScript project by default
- → `npx expo start` — starts Metro and the dev server
- → `init --version` — pin an RN version for a bare app
- → `doctor` — checks Node, JDK, Xcode, SDK paths

IOS

- Install Xcode, its command line tools and a simulator runtime
- CocoaPods installs native deps (`pod install`)
- The iOS Simulator only runs on macOS

ANDROID

- Android Studio provides the SDK, emulator and build tools
- Set `ANDROID_HOME` ; add `platform-tools` to `PATH`
- JDK 17 — Android Studio's bundled JBR works

EXPO VS BARE

TOPIC	EXPO	BARE CLI
Native folders	Generated (`prebuild`)	Checked in, hand-edited
Routing	Expo Router built in	Add React Navigation
Builds	EAS Build or local	Xcode / Gradle
Native APIs	`expo-*` SDK modules	Community libraries
OTA updates	`expo-updates`	DIY or third party
Upgrades	`expo install --fix`	Upgrade Helper diff
Web support	Built in	Add react-native-web

ENVIRONMENT CHECKLIST

TOOL	USED FOR
Xcode + CLT	iOS simulator, builds
CocoaPods / Ruby	iOS dependencies
Android Studio	SDK, emulator, Gradle
JDK 17	Android builds
ANDROID_HOME	SDK path for the CLI
Watchman	Fast file watching (macOS)
Node 22.13+	Metro and CLI runtime

⚠ COMMON MISTAKES

- Running `npx react-native init` ; the old global init was removed, use `@react-native-community/cli init`
- Using Node 18 or 20; current RN requires Node `22.13` or newer
- Spaces or special characters in the project path, which break native build scripts

✓ TIPS & BEST PRACTICES

- Use `nvm` or `fnm` and add `.nvmrc` so every machine uses the same Node
- Expo Go only runs SDK modules; switch to a **development build** once you add custom native code
- Commit `package-lock.json` (or your manager's lockfile) from day one

QUICK SUMMARY

- `create-expo-app` — recommended start
- `cli init` — bare project you own
- `expo start` / `run-ios` — run it
- `doctor` — diagnose setup

Project Structure

WHAT'S IN A NEW APP

An Expo app keeps your code in `app/` (Expo Router screens) plus shared folders you create. The `ios/` and `android/` folders are **generated** by `npx expo prebuild` and usually git-ignored. A bare CLI app commits those folders and starts from `index.js`, which registers the root component with `AppRegistry`.

CONVENTIONS

- `app/` — routes (Expo Router); `src/` optional
- `components/`, `hooks/`, `constants/` — shared code
- `assets/` — images, fonts, splash, icons
- Native folders are build outputs in Expo, source in bare

EXPO ROUTER LAYOUT

```
MyApp/
├── app/                    # every file is a route
│   ├── _layout.tsx         # root layout (Stack)
│   ├── index.tsx           # "/"
│   └── (tabs)/             # group: tab navigator
├── components/
├── assets/images/
├── app.json                # Expo config (name, icons, plugins)
├── package.json            # "main": "expo-router/entry"
├── tsconfig.json
├── metro.config.js         # optional bundler config
├── ios/  android/          # generated by `expo prebuild`
└── .gitignore
```

NOTES

- → `_layout.tsx` — wraps sibling routes
- → `(tabs)` — parentheses group routes without adding to the URL
- → `"main"` — `expo-router/entry` boots the router
- → `app.json` — single source for native config
- → Add `ios/` and `android/` to `.gitignore` when using CNG

IOS

- `ios/` holds the Xcode project, Podfile and Info.plist
- Commit `Podfile.lock` in bare apps; never commit `Pods/`
- App icons live in `Images.xcassets`

ANDROID

- `android/` is a Gradle project; the app module is `android/app`
- `AndroidManifest.xml` declares permissions and activities
- `build/` and `.gradle/` are generated — keep out of git

KEY FILES

FILE	PURPOSE
`index.js`	Bare entry: `AppRegistry.registerComponent()`
`App.tsx`	Root component (bare / blank template)
`app.json / app.config.ts`	Expo app config
`metro.config.js`	Bundler: asset types, aliases
`babel.config.js`	Transforms (Reanimated plugin, etc.)
`react-native.config.js`	CLI: autolinking overrides
`tsconfig.json`	TS config + path aliases
`eas.json`	EAS Build / Submit profiles

NATIVE FOLDERS

PATH	CONTAINS
`ios/Podfile`	CocoaPods dependencies
`ios/MyApp/AppDelegate.swift`	iOS app entry
`ios/MyApp/Info.plist`	iOS config & permissions
`android/app/build.gradle`	App module build
`android/.../MainActivity.kt`	Android entry Activity
`AndroidManifest.xml`	Android config & permissions
`ios/*.xcworkspace`	Open this in Xcode
`android/gradle.properties`	Build flags (Hermes, edge-to-edge)

⚠ COMMON MISTAKES

- Hand-editing `ios/` or `android/` in an Expo CNG project; the next `prebuild --clean` wipes it, so use config plugins
- Renaming `index.js` or `App.tsx` without updating `main` / `registerComponent`
- Committing `ios/Pods` or `android/build`; they're huge and machine-specific

✓ TIPS & BEST PRACTICES

- Use path aliases (`@/components`) via `tsconfig paths`; Expo supports them without Babel config
- Keep platform files next to each other: `Button.ios.tsx`, `Button.android.tsx`
- Treat `app.json` as the place to change names, icons, and permissions

QUICK SUMMARY

- `app/` — routes in Expo Router
- `index.js` — bare entry point
- `ios/` · `android/` — native projects
- `app.json` — native config source

Running & Dev Workflow

THE INNER LOOP

Metro bundles your JS and serves it to the app over the network. **Fast Refresh** swaps edited components in place and keeps state. You run code in **Expo Go** (a prebuilt sandbox app), in a **development build** (your own app with `expo-dev-client`), or in a bare debug build from Xcode/Gradle. Native code changes need a rebuild; JS changes don't.

WHICH RUNNER?

- **Expo Go** — fastest start, only Expo SDK modules
- **Dev build** — any native library, your own icon & config
- **Release build** — no dev menu, optimized bundle
- A new SDK's Expo Go can lag the stores; get it via CLI or `eas go`

COMMANDS

```
npx expo start                  # Metro + QR code
npx expo start --clear          # reset Metro cache
npx expo start --tunnel         # device on another network

npx expo run:ios                # prebuild + build + install
npx expo run:ios --device       # pick a connected iPhone
npx expo run:android --variant release

npx expo install expo-dev-client   # turn into a dev build

# Bare CLI
npx react-native start --reset-cache
npx react-native run-ios --simulator "iPhone 17"
npx react-native run-android --mode debugOptimized
```

NOTES

- → `--clear` — fixes stale bundles after config changes
- → `--tunnel` — uses ngrok; slower but works across networks
- → `run:ios` — compiles native code; needed after native changes
- → `expo-dev-client` — adds the dev launcher to your own build
- → `debugOptimized` — Android build type with faster JS

IOS

- `npx expo run:ios` builds and opens the Simulator
- Dev menu: Cmd+D in the Simulator, shake on a device
- The Simulator reaches Metro on localhost directly

ANDROID

- `npx expo run:android` installs on the emulator or USB device
- Dev menu: Cmd+M / Ctrl+M in the emulator, shake on a device
- USB devices reach Metro via `adb reverse tcp:8081 tcp:8081`

METRO TERMINAL KEYS

KEY	ACTION
`i / a / w`	Open iOS sim / Android / web
`r`	Reload the app
`m`	Toggle dev menu on device
`j`	Open React Native DevTools
`shift+m`	More tools
`?`	Show all commands
`o`	Open project in editor
`s`	Switch Expo Go / dev build

OPENING THE DEV MENU

WHERE	HOW
`iOS Simulator`	`Cmd+D` or `Ctrl+Cmd+Z`
`Android Emulator`	`Cmd+M` / `Ctrl+M`
`Physical device`	Shake the device
`Any`	Press `m` in Metro terminal
`Dev menu items`	Reload · DevTools · Perf Monitor · Element Inspector

⚠ COMMON MISTAKES

- Expecting Fast Refresh to pick up native or `app.json` changes; those need a rebuild
- Phone and computer on different Wi-Fi networks, so the bundle never loads; use `--tunnel` or USB
- Leaving a stale Metro running on port 8081 from another project

✓ TIPS & BEST PRACTICES

- Add `// @refresh reset` to a file to force a remount on every edit
- Create one dev build per native change, then iterate on JS for days
- Use `adb reverse tcp:8081 tcp:8081` for an Android device over USB

QUICK SUMMARY

- `Metro` — JS bundler & dev server
- `Fast Refresh` — keeps state across edits
- `Expo Go` vs dev build — SDK-only vs any native
- `r` / `j` / `m` — reload, DevTools, menu

TypeScript in React Native

tsconfig · @react-native/typescript-config · strict API (0.87) · ViewInstance / TextInputInstance refs · *Props types · deep-import removal

TYPED BY DEFAULT

New Expo and CLI projects are TypeScript out of the box. Metro strips types through Babel, so **type errors never block the bundle**; run `tsc` to check. Since **0.87** the `react-native` package ships a **strict API**: types are generated from source and only what's exported from the package root is public. Deep imports like `react-native/Libraries/...` are gone.

CONFIG AND TYPED COMPONENTS

```
// tsconfig.json (bare)
{ "extends": "@react-native/typescript-config",
  "compilerOptions": { "strict": true,
    "paths": { "@/*": ["./src/*"] } } }
// Expo: { "extends": "expo/tsconfig.base" }

import { useRef } from 'react'
import { Pressable, Text, TextInput,
  type PressableProps, type TextInputInstance } from 'react-native'

type Props = PressableProps & { label: string }
export function Btn({ label, ...rest }: Props) {
  return <Pressable {...rest}><Text>{label}</Text></Pressable>
}

const inputRef = useRef<TextInputInstance>(null)
inputRef.current?.focus()
```

STRICT API CHANGES

- Prop types are `*Props`; old `*Properties` aliases removed
- Ref types: `ViewInstance`, `TextInputInstance`, `HostInstance`
- `NativeMethods` types removed → use `HostInstance`
- Temporary opt-out works through 0.88 only

NOTES

- → `@react-native/typescript-config` — base config for bare apps
- → `expo/tsconfig.base` — Expo's base
- → `type` imports — erased at build time
- → `PressableProps & {...}` — extend built-in props
- → `TextInputInstance` — 0.87 ref type for `TextInput`

IOS

- `Platform.OS === 'ios'` narrows to iOS-only fields (`isPad`)
- `Platform.Version` is a string, e.g. `'18.2'`
- TS needs a base `Button.tsx` / `.d.ts` beside `Button.ios.tsx`

ANDROID

- Android-only props are typed too (`android_ripple`, `elevation`)
- `Platform.Version` is the API level number, e.g. `35`
- Keep `.ios.tsx` and `.android.tsx` exports identical

COMMON TYPES

TYPE	USE
`ViewStyle / TextStyle / ImageStyle`	Style objects
`StyleProp<ViewStyle>`	`style` prop that accepts arrays
`ViewProps / TextProps`	Extend built-in components
`ListRenderItem<T>`	FlatList `renderItem`
`GestureResponderEvent`	`onPress` event
`NativeSyntheticEvent<T>`	Native event payloads
`ColorValue`	Colors incl. `PlatformColor`
`ScrollViewInstance`	ScrollView ref type

STRICT API OPT-OUT (≤ 0.88)

SETTING	VALUE
`customConditions`	`["react-native", "react-native-legacy-deep-imports"]`
`Where`	`compilerOptions` in tsconfig
`Migration`	`/migrate-to-strict-api` skill, migration guide
`Lifetime`	Removed after 0.88
`Root types`	`react-native` exports only
`Old aliases`	`*Properties` → `*Props`

⚠ COMMON MISTAKES

- Assuming a passing Metro build means types are fine; Babel drops types, so run `npx tsc --noEmit` in CI
- Importing from `react-native/Libraries/...`; 0.87 blocks deep imports, use root exports
- Typing `style` as `ViewStyle` instead of `StyleProp<ViewStyle>`, which rejects arrays

✓ TIPS & BEST PRACTICES

- Add `"typecheck": "tsc --noEmit"` to scripts and run it before commits
- Use `satisfies` for style maps or route params to keep literal types
- Prefer `ComponentProps<typeof Foo>` to reuse a third-party component's props

QUICK SUMMARY

- `tsc --noEmit` — type check separately
- Strict API — root exports only (0.87)
- `*Props` / `*Instance` — prop & ref types
- `StyleProp<T>` — style prop type

Sheet 06

Components, JSX & Props

function components · JSX rules · props · children · conditional rendering · lists & key · fragments

REACT REFRESHER: COMPONENTS

A **component** is a function that returns JSX. **Props** are its read-only inputs; `children` is whatever you nest inside it. JSX compiles to function calls, so you embed any JS expression in `{}`. In React Native, the JSX uses RN components (`View`, `Text`) instead of HTML tags, but the rules are identical to React on the web.

JSX RULES

- One root element per return (or a fragment `<>…</>`)
- Components start with a capital letter
- `{}` holds expressions, not statements
- Lists need a stable `key` on each item

PROPS, CHILDREN, CONDITIONS, LISTS

```tsx
type CardProps = {
  title: string; done?: boolean; children?: React.ReactNode }

function Card({ title, done = false, children }: CardProps) {
  return (
    <View style={{ padding: 12 }}>
      <Text>{title}{done && ' ✓'}</Text>
      {children}
    </View>
  )
}

function TodoList({ todos }: { todos: Todo[] }) {
  if (todos.length === 0) return <Text>Nothing to do</Text>
  return (
    <>
      {todos.map(t => (
        <Card key={t.id} title={t.title} done={t.done}>
          {t.note ? <Text>{t.note}</Text> : null}
        </Card>
      ))}
    </>
  )
}
```

NOTES

- → `done = false` — default value via destructuring
- → `children` — nested JSX passed by the parent
- → `&&` — render only when true; ternary for either/or
- → Early `return` — simplest conditional render
- → `key={t.id}` — stable IDs, never array index if order changes

IOS

- Default font is San Francisco (`System`)
- iOS-only props sit beside shared ones, e.g. `contentInsetAdjustmentBehavior`
- `<Text selectable>` shows the native copy menu

ANDROID

- Default font is Roboto
- Android-only props use an `android_` prefix or `*Android` suffix
- `<Button>` titles render uppercase (Material style)

CONDITIONAL PATTERNS

PATTERN	USE WHEN
`cond && <X/>`	Show or nothing (cond must be boolean)
`cond ? <A/> : <B/>`	Pick one of two
`Early return`	Whole component differs
`null`	Render nothing

PROP PATTERNS

PATTERN	EXAMPLE
`Spread`	`<Btn {...rest} />`
`Callback`	`onSave={(v) => setV(v)}`
`Render prop`	`renderItem={({item}) => …}`
`Component prop`	`ListHeaderComponent={Header}`

⚠ COMMON MISTAKES

- `{count && <Text/>}` with `count = 0`; RN tries to render `0` outside `<Text>` and crashes, use `count > 0 &&`
- Mutating props or objects in props; treat them as read-only
- Bare strings inside `View` (`<View>Hi</View>`); text must be in `<Text>`

✓ TIPS & BEST PRACTICES

- Keep components small; extract when a block has its own state or is reused
- Use `React.ReactNode` for `children` and slot props
- Destructure props in the signature so defaults and types sit together

QUICK SUMMARY

- Component — function returning JSX
- `props` — read-only inputs
- `children` — nested content
- `key` — stable identity in lists

State & Effect Hooks

REACT REFRESHER: HOOKS

State (`useState`) is data that triggers a re-render when it changes. **Effects** (`useEffect`) sync your component with something outside React: subscriptions, timers, native listeners. **Refs** (`useRef`) hold mutable values or native handles without re-rendering. `useMemo` and `useCallback` cache values and functions between renders, though the React Compiler can do this for you.

HOOK RULES

- Call hooks at the top level, never in loops or conditions
- Only call them from components or custom hooks
- State updates are async and batched
- Effects run after the frame is painted

HOOKS IN PRACTICE

```tsx
function Timer() {
  const [seconds, setSeconds] = useState(0)
  const [running, setRunning] = useState(false)
  const startedAt = useRef<number | null>(null)

  useEffect(() => {
    if (!running) return
    const id = setInterval(() => setSeconds(s => s + 1), 1000)
    return () => clearInterval(id)          // cleanup
  }, [running])                             // re-run when running changes

  useEffect(() => {
    const sub = AppState.addEventListener('change', st => {
      if (st === 'background') setRunning(false)
    })
    return () => sub.remove()
  }, [])                                    // mount / unmount only

  const label = useMemo(() => formatTime(seconds), [seconds])
  const toggle = useCallback(() => setRunning(r => !r), [])
  return <Pressable onPress={toggle}><Text>{label}</Text></Pressable>
}
```

NOTES

- → `setSeconds(s => s + 1)` — updater form avoids stale state
- → `return () => …` — cleanup on re-run and unmount
- → `[]` — effect runs once on mount
- → `AppState` — RN API: active / background / inactive
- → `useRef` — persists without causing renders

IOS

- `AppState` passes through `'inactive'` (Control Center, app switcher)
- JS timers pause soon after the app is backgrounded
- `useColorScheme` updates live when Dark Mode toggles

ANDROID

- `AppState` has no `'inactive'` — only active / background
- Add `BackHandler` listeners in `useEffect`; return `remove()`
- Test resize and multi-window: dimensions change at runtime

CORE HOOKS

HOOK	PURPOSE
`useState`	Local state + setter
`useEffect`	Sync with external systems
`useLayoutEffect`	Measure before paint
`useRef`	Mutable box / native ref
`useMemo`	Cache a computed value
`useCallback`	Cache a function

DEPENDENCY ARRAYS

ARRAY	EFFECT RUNS
`none`	After every render
`[]`	Once after mount
`[a, b]`	When a or b changes
`cleanup`	Before re-run and on unmount

⚠ COMMON MISTAKES

- Forgetting to remove listeners (`sub.remove()`); leaks pile up across navigation
- Reading state right after `setState` and expecting the new value; it updates on the next render
- Using `useEffect` to derive data from props; compute it during render instead

✓ TIPS & BEST PRACTICES

- Turn on the React Compiler and drop most manual `useMemo` / `useCallback`
- Use `useFocusEffect` (navigation) for work that should pause when a screen is hidden
- Keep effects small, one concern per effect

QUICK SUMMARY

- `useState` — re-render on change
- `useEffect` — side effects + cleanup
- `useRef` — mutable, no re-render
- `useMemo` / `useCallback` — memoize

Sheet 08

Context, Reducers & Custom Hooks

createContext · useContext · providers · useReducer · custom hooks · rules of hooks

REACT REFRESHER: SHARING LOGIC

Context passes a value deep into the tree without prop drilling; theme, auth user, and locale are typical. **useReducer** handles state with several related transitions. **Custom hooks** are functions starting with `use` that bundle hooks into a reusable piece. Together they cover most app state before you reach for a library.

WHEN TO USE WHAT

- Prop passing — 1–2 levels deep
- Context — app-wide, rarely changing values
- `useReducer` — many actions on one state object
- Library (Zustand, Redux) — frequent global updates

AUTH CONTEXT + REDUCER + HOOK

```tsx
type State = { user: User | null; loading: boolean }
type Action = { type: 'login'; user: User } | { type: 'logout' }

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'login':  return { user: action.user, loading: false }
    case 'logout': return { user: null, loading: false }
  }
}

type Ctx = { state: State; dispatch: Dispatch<Action> }
const AuthContext = createContext<Ctx | null>(null)

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [state, dispatch] =
    useReducer(reducer, { user: null, loading: true })
  return <AuthContext value={{ state, dispatch }}>{children}</AuthContext>
}

export function useAuth() {
  const ctx = useContext(AuthContext)
  if (!ctx) throw new Error('useAuth must be inside AuthProvider')
  return ctx
}
```

NOTES

→ Discriminated union — exhaustive, typed actions

→ `<AuthContext value>` — React 19 renders context directly as provider

→ `null` default — lets the hook detect a missing provider

→ `useAuth` — custom hook hides the context plumbing

→ Wrap the root layout in `<AuthProvider>`

IOS

- iOS can terminate a backgrounded app silently — context is lost
- Keep tokens in the Keychain (SecureStore), not only context
- Test cold start: swipe the app away in the switcher

ANDROID

- Low-RAM devices kill background processes often
- Test with Developer options → *Don't keep activities*
- Rehydrate providers from storage before rendering screens

APIS

API	PURPOSE
`createContext(default)`	Create a context object
`<Ctx value={v}>`	Provide (React 19; `.Provider` still works)
`useContext(Ctx) / use(Ctx)`	Read nearest value
`useReducer(fn, init)`	State + dispatch
`useSyncExternalStore`	Subscribe to external stores

CONTEXT PERFORMANCE

ISSUE	FIX
`Every consumer re-renders`	Split contexts by concern
`New object each render`	Memoize value or use the Compiler
`Fast-changing data`	Use a store (Zustand)
`Deep trees`	Keep providers close to users

⚠ COMMON MISTAKES

- Putting rapidly changing values (scroll position, timers) in context; every consumer re-renders
- Calling a custom hook conditionally; hooks inside it still follow the rules of hooks
- Forgetting the provider, so consumers silently get the default value

✓ TIPS & BEST PRACTICES

- Throw a clear error in the custom hook when the provider is missing
- Keep reducers pure; do async work in event handlers, then `dispatch`
- Name hooks `useX` so lint rules check them

QUICK SUMMARY

- `createContext` / `use` — share values
- `useReducer` — action-based state
- Custom hook — reusable `useX` logic
- Split contexts — fewer re-renders

React 19 Features

REACT REFRESHER: WHAT'S NEW

React Native 0.87 ships **React 19.2**. You get `use()` for reading promises and context, **Actions** for async transitions with built-in pending state, `useOptimistic` for instant UI, and `ref` as a normal prop (no more `forwardRef`). The **React Compiler** auto-memoizes components and is on by default in new Expo projects.

GOOD TO KNOW IN RN

- No `<form>` element; call Actions from `onPress`
- `useFormStatus` is web-only (needs `<form>`)
- Concurrent features work because of Fabric
- `<Activity>` (19.2) hides UI while keeping state

ACTIONS + OPTIMISTIC UI

```tsx
function LikeButton({ post }: { post: Post }) {
  const [optimisticLikes, addLike] = useOptimistic(
    post.likes, (current, delta: number) => current + delta)

  const [error, likeAction, isPending] = useActionState(
    async (_prev: string | null) => {
      addLike(1)                                  // instant UI
      try { await api.like(post.id); return null }
      catch { return 'Could not like post' }
    }, null)

  return (
    <Pressable disabled={isPending}
      onPress={() => startTransition(likeAction)}>
      <Text>♥ {optimisticLikes}</Text>
      {error && <Text style={{ color: 'red' }}>{error}</Text>}
    </Pressable>
  )
}

function Profile({ userPromise }: { userPromise: Promise<User> }) {
  const user = use(userPromise)        // suspends until resolved
  return <Text>{user.name}</Text>
}
```

NOTES

- → `useOptimistic` — shows a value until the Action settles
- → `useActionState` — returns `[state, action, isPending]`
- → `startTransition` — needed when calling an action outside a form
- → `use(promise)` — wrap the caller in `<Suspense>`
- → Create promises outside render (cache them)

IOS

- Same React 19 runtime as Android (Hermes)
- Fire Actions from `onSubmitEditing` with `returnKeyType="go"`
- Optimistic UI hides latency on slow cellular links

ANDROID

- `returnKeyType` sets the keyboard's IME action button
- `submitBehavior="blurAndSubmit"` closes the keyboard on submit
- Debug builds are slow — judge transitions in release

REACT 19 APIS

API	REPLACES / ADDS
`use(promise | ctx)`	Read in render; can be conditional
`useActionState`	Pending + result for async work
`useOptimistic`	Temporary optimistic state
`ref prop`	Replaces `forwardRef`
`<Activity mode>`	Hide but preserve subtrees (19.2)
`useEffectEvent`	Non-reactive logic in effects (19.2)

REACT COMPILER

TOPIC	DETAIL
What	Build-time auto-memoization
Expo	`experiments.reactCompiler` in app.json
Bare	`babel-plugin-react-compiler`
Opt out	`'use no memo'` in a component
Lint	`eslint-plugin-react-hooks` compiler rules

⚠ COMMON MISTAKES

- Creating a new promise in render and passing it to `use()`; it refetches forever
- Calling an Action from `onPress` without `startTransition`; `isPending` never updates
- Still wrapping components in `forwardRef` in new code; pass `ref` as a prop

✓ TIPS & BEST PRACTICES

- Pair `use()` with `<Suspense fallback={<ActivityIndicator/>}>` and an error boundary
- Use `<Activity>` for tab screens you want to keep warm without rendering
- Let the Compiler handle memoization before adding manual `memo`

QUICK SUMMARY

- `use()` — read promises / context
- Actions — async with `isPending`
- `useOptimistic` — instant feedback
- React Compiler — auto memo

View, Text & Image

THE THREE BUILDING BLOCKS

`View` is the layout container (like a `div` with Flexbox). `Text` is the only component that can display strings, and nested `Text` inherits styles from its parent. `Image` shows local assets (via `require`) or remote URLs (via `{ uri }`); remote images **need explicit width and height**. Expo apps usually swap in `expo-image` for caching and better formats.

RULES THAT BITE

- Strings outside `<Text>` throw a render error
- `View` can't be inside `Text` on Android (inline views are limited)
- Text styles inherit only within nested `Text`
- Remote `Image` with no size renders at 0×0

TYPICAL USAGE

```
<View style={{ flexDirection: 'row', gap: 12, padding: 16 }}>
  <Image
    source={{ uri: user.avatarUrl }}           // remote: size required
    style={{ width: 48, height: 48, borderRadius: 24 }}
    accessibilityIgnoresInvertColors
  />
  <View style={{ flex: 1 }}>
    <Text style={{ fontWeight: '700' }} numberOfLines={1}>
      {user.name}{' '}
      <Text style={{ color: 'gray' }}>@{user.handle}</Text>
    </Text>
    <Text numberOfLines={2} ellipsizeMode="tail">{post.body}</Text>
  </View>
  <Image source={require('./assets/icon.png')} />  {/* local */}
</View>

// expo-image: caching, blurhash, transitions
<ExpoImage source={url} placeholder={{ blurhash }}
  contentFit="cover" transition={200} style={{ flex: 1 }} />
```

NOTES

- → `{ uri }` — network or file URL; set width/height
- → `require()` — bundled asset; size inferred
- → Nested `Text` — inherits bold, color, font
- → `numberOfLines` + `ellipsizeMode` — truncate
- → `expo-image` — `contentFit` replaces `resizeMode`

IOS

- `shadowColor` / `shadowOffset` / `shadowOpacity` / `shadowRadius` shadows
- `dynamicTypeRamp` on Text scales with Dynamic Type
- `capInsets` makes stretchable images

ANDROID

- `elevation` gives Material shadows
- `includeFontPadding: false` fixes vertical text offset
- `resizeMethod` controls how large bitmaps are downscaled

TEXT PROPS

PROP	EFFECT
`numberOfLines`	Max lines, then truncate
`ellipsizeMode`	`head` · `middle` · `tail` · `clip`
`selectable`	Allow copy
`adjustsFontSizeToFit`	Shrink to fit (with `numberOfLines`)
`allowFontScaling`	Respect OS text size (default true)
`onPress`	Pressable text; auto role `link` (0.84+)

IMAGE PROPS

PROP	EFFECT
`resizeMode`	`cover` · `contain` · `stretch` · `center`
`defaultSource`	Placeholder while loading
`onLoad / onError`	Load events
`blurRadius`	Blur the image
`Image.prefetch(url)`	Warm the cache
`ImageBackground`	Deprecated 0.87 → View + absolute Image

⚠ COMMON MISTAKES

- Rendering `{0}` or a string directly in a `View`; wrap text in `<Text>`
- Remote images without width/height; they're invisible, not broken
- Setting `fontFamily` on a parent `View` and expecting children to inherit; only nested `Text` inherits

✓ TIPS & BEST PRACTICES

- Make a shared `<AppText>` with your default font, color, and size
- Use `@2x` / `@3x` suffixes for local images; RN picks the right density
- Prefer `expo-image` (works in bare apps too) for lists of remote images

QUICK SUMMARY

- `View` — layout container
- `Text` — all strings, inherits when nested
- `Image` — `require()` or `{ uri }` + size
- `expo-image` — caching & placeholders

TextInput & Keyboard

GETTING TEXT FROM USERS

`TextInput` is the native text field. Control it with `value` + `onChangeText`, which gives you the string directly. Choose the right **keyboard** with `keyboardType`, `textContentType` (iOS) and `autoComplete` (Android) for autofill. Then deal with the keyboard covering your form: `KeyboardAvoidingView`, `ScrollView` settings, or the `react-native-keyboard-controller` library.

BEHAVIORS

- `onChangeText(text)` — string; `onChange` — native event
- `multiline` changes Enter from submit to newline
- Focus order: `ref.focus()` from `onSubmitEditing`
- Android pads with `windowSoftInputMode=adjustResize`

A LOGIN FORM

```
const passRef = useRef<TextInputInstance>(null)

<KeyboardAvoidingView style={{ flex: 1 }}
  behavior={Platform.OS === 'ios' ? 'padding' : undefined}>
  <ScrollView keyboardShouldPersistTaps="handled">
    <TextInput
      value={email} onChangeText={setEmail}
      placeholder="Email" keyboardType="email-address"
      autoCapitalize="none" autoComplete="email"
      textContentType="emailAddress"
      returnKeyType="next"
      onSubmitEditing={() => passRef.current?.focus()}
    />
    <TextInput
      ref={passRef} value={password} onChangeText={setPassword}
      placeholder="Password" secureTextEntry
      textContentType="password" returnKeyType="go"
      onSubmitEditing={login}
    />
  </ScrollView>
</KeyboardAvoidingView>
```

NOTES

- → `behavior="padding"` — the usual iOS choice
- → `keyboardShouldPersistTaps="handled"` — taps reach buttons
- → `autoCapitalize="none"` — essential for emails
- → `textContentType` / `autoComplete` — password managers
- → `returnKeyType` + `onSubmitEditing` — field-to-field flow

IOS

- `KeyboardAvoidingView behavior="padding"`
- `textContentType` enables autofill (`'oneTimeCode'` , `'password'`)
- `clearButtonMode` adds the native clear (×) button

ANDROID

- The window resizes for the keyboard; often `behavior="height"` or none
- `autoComplete` drives autofill (`'sms-otp'` , `'password'`)
- `underlineColorAndroid="transparent"` removes the underline

KEYBOARDTYPE VALUES

VALUE	SHOWS
`default`	Standard keyboard
`email-address`	@ and . keys
`numeric` / `number-pad`	Digits (decimal varies)
`decimal-pad`	Digits + decimal point
`phone-pad`	Phone dial pad
`url`	/ and .com (iOS)

KEYBOARD API

API	PURPOSE
`Keyboard.dismiss()`	Close the keyboard
`Keyboard.addListener('keyboardDidShow')`	Height in `e.endCoordinates`
`maxLength`	Limit characters
`inputMode`	Web-style alternative to `keyboardType`
`editable={false}`	Read-only field
`clearButtonMode`	iOS clear (×) button

⚠ COMMON MISTAKES

- Using `onChange` and reading `e.target.value` ; RN gives `e.nativeEvent.text` , or just use `onChangeText`
- Forgetting `keyboardShouldPersistTaps` , so the first tap on Submit only closes the keyboard
- Leaving `autoCorrect` on for usernames and codes

✓ TIPS & BEST PRACTICES

- Use `react-native-keyboard-controller` for consistent avoidance across iOS and Android
- Wrap a screen in `Pressable onPress={Keyboard.dismiss}` to close on outside taps
- Set `textContentType="oneTimeCode"` for SMS code autofill on iOS

QUICK SUMMARY

- `value` + `onChangeText` — controlled input
- `keyboardType` — right keyboard
- `KeyboardAvoidingView` — keep fields visible
- `returnKeyType` — next / go / done

Pressable & Buttons

HANDLING TAPS

`Pressable` is the modern, flexible way to make anything tappable. It exposes press states (`pressed`, `hovered`, `focused`) so you can style feedback, and fires `onPressIn`, `onPress`, `onLongPress`, `onPressOut`. The built-in `Button` is quick but barely styleable. The old `TouchableOpacity` family still works but `Pressable` is recommended.

PRESS LIFECYCLE

- `onPressIn` → (hold 500ms → `onLongPress`) → `onPressOut`
- `onPress` fires on release if not long-pressed
- `delayLongPress` changes the 500ms default
- `disabled` blocks all handlers

PRESSABLE PATTERNS

```
<Pressable
  onPress={save}
  onLongPress={showMenu}
  disabled={saving}
  hitSlop={8}                                  // bigger touch area
  android_ripple={{ color: '#ffffff33' }}
  accessibilityRole="button"
  accessibilityState={{ disabled: saving }}
  style={({ pressed }) => [
    styles.btn,
    pressed && { opacity: 0.6 },
    saving && styles.btnDisabled,
  ]}>
  {({ pressed }) => (
    <Text style={styles.btnText}>{pressed ? 'Saving…' : 'Save'}</Text>
  )}
</Pressable>

<Button title="Quick action" onPress={doIt} color="#5c6bc0" />
```

NOTES

- → `style` function — receives `{ pressed, hovered }`
- → Children function — change content while pressed
- → `hitSlop` — number or `{ top, bottom, left, right }`
- → `android_ripple` — native ripple; add `borderless` for icons
- → `accessibilityRole="button"` — screen readers announce it

IOS

- No ripple — show feedback with pressed-state opacity
- Pair presses with `expo-haptics` for native feel
- Use `hitSlop` to reach 44 × 44 pt targets

ANDROID

- `android_ripple={{ color, borderless, foreground }}`
- Core `<Button>` renders Material-style, uppercase
- Use `hitSlop` to reach 48 × 48 dp targets

TAPPABLE COMPONENTS

COMPONENT	NOTES
`Pressable`	Recommended; press states, ripple
`Button`	Native look, only `title` and `color`
`TouchableOpacity`	Legacy; fades on press
`TouchableHighlight`	Legacy; underlay color
`TouchableWithoutFeedback`	Legacy; no visual feedback
`Text onPress`	Inline links

USEFUL PROPS

PROP	PURPOSE
`pressRetentionOffset`	How far a finger can drift
`unstable_pressDelay`	Delay `onPressIn` (scroll lists)
`testID`	Hook for tests
`onHoverIn / onHoverOut`	Pointer: iPad, web, desktop
`focusable`	Keyboard / TV focus

⚠ COMMON MISTAKES

- Touch targets under 44×44 pt; use `hitSlop` or padding to meet platform guidelines
- Using `TouchableWithoutFeedback` with no visual cue; users think the tap failed
- Putting `onPress` on a `View`; Views don't handle presses, wrap in `Pressable`

✓ TIPS & BEST PRACTICES

- Build one `<AppButton>` with variants instead of styling every `Pressable`
- Add haptics on important presses (`expo-haptics` `impactAsync`)
- Set `accessibilityLabel` on icon-only buttons

QUICK SUMMARY

- `Pressable` — default tap handler
- `style={({pressed}) => …}` — feedback
- `hitSlop` — larger target
- `Button` — quick, minimal styling

ScrollView

SCROLLING CONTENT

`ScrollView` renders **all** its children at once and lets the user scroll. That's fine for forms, settings, and screens with a bounded amount of content. For long or unknown-length data, use `FlatList`, which renders lazily. A `ScrollView` needs a bounded height (usually `flex: 1` from its parent) to scroll at all.

LAYOUT RULES

- `style` sizes the scroll box; `contentContainerStyle` styles the content
- Use `contentContainerStyle={{ flexGrow: 1 }}` to center short content
- Nested vertical ScrollViews in the same direction fight each other
- Don't nest a `FlatList` inside a vertical `ScrollView`

COMMON CONFIGURATIONS

```
const scrollRef = useRef<ScrollViewInstance>(null)
const [refreshing, setRefreshing] = useState(false)

<ScrollView
  ref={scrollRef}
  style={{ flex: 1 }}
  contentContainerStyle={{ padding: 16, gap: 12 }}
  keyboardShouldPersistTaps="handled"
  keyboardDismissMode="on-drag"
  showsVerticalScrollIndicator={false}
  refreshControl={
    <RefreshControl refreshing={refreshing} onRefresh={reload} />
  }
  onScroll={e => setY(e.nativeEvent.contentOffset.y)}
  scrollEventThrottle={16}>
  {sections}
</ScrollView>

scrollRef.current?.scrollTo({ y: 0, animated: true })
scrollRef.current?.scrollToEnd()

<ScrollView horizontal pagingEnabled
  showsHorizontalScrollIndicator={false}>{slides}</ScrollView>
```

NOTES

- → `keyboardDismissMode` — `none` · `on-drag` · `interactive` (iOS)
- → `RefreshControl` — pull-to-refresh
- → `scrollEventThrottle={16}` — ~60 events/sec on iOS
- → `pagingEnabled` — snap by full width (carousels)
- → `scrollTo` / `scrollToEnd` — imperative scrolling

IOS

- Bounces by default; `bounces={false}` turns it off
- `contentInsetAdjustmentBehavior="automatic"` respects safe areas
- `automaticallyAdjustKeyboardInsets` scrolls inputs into view

ANDROID

- Overscroll stretch / glow; `overScrollMode="never"` disables it
- `nestedScrollEnabled` for scroll views inside scroll views
- `fadingEdgeLength` fades content at the edges

MORE PROPS

PROP	EFFECT
`stickyHeaderIndices`	Pin children by index
`snapToInterval`	Snap every N points
`decelerationRate`	`normal` · `fast` · number
`bounces`	iOS overscroll bounce
`overScrollMode`	Android overscroll glow
`automaticallyAdjustKeyboardInsets`	iOS keyboard inset

SCROLLVIEW OR LIST?

CONTENT	USE
`Form / settings`	`ScrollView`
`Under ~50 small items`	`ScrollView` is fine
`Long feed / unknown length`	`FlatList` / `FlashList`
`Grouped data`	`SectionList`
`Horizontal carousel`	`FlatList horizontal` + `pagingEnabled`

⚠ COMMON MISTAKES

- Putting `alignItems` or `justifyContent` in `style`; content layout goes in `contentContainerStyle`
- Rendering hundreds of items in a `ScrollView`; memory spikes and startup slows
- Passing the boolean `keyboardShouldPersistTaps`; removed in 0.87, use `"handled"` / `"always"` / `"never"`

✓ TIPS & BEST PRACTICES

- Pair `onScroll` animations with Reanimated's `useAnimatedScrollHandler` for smooth 60/120fps
- `contentInsetAdjustmentBehavior="automatic"` handles iOS large titles
- Use `maintainVisibleContentPosition` for chat-style prepends

QUICK SUMMARY

- `ScrollView` — renders everything
- `contentContainerStyle` — inner layout
- `RefreshControl` — pull to refresh
- Long data — use `FlatList`

FlatList & SectionList

VIRTUALIZED LISTS

`FlatList` renders only what's near the viewport and recycles memory as you scroll, so it handles thousands of rows. Give it `data`, a `renderItem` function, and a `keyExtractor`. `SectionList` adds grouped data with headers. Shopify's **FlashList** is a drop-in alternative that recycles views and is usually faster for big feeds.

PERFORMANCE LEVERS

- Keep `renderItem` items light and memoized
- `getItemLayout` skips measurement for fixed heights
- `initialNumToRender`, `windowSize` tune the window
- Avoid inline anonymous objects in item props

INFINITE FEED + SECTIONS

```
<FlatList
  data={posts}
  keyExtractor={item => item.id}
  renderItem={({ item }) => <PostRow post={item} />}
  ItemSeparatorComponent={Divider}
  ListHeaderComponent={<Header />}
  ListEmptyComponent={<Text>No posts yet</Text>}
  ListFooterComponent={loading ? <ActivityIndicator /> : null}
  onEndReached={loadMore}
  onEndReachedThreshold={0.5}
  refreshing={refreshing} onRefresh={refresh}
  getItemLayout={(_, i) => ({ length: 72, offset: 72 * i, index: i })}
/>

<SectionList
  sections={[{ title: 'Today', data: today },
             { title: 'Earlier', data: older }]}
  keyExtractor={item => item.id}
  renderItem={({ item }) => <Row item={item} />}
  renderSectionHeader={({ section }) => <H>{section.title}</H>}
  stickySectionHeadersEnabled
/>
```

NOTES

- → `keyExtractor` — stable string key per item
- → `onEndReachedThreshold={0.5}` — load at half a screen from end
- → `refreshing` + `onRefresh` — built-in pull-to-refresh
- → `getItemLayout` — only for fixed-height rows
- → `sections` — each needs a `data` array

IOS

- Status-bar tap scrolls to top (`scrollsToTop`, one list at a time)
- RefreshControl: `tintColor` and `title`
- ProMotion (120 Hz) makes dropped frames more visible

ANDROID

- `removeClippedSubviews` defaults to true on Android
- RefreshControl: `colors` and `progressBackgroundColor`
- Lower `windowSize` on low-end devices to save memory

FLATLIST PROPS

PROP	PURPOSE
`numColumns`	Grid layout (fixed)
`horizontal`	Side-scrolling list
`inverted`	Chat (newest at bottom)
`extraData`	Re-render when outside state changes
`scrollToIndex()`	Jump via ref
`viewabilityConfig + onViewableItemsChanged`	Track visible rows

FLASHLIST DIFFERENCES

TOPIC	FLASHLIST
`Package`	`@shopify/flash-list`
`Rendering`	Recycles views (no blanking)
`v2`	No `estimatedItemSize` needed
`getItemType`	Separate pools for row types
`Masonry`	`masonry` prop

⚠ COMMON MISTAKES

- Using array index as the key when items reorder or delete; rows show the wrong data
- Forgetting `extraData` when `renderItem` reads external state; rows don't update
- Wrapping a `FlatList` in a vertical `ScrollView`; virtualization is lost and RN warns

✓ TIPS & BEST PRACTICES

- Wrap row components in `memo` (or rely on the React Compiler) and pass primitive props
- Guard `onEndReached` with a loading flag; it can fire several times
- Use `inverted` for chat instead of reversing data and scrolling to end

QUICK SUMMARY

- `FlatList` — virtualized list
- `keyExtractor` / `renderItem` — required pair
- `onEndReached` — infinite scroll
- `SectionList` / FlashList — groups / speed

Modal & Feedback Components

OVERLAYS AND SMALL CONTROLS

`Modal` shows content above the current screen. `Alert.alert` opens a native dialog with buttons. `ActivityIndicator` is the native spinner, `Switch` the native toggle, and `StatusBar` controls the status bar style. With Expo Router you often use a **modal route** instead of `<Modal>`, so it gets a URL and native gestures.

0.87 CHANGES

- `Modal` `animated` prop removed → use `animationType`
- `StatusBar` `backgroundColor` / `translucent` removed (edge-to-edge)
- `Modal` `style` now reaches the inner container (0.86)
- `Alert.prompt` is iOS only

MODAL, ALERT, SPINNER, SWITCH

```
<Modal
  visible={open}
  animationType="slide"                 // none | slide | fade
  presentationStyle="pageSheet"         // iOS card-style sheet
  onRequestClose={() => setOpen(false)} // Android back button
  transparent={false}>
  <SafeAreaView style={{ flex: 1 }}>
    <Button title="Close" onPress={() => setOpen(false)} />
  </SafeAreaView>
</Modal>

Alert.alert('Delete photo?', 'This cannot be undone.', [
  { text: 'Cancel', style: 'cancel' },
  { text: 'Delete', style: 'destructive', onPress: remove },
])

<ActivityIndicator size="large" color="#7e57c2" />
<Switch value={on} onValueChange={setOn}
  trackColor={{ true: '#5c6bc0' }} />
<StatusBar barStyle="light-content" />
```

NOTES

- → `presentationStyle` — `fullScreen` · `pageSheet` · `formSheet` · `overFullScreen`
- → `onRequestClose` — required for Android back
- → `transparent` — draw your own dim backdrop
- → `style: 'destructive'` — red button on iOS
- → `barStyle` — `default` · `light-content` · `dark-content`

IOS

- `presentationStyle="pageSheet"` gives a native card sheet
- `Alert.prompt` and `ActionSheetIOS` are iOS-only
- ActivityIndicator `size` is `'small'` or `'large'` only

ANDROID

- Modal needs `onRequestClose` for the back button
- Alert supports up to 3 buttons; `cancelable` allows outside tap
- `ToastAndroid.show()` for brief messages

ALERT BEHAVIOR

PLATFORM	NOTES
iOS	Unlimited buttons; `cancel` style shown last
Android	Max 3 buttons (neutral, negative, positive)
Android	Tap outside dismisses if `{ cancelable: true }`
Web (RNW)	Uses `window.alert` / `confirm`

OTHER SMALL COMPONENTS

COMPONENT	USE
`ActivityIndicator`	Indeterminate spinner
`Switch`	On/off toggle
`StatusBar`	Style, `hidden`, `animated`
`RefreshControl`	Pull-to-refresh spinner
`expo-status-bar`	`<StatusBar style="auto" />`
`@expo/ui BottomSheet`	Native sheets (SDK 56)

⚠ COMMON MISTAKES

- Omitting `onRequestClose`; the Android back button won't close the modal
- Using `StatusBar backgroundColor` for Android color; removed in 0.87 since apps draw edge-to-edge
- Stacking two RN `Modal`s at once; iOS can only present one at a time from the same parent

✓ TIPS & BEST PRACTICES

- In Expo Router, prefer `presentation: 'modal'` or `'formSheet'` on a route for native sheets
- Use `formSheet` with `sheetAllowedDetents` for half-height sheets
- Keep alerts for confirmations; use toasts or inline errors for info

QUICK SUMMARY

- `Modal` — overlay with `animationType`
- `Alert.alert` — native dialog
- `ActivityIndicator` / `Switch` — native controls
- `StatusBar` — bar style only on Android now

StyleSheet Basics

StyleSheet.create() · style arrays · unitless numbers · no cascade · Text style inheritance · boxShadow / filter

STYLING WITHOUT CSS FILES

Styles are JavaScript objects with **camelCased** properties and **unitless** numbers (density-independent pixels). There's no cascade or selectors: each component gets its own `style` prop. `StyleSheet.create` validates styles and lets you reference them by name. Combine styles with arrays; later entries win. Modern RN supports web-like `boxShadow`, `filter`, and `mixBlendMode`.

DIFFERENCES FROM CSS

- `margin: 8` = 8 dp, no `px`; `'50%'` strings for percent
- Flexbox only; `display: 'none'` hides
- `position: 'absolute' | 'relative' | 'static'`
- No `rem` / `em`; no media queries in StyleSheet

STYLES IN PRACTICE

```
const styles = StyleSheet.create({
  card: {
    padding: 16,
    borderRadius: 12,
    backgroundColor: '#1e2130',
    boxShadow: '0 4px 12px rgba(0,0,0,0.3)',   // 0.76+
    borderWidth: StyleSheet.hairlineWidth,
    borderColor: '#2e3348',
  },
  title: { fontSize: 18, fontWeight: '600', color: '#e8eaf6' },
  muted: { color: '#9aa0b8' },
  selected: { borderColor: '#7e57c2' },
})

<View style={[styles.card, isSelected && styles.selected,
  { marginTop: index === 0 ? 0 : 12 }]}>
  <Text style={[styles.title, disabled && styles.muted]}>Title</Text>
</View>

const flat = StyleSheet.flatten([styles.card, override])
StyleSheet.absoluteFill          // { position:'absolute', inset 0 }
```

NOTES

- → Arrays — falsy entries are ignored
- → `boxShadow` — CSS syntax, both platforms
- → `hairlineWidth` — thinnest line the screen can draw
- → `flatten` — merge into one object (for reading)
- → `absoluteFill` — overlay helper

IOS

- Sizes are points (pt)
- All nine `fontWeight`s work with the system font
- `shadow*` props draw shadows

ANDROID

- Sizes are density-independent pixels (dp)
- Custom fonts need a file per weight
- `textAlignVertical` and `includeFontPadding` are Android-only

COMMON STYLE PROPS

PROP	VALUES
`fontWeight`	`'400'` … `'900'`, `'bold'`
`textAlign`	`left` · `center` · `right` · `justify`
`transform`	`[{ rotate: '45deg' }, { scale: 1.1 }]`
`opacity`	0–1
`overflow`	`visible` · `hidden` · `scroll`
`aspectRatio`	`16 / 9`

NEWER WEB-STYLE PROPS

PROP	SINCE / NOTE
`boxShadow`	0.76; replaces `shadow*` / `elevation`
`filter`	0.76; `brightness`, `blur` (Android) …
`mixBlendMode`	0.77
`outline*`	0.77; focus rings
`display: 'contents'`	0.77
`experimental_backgroundImage`	Gradients

⚠ COMMON MISTAKES

- Writing `'16px'` or `'1rem'`; RN wants numbers (or `'%'` strings)
- Expecting a `View`'s `color` or `fontSize` to reach child `Text`; only nested `Text` inherits
- Building a new style object inline on every render in hot lists; hoist it or use `StyleSheet.create`

✓ TIPS & BEST PRACTICES

- Keep a `theme.ts` with spacing, colors, and radii constants
- Use `gap`, `rowGap`, `columnGap` instead of margins between children
- Try NativeWind or Unistyles if you want utility classes or breakpoints

QUICK SUMMARY

- `StyleSheet.create` — named style objects
- `[a, cond && b]` — compose styles
- Unitless numbers — dp, not px
- `boxShadow` / `filter` — web-like effects

Flexbox in React Native

column default · flex: 1 · justifyContent · alignItems · gap · position: absolute · Yoga differences from web

LAYOUT IS FLEXBOX

Every `View` is a flex container, laid out by the **Yoga** engine. The defaults differ from the web: `flexDirection` is **column**, `alignContent` is `flex-start`, `flexShrink` is `0`, and `flex` accepts a single number. Use `flex: 1` to fill available space, `justifyContent` for the main axis, and `alignItems` for the cross axis.

DEFAULTS VS WEB

- `flexDirection: 'column'` (web: row)
- `flexShrink: 0` (web: 1)
- `position: 'relative'` by default; `static` supported (0.74+)
- `flex: N` = grow N, shrink 1, basis 0 when positive

LAYOUT RECIPES

```
// Full screen, centered
<View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }} />

// Header / scrolling body / footer
<View style={{ flex: 1 }}>
  <Header />                          {/* fixed height */}
  <ScrollView style={{ flex: 1 }} />   {/* takes the rest */}
  <Footer />
</View>

// Row: avatar, text that truncates, badge pinned right
<View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
  <Avatar />
  <Text style={{ flex: 1 }} numberOfLines={1}>{name}</Text>
  <Badge />
</View>

// Wrap chips
<View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 6 }} />

// Floating button
<Pressable style={{ position: 'absolute', right: 16, bottom: 24 }} />
```

NOTES

- → `flex: 1` on body — it fills between header and footer
- → `flex: 1` on `Text` — lets it shrink and truncate
- → `gap` — spacing between children (0.71+)
- → `flexWrap` — chips / tags
- → `absolute` — positioned against the parent

IOS

- Yoga lays out identically on both platforms
- Text metrics differ — check vertical centering on both
- `StyleSheet.hairlineWidth` = 1 physical pixel (≈ 0.33 pt @3x)

ANDROID

- `includeFontPadding` shifts text in centered rows — set false
- `elevation` changes drawing order, overriding `zIndex`
- Check RTL: `I18nManager` flips `start` / `end`

JUSTIFYCONTENT (MAIN AXIS)

VALUE	EFFECT
`flex-start`	Pack at start (default)
`center`	Center
`flex-end`	Pack at end
`space-between`	Edges flush, even gaps
`space-around`	Half gaps at edges
`space-evenly`	Equal gaps everywhere

ALIGNITEMS / ALIGNSELF (CROSS AXIS)

VALUE	EFFECT
`stretch`	Fill cross axis (default)
`flex-start`	Start edge
`center`	Center
`flex-end`	End edge
`baseline`	Align text baselines
`alignSelf`	Override for one child

⚠ COMMON MISTAKES

- Parent without a height (no `flex: 1`) so `flex: 1` children collapse to zero
- Long `Text` in a row pushing siblings off screen; give it `flex: 1` or `flexShrink: 1`
- Copying web CSS assuming `row` default; RN stacks vertically

✓ TIPS & BEST PRACTICES

- Debug layout with a temporary `borderWidth: 1` or the DevTools element inspector
- Use `aspectRatio` for images and cards instead of computing heights
- Use `inset` / `start` / `end` for RTL-friendly positioning

QUICK SUMMARY

- Column by default
- `flex: 1` — fill remaining space
- `justifyContent` / `alignItems` — main / cross axis
- `gap` — spacing between children

Dimensions, Safe Areas & Responsive Layout

FITTING EVERY SCREEN

Phones have notches, Dynamic Islands, home indicators, and (on Android 15+) apps draw **edge-to-edge** under the system bars. Use `react-native-safe-area-context` to pad content away from those areas. Use `useWindowDimensions` (which updates on rotation and resizing) to adapt layouts. Breakpoints are plain JavaScript conditions on the width.

KEY FACTS

- Core `SafeAreaView` is iOS-only and deprecated in favor of the library
- `useWindowDimensions` re-renders on change; `Dimensions.get` doesn't
- Insets differ by device and orientation
- iPads and foldables resize windows at runtime

SAFE AREAS AND BREAKPOINTS

```
// Root: once, near the top of the app
<SafeAreaProvider><App /></SafeAreaProvider>

function Screen() {
  const insets = useSafeAreaInsets()
  const { width, height, fontScale } = useWindowDimensions()
  const isTablet = width >= 768
  const columns = width >= 1024 ? 3 : isTablet ? 2 : 1
  const landscape = width > height

  return (
    <View style={{ flex: 1, paddingTop: insets.top,
                   paddingBottom: insets.bottom }}>
      <FlatList key={columns} numColumns={columns} data={items}
        renderItem={renderCard} />
    </View>
  )
}

// Or the component form, choosing edges
<SafeAreaView edges={['top', 'left', 'right']} style={{ flex: 1 }} />
```

NOTES

- → `SafeAreaProvider` — required once; Expo Router adds it
- → `insets.top` — status bar / notch height
- → `fontScale` — the user's text size setting
- → `key={columns}` — FlatList must remount when `numColumns` changes
- → `edges` — pad only the sides you need

IOS

- Insets cover the notch / Dynamic Island and home indicator
- `screen` and `window` match on iPhone
- iPad Split View and Stage Manager change the window size

ANDROID

- Edge-to-edge: content draws under the system bars
- Pad with `useSafeAreaInsets()`, not hardcoded heights
- Display cutouts (punch-hole cameras) appear in insets

SIZE APIS

API	RETURNS
`useWindowDimensions()`	App window `width`, `height`, `scale`, `fontScale`
`Dimensions.get('screen')`	Whole physical screen
`PixelRatio.get()`	Device pixel density (2, 3 …)
`PixelRatio.roundToNearestPixel(n)`	Crisp hairlines
`onLayout`	A view's own size after layout
`useSafeAreaFrame()`	Frame of the safe area provider

SIMPLE BREAKPOINTS

WIDTH (DP)	TYPICAL DEVICE
`< 380`	Small phones
`380–767`	Phones
`768–1023`	Tablets portrait, foldables open
`≥ 1024`	Tablets landscape, desktop windows

⚠ COMMON MISTAKES

- Reading `Dimensions.get('window')` once at module scope; it never updates on rotation or split view
- Wrapping every screen in `SafeAreaView` and also adding header padding; content gets double insets
- Assuming the window equals the screen on iPad or Android multi-window

✓ TIPS & BEST PRACTICES

- Let headers and tab bars handle their own insets; pad only custom edges
- Test with the largest accessibility text size; use `fontScale` to adjust layouts
- Expect Android apps to draw under the system bars (edge-to-edge is the default on Android 15+); never hardcode bar heights

QUICK SUMMARY

- `useSafeAreaInsets` — notch & bar padding
- `useWindowDimensions` — reactive size
- Breakpoints — JS conditions on width
- `onLayout` — measure a view

Theming & Dark Mode

LIGHT, DARK, AND BRAND THEMES

`useColorScheme()` returns `'light'` or `'dark'` from the OS and re-renders when it changes. Build a theme object per scheme and share it through context (or your navigation theme). `PlatformColor` and `DynamicColorIOS` use native semantic colors that adapt automatically. Libraries like **NativeWind** (Tailwind classes) and **Unistyles** add theme-aware styling with less boilerplate.

SETUP REQUIREMENTS

- Expo: `"userInterfaceStyle": "automatic"` in app.json
- `useColorScheme()` no longer returns `'unspecified'` (0.87)
- `Appearance.setColorScheme('auto')` resets to the OS setting
- Navigation libraries have their own theme prop

THEME HOOK

```tsx
const palette = {
  light: { bg: '#fff', text: '#111827', card: '#f3f4f6', tint: '#5c6bc0' },
  dark:  { bg: '#0f1117', text: '#e8eaf6', card: '#1e2130', tint: '#7e57c2' },
}

export function useTheme() {
  const scheme = useColorScheme() ?? 'light'
  return palette[scheme]
}

function Card({ children }: { children: React.ReactNode }) {
  const t = useTheme()
  return <View style={{ backgroundColor: t.card }}>{children}</View>
}

// Force a scheme (e.g. user setting)
Appearance.setColorScheme('dark')   // 'light' | 'dark' | 'auto'

// Native semantic colors
const label = Platform.select({
  ios: PlatformColor('label'),
  android: PlatformColor('?android:attr/textColorPrimary'),
})
```

NOTES

- → `?? 'light'` — fallback for null on some platforms
- → Keep the palette typed so every key exists in both themes
- → `setColorScheme` — overrides for this app only
- → `PlatformColor` — native named colors, auto light/dark
- → Pass the scheme to React Navigation's `ThemeProvider`

IOS

- `PlatformColor('systemBackground')` follows Dark Mode
- `DynamicColorIOS({ light, dark })` for custom pairs
- `UIUserInterfaceStyle` in Info.plist sets the default

ANDROID

- `PlatformColor('?attr/colorPrimary')` reads theme attributes
- The app theme must be DayNight for dark mode
- Material You colors: `@android:color/system_accent1_500` (API 31+)

STYLING LIBRARIES

LIBRARY	APPROACH
NativeWind	Tailwind `className` → RN styles
Unistyles	StyleSheet API with themes & breakpoints
Tamagui	Component kit + compiler
Restyle (Shopify)	Typed theme props
styled-components	CSS-in-JS for RN

NATIVE COLOR APIS

API	PLATFORM
`PlatformColor('systemBlue')`	iOS semantic colors
`PlatformColor('@android:color/…')`	Android resources
`DynamicColorIOS({ light, dark })`	iOS custom pair
`useMaterialColors()`	Expo UI Material You (SDK 56)

⚠ COMMON MISTAKES

- Hardcoding `'#fff'` across components, which makes dark mode a rewrite later
- Forgetting `userInterfaceStyle: automatic`, so `useColorScheme` always returns `light`
- Theming screens but not the status bar and navigation headers

✓ TIPS & BEST PRACTICES

- Store the user's choice (`light` / `dark` / `system`) and call `Appearance.setColorScheme`
- Check contrast in both themes; dark grays on black often fail WCAG
- Use `expo-system-ui` to set the root background so no white flash appears

QUICK SUMMARY

- `useColorScheme` — OS scheme, reactive
- Theme object — one per scheme
- `PlatformColor` — native semantic colors
- NativeWind / Unistyles — themed styling

Expo Router Basics

FILES BECOME SCREENS

Expo Router maps files in `app/` to routes, like Next.js does on the web. Every screen gets a URL, so deep links and web support come for free. Navigate declaratively with `<Link>` or imperatively with `router`. Dynamic segments use brackets: `app/user/[id].tsx` matches `/user/42`. It's built on native navigators (native stack, tabs) under the hood.

FILE NAMING

- `index.tsx` → the folder's `/` route
- `[id].tsx` → one dynamic segment
- `[...rest].tsx` → catch-all
- `+not-found.tsx` → unmatched URLs; `_layout.tsx` → wrapper

ROUTES AND NAVIGATION

```tsx
// app/index.tsx  →  "/"
import { Link, router } from 'expo-router'

export default function Home() {
  return (
    <View>
      <Link href="/settings">Settings</Link>
      <Link href={{ pathname: '/user/[id]', params: { id: '42' } }}>
        View profile
      </Link>
      <Link href="/about" asChild>
        <Pressable><Text>About</Text></Pressable>
      </Link>
      <Button title="Go" onPress={() => router.push('/user/7?tab=posts')} />
    </View>
  )
}

// app/user/[id].tsx  →  "/user/42"
import { useLocalSearchParams, Stack } from 'expo-router'
export default function User() {
  const { id, tab } = useLocalSearchParams<{ id: string; tab?: string }>()
  return <>
    <Stack.Screen options={{ title: `User ${id}` }} />
    <Text>Tab: {tab ?? 'overview'}</Text>
  </>
}
```

NOTES

- → `href` object — typed params with `pathname`
- → `asChild` — pass navigation to a custom pressable
- → Query strings arrive as params too (`tab`)
- → `<Stack.Screen options>` inside a screen — set its header
- → Typed routes — enable `experiments.typedRoutes`

IOS

- The Stack uses native navigation — swipe-back works by default
- `headerLargeTitle` for collapsing large titles
- `router.back()` matches the swipe gesture

ANDROID

- System back calls `router.back()` automatically
- Default push animation follows the Android version
- Back on the first screen exits the app

ROUTER METHODS

METHOD	EFFECT
`router.push(href)`	Add to history
`router.navigate(href)`	Go to existing route or push
`router.replace(href)`	Replace current screen
`router.back()`	Go back
`router.dismiss() / dismissAll()`	Close modals / stack
`router.setParams({…})`	Update current params

HOOKS & COMPONENTS

API	RETURNS / DOES
`useLocalSearchParams()`	Params for this screen
`useGlobalSearchParams()`	Params of the focused route
`usePathname()`	Current path string
`useSegments()`	Path segments array
`<Redirect href>`	Redirect on render
`useFocusEffect(cb)`	Run when screen focuses

⚠ COMMON MISTAKES

- Forgetting the default export; a route file without one is ignored with a warning
- Using `useGlobalSearchParams` everywhere; it re-renders screens in the background on every change
- Treating params as numbers; they're always strings, so convert them

✓ TIPS & BEST PRACTICES

- Turn on typed routes to catch broken `href`s at compile time
- Keep non-route files (components, utils) outside `app/`
- Use `router.replace` after login so Back doesn't return to the login screen

QUICK SUMMARY

- `app/` file — a route
- `[id]` — dynamic param
- `<Link>` / `router.push` — navigate
- `useLocalSearchParams` — read params

Expo Router Layouts

NAVIGATORS LIVE IN LAYOUTS

A `_layout.tsx` file decides how its folder's routes are presented: a **Stack** (push/pop), **Tabs**, a drawer, or a plain `<Slot />`. Layouts nest, so a root Stack can contain a `(tabs)` group with its own Tabs layout. **Groups** in parentheses organize routes without changing URLs. `Stack.Protected` hides routes behind a condition such as being logged in.

PRESENTATION OPTIONS

- `presentation: 'modal'` — card from bottom (iOS)
- `presentation: 'formSheet'` — native sheet with detents
- `presentation: 'transparentModal'` — overlay
- Native tabs (`NativeTabs`) stable in SDK 58

ROOT STACK, TABS GROUP, AUTH GUARD

```tsx
// app/_layout.tsx
export default function RootLayout() {
  const { user } = useAuth()
  return (
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Protected guard={!!user}>
        <Stack.Screen name="(tabs)" />
        <Stack.Screen name="compose" options={{
          presentation: 'formSheet', sheetAllowedDetents: [0.5, 1],
          headerShown: true, title: 'New post' }} />
      </Stack.Protected>
      <Stack.Protected guard={!user}>
        <Stack.Screen name="login" />
      </Stack.Protected>
    </Stack>
  )
}

// app/(tabs)/_layout.tsx
export default function TabLayout() {
  return (
    <Tabs screenOptions={{ tabBarActiveTintColor: '#7e57c2' }}>
      <Tabs.Screen name="index" options={{ title: 'Home',
        tabBarIcon: ({ color }) => <Icon name="home" color={color} /> }} />
      <Tabs.Screen name="profile" options={{ title: 'Me' }} />
    </Tabs>
  )
}
```

NOTES

- → `name` — the file or folder name (no extension)
- → `Stack.Protected guard` — false hides & redirects
- → `formSheet` + detents — half-height sheet
- → `(tabs)` — URL is `/` , `/profile` , not `/(tabs)/…`
- → `tabBarIcon` — receives `color` , `size` , `focused`

IOS

- `formSheet` uses the native sheet with detents
- Swipe down dismisses modals; `gestureEnabled: false` stops it
- NativeTabs render a native UITabBar

ANDROID

- `formSheet` renders as a Material bottom sheet
- Modals have no swipe-down — back closes them
- NativeTabs use Material bottom navigation

LAYOUT NAVIGATORS

COMPONENT	BEHAVIOR
`<Stack>`	Native stack (push / pop, gestures)
`<Tabs>`	JS bottom tabs
`NativeTabs`	Platform tab bar (UITabBar / Material)
`<Drawer>`	Side drawer (`expo-router/drawer`)
`<Slot>`	Render child, no navigator

COMMON SCREEN OPTIONS

OPTION	EFFECT
`title`	Header / tab title
`headerShown`	Show or hide header
`headerLargeTitle`	iOS large title
`headerRight`	Component in header
`gestureEnabled`	Swipe back on/off
`animation`	`slide_from_right` · `fade` · `none` …

⚠ COMMON MISTAKES

- Naming a `Stack.Screen` that doesn't match a file; the options silently never apply
- Doing auth redirects in `useEffect` before navigation mounts; use `Stack.Protected` or `<Redirect>`
- Putting a `Tabs` layout inside a screen file instead of `_layout.tsx`

✓ TIPS & BEST PRACTICES

- Use groups like `(auth)` and `(app)` to split layouts without URL changes
- Keep the root layout for providers (theme, query client, auth) and the root Stack
- Prefer `formSheet` over a custom bottom-sheet library when native detents suffice

QUICK SUMMARY

- `_layout.tsx` — picks the navigator
- `(group)` — organize without URL
- `presentation` — modal / sheet
- `Stack.Protected` — auth gating

React Navigation

NavigationContainer · createNativeStackNavigator · bottom tabs · drawer · navigation.navigate() · route.params · static config

NAVIGATION WITHOUT FILE ROUTING

React Navigation is the standard library for bare apps or teams that prefer configuring screens in code. Version 7 offers a **static API** (config objects with automatic types and linking) and the classic **dynamic API** (JSX navigators). Native stack uses platform navigation controllers via `react-native-screens`. Version 8 is in pre-release with deeper type inference.

INSTALL

- `@react-navigation/native` + `react-native-screens` + `react-native-safe-area-context`
- Stack: `@react-navigation/native-stack`
- Tabs: `@react-navigation/bottom-tabs`; drawer needs Gesture Handler + Reanimated
- v8 (pre-release) needs Gesture Handler 3 and screens 4.25+

STATIC API (V7)

```tsx
import { createStaticNavigation, useNavigation }
  from '@react-navigation/native'
import { createNativeStackNavigator } from '@react-navigation/native-stack'
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs'

const HomeTabs = createBottomTabNavigator({
  screens: { Feed: FeedScreen, Profile: ProfileScreen },
})

const RootStack = createNativeStackNavigator({
  screens: {
    Home: { screen: HomeTabs, options: { headerShown: false } },
    Details: { screen: DetailsScreen, linking: 'details/:id' },
  },
})

const Navigation = createStaticNavigation(RootStack)
export default () => <Navigation linking={{ prefixes: ['myapp://'] }} />

function FeedScreen() {
  const navigation = useNavigation()
  return <Button title="Open"
    onPress={() => navigation.navigate('Details', { id: '42' })} />
}
function DetailsScreen({ route }) { return <Text>{route.params.id}</Text> }
```

NOTES

- → `createStaticNavigation` — wraps the container for you
- → `linking: 'details/:id'` — deep link path per screen
- → Nested navigators — tabs inside a stack
- → `navigation.navigate(name, params)` — go to a screen
- → Declare `RootParamList` globally for typed `useNavigation`

IOS

- `headerBlurEffect`, `headerLargeTitle`, `headerTransparent`
- `fullScreenGestureEnabled` allows swipe-back from anywhere
- Native stack headers are real UINavigationBars

ANDROID

- Navigators handle the system back button for you
- `animation: 'slide_from_right'` for an iOS-like push
- A drawer is the common Android top-level pattern

NAVIGATION METHODS

METHOD	EFFECT
`navigate(name, params)`	Go (reuse if exists)
`push(name, params)`	Always add a new screen
`goBack()`	Pop
`replace(name)`	Swap current
`popToTop()`	Back to first screen
`setOptions({ title })`	Change header at runtime

EXPO ROUTER VS REACT NAVIGATION

TOPIC	EXPO ROUTER	REACT NAVIGATION
Config	Files in app/	Code objects / JSX
URLs	Automatic	`linking` config
Web	Built in	Supported with setup
Since SDK 56	Own navigation core	Independent library

⚠ COMMON MISTAKES

- Forgetting `react-native-screens` or Gesture Handler peer deps; crashes or no native transitions
- Passing functions or large objects in `params`; params should be serializable IDs
- Using `navigate` when you want a second copy of the same screen; use `push`

✓ TIPS & BEST PRACTICES

- Pass an ID in params and fetch the object on the screen
- Use `useFocusEffect` for refetching when returning to a screen
- Prefer native stack over JS stack for performance and platform feel

QUICK SUMMARY

- `createNativeStackNavigator` — native push/pop
- Static API — config + automatic types
- `navigate` / `push` / `goBack` — moves
- `route.params` — screen input

Sheet 23

Deep Linking & Universal Links

scheme · Linking.openURL() · Linking.getInitialURL() · universal links · Android App Links · expo-linking

OPENING YOUR APP FROM A URL

A **custom scheme** (`myapp://post/42`) opens your app from other apps and notifications. **Universal links** (iOS) and **App Links** (Android) use your real `https://` domain, open the app if installed, and fall back to the website if not. Expo Router and React Navigation route incoming URLs to screens automatically once linking is configured.

REQUIREMENTS

- Scheme set in app config (rebuild needed)
- iOS: `associatedDomains` + `apple-app-site-association` file
- Android: `intentFilters` with `autoVerify` + `assetlinks.json`
- Both JSON files served over HTTPS at `/.well-known/`

CONFIG AND HANDLING

```
// app.json (Expo)
{ "expo": {
    "scheme": "myapp",
    "ios": { "associatedDomains": ["applinks:example.com"] },
    "android": { "intentFilters": [{
      "action": "VIEW", "autoVerify": true,
      "data": [{ "scheme": "https", "host": "example.com",
                 "pathPrefix": "/post" }],
      "category": ["BROWSABLE", "DEFAULT"] }] } } }

// Expo Router: /post/[id].tsx handles both
//   myapp://post/42  and  https://example.com/post/42

// Manual handling (any app)
import * as Linking from 'expo-linking'
const url = Linking.createURL('post/42')   // myapp://post/42 (dev: exp://…)
const initial = await Linking.getInitialURL()
const sub = Linking.addEventListener('url', ({ url }) => {
  const { path, queryParams } = Linking.parse(url)
})
await Linking.openURL('https://example.com')
await Linking.openSettings()               // this app's OS settings
```

NOTES

- → `scheme` — custom URL prefix for your app
- → `applinks:` — iOS domain association
- → `autoVerify` — Android verifies `assetlinks.json`
- → `createURL` — correct scheme in dev and prod
- → `getInitialURL` — URL that cold-started the app

IOS

- Universal Links: `applinks:example.com` in associated domains
- Serve the AASA file without redirects
- Test: `xcrun simctl openurl booted myapp://path`

ANDROID

- App Links: `autoVerify` + SHA-256 of the Play signing key
- Test: `adb shell am start -a android.intent.action.VIEW -d "myapp://path"`
- Check: `adb shell pm get-app-links com.example.app`

TESTING LINKS

PLATFORM	COMMAND
iOS Simulator	`xcrun simctl openurl booted myapp://post/42`
Android	`adb shell am start -W -a android.intent.action.VIEW -d "myapp://post/42"`
Expo	`npx uri-scheme open myapp://post/42 --ios`
Android verify	`adb shell pm verify-app-links --re-verify <pkg>`

DOMAIN FILES

FILE	CONTENTS
apple-app-site-association	appIDs (`TEAMID.bundle.id`) + paths
assetlinks.json	Package name + SHA-256 signing cert
Location	`https://example.com/.well-known/`
Content-Type	`application/json`, no redirects

⚠ COMMON MISTAKES

- Using the debug keystore's SHA-256 in `assetlinks.json` for a Play build; Play App Signing uses a different key
- Changing `scheme` and expecting it to work without a native rebuild
- Serving the AASA file behind a redirect; iOS refuses it

✓ TIPS & BEST PRACTICES

- Add both the Play signing and upload key fingerprints to `assetlinks.json`
- Handle unknown paths with `+not-found` instead of crashing
- Use Expo Router's `+native-intent.tsx` to rewrite third-party URLs before routing

QUICK SUMMARY

- `scheme` — `myapp://` links
- Universal / App Links — `https://` links
- `.well-known/` files — domain proof
- `Linking` — read and open URLs

Platform Module & File Extensions

WRITING PLATFORM-SPECIFIC CODE

Most code is shared, but sometimes iOS and Android need different values, components, or whole files. Use `Platform.OS` and `Platform.select` for small differences inline. For bigger ones, create **platform files**: Metro picks `Button.ios.tsx` on iOS and `Button.android.tsx` on Android when you import `./Button`. Out-of-tree platforms add their own extensions like `.windows.tsx` and `.macos.tsx`.

INLINE AND FILE-BASED

```
import { Platform, StyleSheet } from 'react-native'

// Platform.OS: 'ios' | 'android' | 'web' | 'windows' | 'macos'
const isIOS = Platform.OS === 'ios'

const styles = StyleSheet.create({
  card: {
    ...Platform.select({
      ios:     { shadowColor: '#000', shadowOpacity: 0.2, shadowRadius: 6 },
      android: { elevation: 4 },
      default: { boxShadow: '0 2px 6px rgba(0,0,0,.2)' },
    }),
  },
})

if (Platform.OS === 'android' && Platform.Version >= 33) {
  // Android 13+: request POST_NOTIFICATIONS
}
if (Platform.OS === 'ios' && parseInt(Platform.Version, 10) >= 17) {}

// Files
// DatePicker.ios.tsx      ← iOS
// DatePicker.android.tsx  ← Android
// DatePicker.tsx          ← everything else (web)
import DatePicker from './DatePicker'
```

RESOLUTION ORDER

- `.ios.tsx` / `.android.tsx` / `.web.tsx` first
- then `.native.tsx` (iOS + Android + other native)
- then plain `.tsx`
- Import without the extension: `import Button from './Button'`

NOTES

- → `Platform.select` — `ios`, `android`, `native`, `web`, `default` keys
- → `Platform.Version` — number on Android (API level), string on iOS
- → Android API 33 = Android 13
- → File extensions — Metro chooses at bundle time
- → Types: give platform files the same exported signature

IOS

- `Platform.isPad`, `Platform.isTV`
- `Platform.Version` is a string (`'18.2'`)
- `Platform.constants.interfaceIdiom` — `'phone'`, `'pad'`…

ANDROID

- `Platform.Version` is the API level (`35`)
- `Platform.constants.Brand`, `Model`, `Release`
- `.android.tsx` wins over `.native.tsx`

PLATFORM PROPERTIES

PROPERTY	VALUE
`Platform.OS`	Current platform string
`Platform.Version`	OS version / API level
`Platform.isPad`	iPad (iOS only)
`Platform.isTV`	Apple TV / Android TV
`Platform.isVision`	visionOS build (out-of-tree)
`Platform.constants`	Model, brand, `reactNativeVersion`…

EXTENSIONS

EXTENSION	USED ON
`.ios.tsx`	iOS, iPadOS (and tvOS)
`.android.tsx`	Android
`.native.tsx`	All native platforms
`.web.tsx`	Web (react-native-web)
`.windows.tsx / .macos.tsx`	RN Windows / macOS
`.visionos.tsx`	RN visionOS

⚠ COMMON MISTAKES

- Comparing `Platform.Version` as a number on iOS; it's a string like `'18.2'`
- Platform files with different exports; TypeScript only sees the plain file, so others break at runtime
- Sprinkling `Platform.OS` checks everywhere instead of isolating them in one component

✓ TIPS & BEST PRACTICES

- Keep a shared `Button.types.ts` that each platform file implements
- Prefer capability checks (`await Notifications.getPermissionsAsync()`) over version checks
- Use `.native.tsx` + `.web.tsx` when only web differs

QUICK SUMMARY

- `Platform.OS` — which platform
- `Platform.select` — per-platform values
- `.ios` / `.android` files — split components
- `Platform.Version` — API level / OS string

App Config & Config Plugins

NATIVE CONFIG WITHOUT NATIVE FOLDERS

Expo's **Continuous Native Generation (CNG)** treats `ios/` and `android/` as build output. Your app config (`app.json` or `app.config.ts`) and **config plugins** describe everything native: names, icons, permissions, entitlements, Gradle and Podfile settings. `npx expo prebuild` generates the folders from that config; since SDK 57 it **clears and regenerates** them by default.

WHY CNG

- Upgrades become config changes, not merges into Xcode projects
- Libraries ship plugins that edit native files for you
- Native folders can stay out of git
- Bare apps can still use plugins via `prebuild` once

DYNAMIC CONFIG + PLUGINS

```ts
// app.config.ts
import type { ExpoConfig } from 'expo/config'

const IS_DEV = process.env.APP_VARIANT === 'development'

export default ({ config }: { config: ExpoConfig }): ExpoConfig => ({
  ...config,
  name: IS_DEV ? 'MyApp (Dev)' : 'MyApp',
  slug: 'myapp',
  ios: { bundleIdentifier: IS_DEV ? 'com.acme.app.dev' : 'com.acme.app' },
  android: { package: IS_DEV ? 'com.acme.app.dev' : 'com.acme.app' },
  plugins: [
    'expo-router',
    ['expo-camera', { cameraPermission: 'Scan receipts with your camera' }],
    ['expo-build-properties', {
      android: { minSdkVersion: 24, enableMinifyInReleaseBuilds: true },
      ios: { deploymentTarget: '16.4' },
    }],
    './plugins/withCustomManifest',
  ],
})
```

NOTES

- → `app.config.ts` — JS logic, env vars, variants
- → Different bundle IDs — dev and prod side by side
- → Plugin tuple — `[name, options]`
- → `expo-build-properties` — SDK levels, minify, deployment target
- → Local plugin — your own `withX` function

IOS

- `ios.infoPlist` merges keys into Info.plist
- `ios.entitlements` adds capabilities
- `ios.bundleIdentifier` is permanent once on the App Store

ANDROID

- `android.permissions` adds manifest entries; `blockedPermissions` removes
- `android.package` is permanent once on Google Play
- Plugins use `withAndroidManifest`, `withAppBuildGradle`

PREBUILD COMMANDS

COMMAND	EFFECT
`npx expo prebuild`	Regenerate `ios/` + `android/` (cleans, SDK 57+)
`--no-clean`	Apply to existing folders
`--platform ios`	One platform only
`npx expo config --type public`	Print resolved config
`npx expo-doctor`	Check config & versions

WRITING A PLUGIN

MOD	EDITS
`withInfoPlist`	iOS `Info.plist`
`withEntitlementsPlist`	iOS entitlements
`withAndroidManifest`	`AndroidManifest.xml`
`withAppBuildGradle`	`android/app/build.gradle`
`withPodfile`	`ios/Podfile`
`withDangerousMod`	Any file (last resort)

⚠ COMMON MISTAKES

- Editing `ios/` by hand in a CNG project; the next `prebuild` (which now cleans) throws it away
- Changing a plugin option and only restarting Metro; native config needs a rebuild
- Putting secrets in `app.config.ts` `extra`; it ships inside the app bundle

✓ TIPS & BEST PRACTICES

- Use `APP_VARIANT` with EAS build profiles for dev / preview / prod apps on one device
- Search for a library's config plugin before writing native code yourself
- Run `npx expo prebuild --clean` locally to inspect what EAS will build

QUICK SUMMARY

- CNG — native folders are generated
- `app.config.ts` — dynamic config
- Config plugins — edit native files safely
- `expo prebuild` — regenerate

Cross-Platform Device APIs

HARDWARE THROUGH ONE API

The **Expo SDK** wraps device features in modules that work on iOS and Android (and often web), and you can install them in bare apps too. Most follow the same pattern: check or request permission, then call the API. Permission prompt text comes from your config so the stores accept it. The OS-specific sheets later cover the native details.

PERMISSION PATTERN

- `getPermissionsAsync()` — current status, no prompt
- `requestPermissionsAsync()` — prompt if allowed
- `status`: `granted` · `denied` · `undetermined`
- `canAskAgain: false` → send the user to Settings

LOCATION + CAMERA + HAPTICS

```
import * as Location from 'expo-location'
import * as Haptics from 'expo-haptics'
import { CameraView, useCameraPermissions } from 'expo-camera'

async function getPosition() {
  const { status, canAskAgain } =
    await Location.requestForegroundPermissionsAsync()
  if (status !== 'granted') {
    if (!canAskAgain) Linking.openSettings()
    return null
  }
  const pos = await Location.getCurrentPositionAsync({
    accuracy: Location.Accuracy.Balanced })
  return pos.coords            // { latitude, longitude, … }
}

function Scanner() {
  const [permission, request] = useCameraPermissions()
  if (!permission?.granted)
    return <Button title="Allow camera" onPress={request} />
  return <CameraView style={{ flex: 1 }} facing="back"
    barcodeScannerSettings={{ barcodeTypes: ['qr'] }}
    onBarcodeScanned={({ data }) => {
      Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success)
      open(data)
    }} />
}
```

NOTES

- → `requestForegroundPermissionsAsync` — while app is open
- → `canAskAgain` — false after a hard deny
- → `Accuracy.Balanced` — battery-friendly
- → `useCameraPermissions` — hook form of the pattern
- → Haptics — tactile confirmation on success

IOS

- Every prompt needs an `NS…UsageDescription` string
- Denied once → no re-prompt; use `Linking.openSettings()`
- "Allow Once" location expires when the app closes

ANDROID

- Permissions must also be declared in the manifest
- After two denials the prompt stops showing
- Users choose approximate vs precise location (12+)

COMMON EXPO MODULES

MODULE	PROVIDES
`expo-camera`	Preview, photos, video, barcodes
`expo-location`	Position, watch, geofencing
`expo-notifications`	Local & push notifications
`expo-image-picker`	Photo library / camera picker
`expo-haptics`	Vibration patterns
`expo-sensors`	Accelerometer, gyroscope, pedometer

MORE DEVICE MODULES

MODULE	PROVIDES
`expo-file-system`	Files, downloads, uploads
`expo-contacts / expo-calendar`	Address book / events
`expo-media-library`	Save & read photos
`expo-local-authentication`	Face ID / fingerprint
`expo-audio / expo-video`	Playback & recording
`expo-clipboard`	Copy & paste

⚠ COMMON MISTAKES

- Requesting every permission at launch; users deny and stores may reject; ask in context
- Ignoring `canAskAgain`; calling `request` again shows nothing after a permanent deny
- Forgetting the usage-description text in config; iOS crashes the app when the API is called

✓ TIPS & BEST PRACTICES

- Show a short explainer screen before the OS prompt to raise acceptance
- Use the hooks (`useCameraPermissions`) to re-check after returning from Settings
- Check reactnative.directory for maintained community alternatives when Expo lacks a module

QUICK SUMMARY

- `expo-*` modules — cross-platform hardware
- Check → request → use
- `canAskAgain` — fall back to Settings
- Config text — required prompt strings

Sheet 27

Networking & Data Fetching

fetch() · expo/fetch · AbortController · WebSocket · TanStack Query · NetInfo · cleartext / ATS rules

TALKING TO SERVERS

React Native provides the web-standard `fetch`, `WebSocket`, URL, and `AbortController`. In Expo SDK 56+, `expo/fetch` (with streaming support) is the default global `fetch`. For real apps, wrap requests in **TanStack Query** for caching, retries, background refetch, and offline awareness. Both OSes block plain `http://` by default.

PLATFORM RULES

- iOS **ATS** blocks non-HTTPS unless exempted in Info.plist
- Android 9+ blocks cleartext unless `usesCleartextTraffic`
- `localhost` on an Android emulator is `10.0.2.2`
- No CORS on native; it only applies on web

FETCH + TANSTACK QUERY

```
async function getPosts(signal?: AbortSignal): Promise<Post[]> {
  const res = await fetch('https://api.example.com/posts', {
    headers: { Authorization: `Bearer ${token}` }, signal })
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  return res.json()
}

const queryClient = new QueryClient()
// root: <QueryClientProvider client={queryClient}>…</QueryClientProvider>

function Feed() {
  const { data, isPending, error, refetch, isRefetching } = useQuery({
    queryKey: ['posts'],
    queryFn: ({ signal }) => getPosts(signal),
    staleTime: 60_000,
  })
  const mutation = useMutation({
    mutationFn: (p: NewPost) => api.create(p),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['posts'] }),
  })
  if (isPending) return <ActivityIndicator />
  if (error) return <Text>{error.message}</Text>
  return <FlatList data={data} refreshing={isRefetching} onRefresh={refetch}
    renderItem={({ item }) => <PostRow post={item} />} />
}
```

NOTES

- → `res.ok` — fetch doesn't throw on 4xx/5xx
- → `signal` — cancels when the query is abandoned
- → `staleTime` — how long data counts as fresh
- → `invalidateQueries` — refetch after a mutation
- → Wire `focusManager` to `AppState` for refetch on resume

IOS

- The Simulator shares the Mac's network — localhost works
- A physical iPhone needs your Mac's LAN IP
- Proxies (Charles, Proxyman) need their root cert trusted

ANDROID

- User-installed CAs aren't trusted unless the network config allows them
- `adb reverse tcp:3000 tcp:3000` lets a USB device hit localhost
- fetch runs on OkHttp under the hood

NETWORKING TOOLS

TOOL	USE
`fetch / expo/fetch`	HTTP (streaming in expo/fetch)
`WebSocket`	Realtime connections
`TanStack Query`	Server-state cache
`@react-native-community/netinfo`	Online / offline status
`axios / ky`	Alternative HTTP clients
`expo-file-system upload tasks`	Large uploads/downloads (SDK 56)

REACHING YOUR DEV SERVER

FROM	USE
`iOS Simulator`	`localhost`
`Android Emulator`	`10.0.2.2`
`Physical device`	Computer's LAN IP
`Android + USB`	`adb reverse tcp:3000 tcp:3000`
`Anywhere`	Tunnel (ngrok, Expo tunnel)

⚠ COMMON MISTAKES

- Calling `localhost` from an Android emulator; that's the emulator itself, use `10.0.2.2`
- Assuming `fetch` rejects on 404 or 500; check `res.ok`
- Fetching in `useEffect` without cancellation; responses land after the screen unmounts

✓ TIPS & BEST PRACTICES

- Configure TanStack Query's `onlineManager` with NetInfo for offline pause/resume
- Keep API calls in one module so auth headers and base URLs live in one place
- Inspect traffic in React Native DevTools' Network panel

QUICK SUMMARY

- `fetch` — web-standard HTTP
- TanStack Query — cache, retry, refetch
- ATS / cleartext — HTTPS by default
- `10.0.2.2` — emulator → host

State Management Libraries

Zustand · Redux Toolkit · Jotai · Legend State · persisting state · when context is enough

CLIENT STATE BEYOND USESTATE

Keep **server state** in TanStack Query and **UI state** local. For client state shared across many screens (cart, settings, drafts), use a small store. **Zustand** is the common choice: a hook-based store with selectors and no provider. **Redux Toolkit** suits large teams that want strict patterns and DevTools. **Jotai** uses atoms; **Legend State** focuses on speed and sync.

PICKING ONE

- Few shared values, rare changes → Context
- App-wide client state → Zustand
- Big team, strict conventions → Redux Toolkit
- Fine-grained reactive/offline sync → Legend State

ZUSTAND STORE WITH PERSISTENCE

```ts
import { create } from 'zustand'
import { persist, createJSONStorage } from 'zustand/middleware'
import AsyncStorage from '@react-native-async-storage/async-storage'

type CartState = {
  items: CartItem[]
  add: (item: CartItem) => void
  remove: (id: string) => void
  clear: () => void
}

export const useCart = create<CartState>()(
  persist(
    (set) => ({
      items: [],
      add: (item) => set((s) => ({ items: [...s.items, item] })),
      remove: (id) =>
        set((s) => ({ items: s.items.filter(i => i.id !== id) })),
      clear: () => set({ items: [] }),
    }),
    { name: 'cart', storage: createJSONStorage(() => AsyncStorage) },
  ),
)

// Select only what you need → fewer re-renders
const count = useCart((s) => s.items.length)
const add = useCart((s) => s.add)
useCart.getState().clear()        // outside React
```

NOTES

- → `create<T>()(…)` — curried form for TS + middleware
- → `set(fn)` — immutable updates
- → `persist` — saves to storage, rehydrates on launch
- → Selectors — component re-renders only when slice changes
- → `getState()` — read/act from non-React code

IOS

- iOS kills background apps without warning
- Persist on `AppState` change to `'background'`
- Rehydrate before the first screen renders

ANDROID

- Process death is common on low-RAM devices
- Test with *Don't keep activities*
- AsyncStorage defaults to a 6 MB limit on Android

LIBRARIES COMPARED

LIBRARY	MODEL
Zustand	Hook store, selectors, tiny
Redux Toolkit	Slices, reducers, RTK Query
Jotai	Atoms composed bottom-up
Legend State	Observables, sync & persistence
MobX	Observable classes
XState	State machines for complex flows

WHERE STATE BELONGS

STATE	PUT IT IN
Server data	TanStack Query / RTK Query
Form fields	Component state / React Hook Form
Navigation params	The router
Theme, auth user	Context or store
Cart, drafts	Persisted store
Secrets / tokens	`expo-secure-store`

⚠ COMMON MISTAKES

- Copying server data into a global store and syncing it by hand; let the query cache own it
- Selecting the whole store (`useCart()`) so every change re-renders the component
- Persisting auth tokens with AsyncStorage; it isn't encrypted

✓ TIPS & BEST PRACTICES

- Use `useShallow` from `zustand/react/shallow` when selecting several fields
- Persist with MMKV for synchronous, faster hydration
- Version persisted stores (`version` , `migrate`) so app updates don't break saved state

QUICK SUMMARY

- Server state — query library
- Zustand — simple global store
- Selectors — limit re-renders
- `persist` — survive restarts

Key-Value & Secure Storage

SAVING DATA ON THE DEVICE

There's no `localStorage` on native. **AsyncStorage** is the basic async key-value store. **MMKV** is a much faster synchronous store backed by memory-mapped files. Anything sensitive (tokens, passwords) belongs in **`expo-secure-store`**, which uses the iOS Keychain and Android Keystore-backed encryption. Files and blobs go through `expo-file-system`.

LIMITS AND BEHAVIOR

- AsyncStorage and MMKV store strings; JSON-encode objects
- SecureStore values ~2 KB; meant for small secrets
- Data is deleted when the app is uninstalled (Keychain may persist on iOS)
- All of these are unencrypted except SecureStore (MMKV can encrypt)

THREE STORES SIDE BY SIDE

```
// AsyncStorage — async, simple
import AsyncStorage from '@react-native-async-storage/async-storage'
await AsyncStorage.setItem('settings', JSON.stringify({ units: 'metric' }))
const raw = await AsyncStorage.getItem('settings')   // string | null
const settings = raw ? JSON.parse(raw) : defaults
await AsyncStorage.removeItem('settings')

// MMKV — synchronous, fast
import { createMMKV } from 'react-native-mmkv'
const storage = createMMKV({ id: 'app' })
storage.set('onboarded', true)
storage.getBoolean('onboarded')                    // true
storage.set('user', JSON.stringify(user))
const [name, setName] = useMMKVString('name')      // reactive hook

// SecureStore — Keychain / Keystore
import * as SecureStore from 'expo-secure-store'
await SecureStore.setItemAsync('token', jwt, {
  keychainAccessible: SecureStore.AFTER_FIRST_UNLOCK })
const token = await SecureStore.getItemAsync('token')
await SecureStore.deleteItemAsync('token')
```

NOTES

- → `JSON.stringify` / `parse` — store objects as strings
- → `createMMKV` — MMKV v4 API (`new MMKV` in v2/v3)
- → `getBoolean` / `getNumber` — typed MMKV getters
- → `useMMKVString` — re-renders when the key changes
- → `keychainAccessible` — when iOS can read the item

IOS

- SecureStore uses the Keychain — items can survive uninstall
- `keychainAccessible` controls access while locked
- Documents files are backed up to iCloud by default

ANDROID

- SecureStore uses Keystore-encrypted storage
- Auto Backup can restore app data after reinstall
- Set `allowBackup` false or exclude secrets from backup

STORAGE OPTIONS

STORE	SYNC?	ENCRYPTED
AsyncStorage	Async	✖
MMKV	Sync	Optional key
SecureStore	Async	✔
expo-sqlite / kv-store	Both	✖ (SQLCipher option)
File system	Async	✖

WHAT GOES WHERE

DATA	STORE
Auth / refresh tokens	SecureStore
Feature flags, UI prefs	MMKV / AsyncStorage
Persisted Zustand store	MMKV / AsyncStorage
Images, downloads	`expo-file-system`
Large structured data	SQLite

⚠ COMMON MISTAKES

- Storing tokens in AsyncStorage; a rooted or backed-up device exposes them in plain text
- Forgetting `await` on AsyncStorage calls and reading before the write finishes
- Putting large blobs in SecureStore; it has size limits and is slow

✓ TIPS & BEST PRACTICES

- Wrap storage access in a small typed module so keys aren't magic strings
- Clear SecureStore on first launch after reinstall if you don't want iOS Keychain items to survive
- Use `expo-sqlite/kv-store` as a drop-in AsyncStorage API backed by SQLite

QUICK SUMMARY

- AsyncStorage — simple async KV
- MMKV — fast sync KV
- SecureStore — secrets only
- JSON — serialize objects

Local Databases & Offline

STRUCTURED DATA ON THE DEVICE

For relational data, search, or offline-first apps, use **SQLite**. `expo-sqlite` offers async and sync APIs, prepared statements, and a React hook for live queries. `op-sqlite` is a fast JSI alternative. Layer **Drizzle ORM** on top for typed queries and migrations, or use **WatermelonDB** / **Legend State** / **PowerSync** when you need sync with a backend.

OFFLINE-FIRST BASICS

- Read from the local DB; sync in the background
- Queue writes while offline, replay when online
- Track `updated_at` / version columns for conflicts
- Run migrations on launch before rendering data

EXPO-SQLITE + DRIZZLE

```
import { SQLiteProvider, useSQLiteContext } from 'expo-sqlite'

<SQLiteProvider databaseName="app.db" onInit={migrate}>
  <App />
</SQLiteProvider>

async function migrate(db: SQLiteDatabase) {
  await db.execAsync(`
    PRAGMA journal_mode = WAL;
    CREATE TABLE IF NOT EXISTS todos (
      id INTEGER PRIMARY KEY NOT NULL,
      title TEXT NOT NULL, done INTEGER DEFAULT 0);`)
}

function Todos() {
  const db = useSQLiteContext()
  const add = (t: string) =>
    db.runAsync('INSERT INTO todos (title) VALUES (?)', t)
  const load = () =>
    db.getAllAsync<Todo>('SELECT * FROM todos WHERE done = ?', 0)
}

// Drizzle on top
import { drizzle } from 'drizzle-orm/expo-sqlite'
const orm = drizzle(db)
const open = await orm.select().from(todos).where(eq(todos.done, false))
```

NOTES

- → `SQLiteProvider` — opens the DB once for the tree
- → `onInit` — run schema setup / migrations
- → `WAL` — faster concurrent reads and writes
- → `?` placeholders — never build SQL with string concat
- → Drizzle — typed schema, `useLiveQuery`, migrations

IOS

- The DB file lives in the app sandbox
- Find it: `xcrun simctl get_app_container booted <id> data`
- Included in iCloud backups unless excluded

ANDROID

- The DB lives under the app's private `files/` dir
- Pull it: `adb exec-out run-as <pkg> cat <path>`
- Auto Backup may restore an older DB — migrations must cope

EXPO-SQLITE METHODS

METHOD	RETURNS
`execAsync(sql)`	Run statements, no results
`runAsync(sql, …params)`	`{ lastInsertRowId, changes }`
`getFirstAsync<T>()`	First row or null
`getAllAsync<T>()`	All rows
`getEachAsync<T>()`	Async iterator
`withTransactionAsync(fn)`	Transaction

OPTIONS

LIBRARY	STRENGTH
`expo-sqlite`	Built into Expo, sync & async APIs
`op-sqlite`	JSI speed, SQLCipher, vector ext.
`Drizzle ORM`	Typed queries, migrations
`WatermelonDB`	Lazy loading, sync protocol
`PowerSync / ElectricSQL`	Postgres ↔ SQLite sync
`Realm (Atlas Device SDK)`	Deprecated; plan a migration

⚠ COMMON MISTAKES

- Building queries with template strings from user input; SQL injection works on-device too
- Running schema changes without a version check; reinstalls work but updates crash
- Doing large reads on the JS thread in a loop instead of one query with `WHERE` / `LIMIT`

✓ TIPS & BEST PRACTICES

- Store a `PRAGMA user_version` and migrate step by step
- Use transactions for batch inserts; they're dramatically faster
- Inspect the DB with the Expo DevTools plugin or by pulling the file from the simulator

QUICK SUMMARY

- SQLite — structured local data
- `runAsync` / `getAllAsync` — write / read
- Drizzle — typed ORM layer
- Sync libraries — offline-first apps

Animated API

BUILT-IN ANIMATIONS

The core **Animated** API drives values over time and binds them to `Animated.View`, `Animated.Text`, and friends. With `useNativeDriver: true`, the animation runs on the UI thread, so JS work can't make it stutter, but only transform and opacity can be animated that way. `LayoutAnimation` animates the next layout change automatically. For gestures and complex motion, use Reanimated.

NATIVE DRIVER LIMITS

- Supported: `transform`, `opacity`
- Not supported: `width`, `height`, colors, `top`
- Can't switch drivers on the same value
- Gesture-linked animation → Reanimated

FADE, SPRING, INTERPOLATE

```
function FadeInCard({ children }: { children: React.ReactNode }) {
  const opacity = useRef(new Animated.Value(0)).current
  const scale = useRef(new Animated.Value(0.9)).current

  useEffect(() => {
    Animated.parallel([
      Animated.timing(opacity, { toValue: 1, duration: 300,
        easing: Easing.out(Easing.cubic), useNativeDriver: true }),
      Animated.spring(scale, { toValue: 1, friction: 6,
        useNativeDriver: true }),
    ]).start()
  }, [])

  const translateY = opacity.interpolate({
    inputRange: [0, 1], outputRange: [20, 0] })

  return (
    <Animated.View style={{ opacity,
      transform: [{ scale }, { translateY }] }}>
      {children}
    </Animated.View>
  )
}

// Animate the next layout change (expand / collapse)
LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut)
setExpanded(e => !e)
```

NOTES

- → `useRef(...).current` — keep the same value across renders
- → `parallel` — run animations together
- → `useNativeDriver: true` — UI-thread playback
- → `interpolate` — map one range to another
- → `LayoutAnimation` — call right before the state change

IOS

- `shouldRasterizeIOS` caches complex views while animating
- Respect Reduce Motion: `AccessibilityInfo.isReduceMotionEnabled()`
- ProMotion runs at 120 Hz — use durations, not frame counts

ANDROID

- `renderToHardwareTextureAndroid` during opacity / transform
- Low-end devices drop frames first — test on one
- Judge smoothness in release; debug is much slower

ANIMATED FUNCTIONS

FUNCTION	USE
`timing`	Duration + easing
`spring`	Physics (`friction` / `tension` or `stiffness`)
`decay`	Slow to a stop (flings)
`sequence / parallel / stagger`	Compose
`loop`	Repeat
`Animated.event`	Map scroll/gesture events to values

ANIMATED OR REANIMATED?

NEED	USE
`Simple fade / slide`	Animated (native driver)
`Width, height, color`	Reanimated
`Gesture-driven`	Reanimated + Gesture Handler
`Enter / exit / layout`	Reanimated layout animations
`Expand/collapse once`	`LayoutAnimation`

⚠ COMMON MISTAKES

- Creating `new Animated.Value()` in render without `useRef`; it resets every render
- Animating `height` with the native driver; it throws, use Reanimated or `scaleY`
- Forgetting `useNativeDriver`; animations stutter whenever JS is busy

✓ TIPS & BEST PRACTICES

- Use `Animated.event` with `useNativeDriver: true` for scroll-linked headers
- Respect `AccessibilityInfo.isReduceMotionEnabled()` and shorten or skip motion
- Move to Reanimated as soon as animations depend on gestures or layout

QUICK SUMMARY

- `Animated.Value` — animated number
- `timing` / `spring` — drivers
- `useNativeDriver` — UI thread, transform/opacity
- `LayoutAnimation` — auto layout transitions

Reanimated

HIGH-PERFORMANCE ANIMATION

Reanimated runs animation logic on the UI thread as **worklets**, small JS functions executed outside the main JS thread via `react-native-worklets`. You store animated state in **shared values**, derive styles with `useAnimatedStyle`, and animate with `withTiming` / `withSpring`. Reanimated 4 (New Architecture only) adds **CSS-style transitions and keyframe animations** and declarative layout animations.

SETUP (V4)

- Install `react-native-reanimated` + `react-native-worklets`
- Babel plugin: `react-native-worklets/plugin` (Expo adds it)
- Requires the New Architecture
- Expo SDK 57 ships Reanimated 4.5

SHARED VALUES, LAYOUT, CSS

```
import Animated, { useSharedValue, useAnimatedStyle, withSpring,
  withTiming, FadeIn, FadeOut, LinearTransition }
  from 'react-native-reanimated'

function Toggle() {
  const x = useSharedValue(0)
  const style = useAnimatedStyle(() => ({
    transform: [{ translateX: x.value }],
    backgroundColor: x.value > 20 ? '#7e57c2' : '#444',
  }))
  return (
    <Pressable onPress={() => {
      x.value = withSpring(x.value === 0 ? 40 : 0, { damping: 15 }) }}>
      <Animated.View style={[styles.knob, style]} />
    </Pressable>
  )
}

// Layout animations: mount, unmount, reorder
<Animated.View entering={FadeIn.duration(250)} exiting={FadeOut}
  layout={LinearTransition.springify()} />

// CSS transitions (v4): animate when style changes
<Animated.View style={{ width: open ? 240 : 80,
  transitionProperty: 'width', transitionDuration: 300 }} />
```

NOTES

- → `x.value` — read/write shared values via `.value`
- → `useAnimatedStyle` — worklet; re-runs on the UI thread
- → `withSpring` / `withTiming` — assign to animate
- → `entering` / `exiting` — mount/unmount animations
- → `transitionProperty` — CSS-like transitions in v4

IOS

- ProMotion runs at 120 Hz — animate by time, not frames
- `reduceMotion: ReduceMotion.System` honours Reduce Motion
- Profile with Xcode Instruments in a release build

ANDROID

- Debug builds are much slower — test in release
- `ReduceMotion.System` honours *Remove animations*
- Profile with Android Studio Profiler or Perfetto

CORE APIS

API	PURPOSE
`useSharedValue(v)`	Animated state
`useAnimatedStyle(fn)`	Style from shared values
`useDerivedValue(fn)`	Computed shared value
`withTiming / withSpring / withDecay`	Animate
`withSequence / withRepeat / withDelay`	Compose
`useAnimatedScrollHandler`	Scroll-linked effects

CROSSING THREADS

API	DIRECTION
`scheduleOnRN(fn, …args)`	UI → JS (v4; was `runOnJS`)
`scheduleOnUI(fn)`	JS → UI (was `runOnUI`)
`useAnimatedReaction`	React to shared value changes
`interpolate / interpolateColor`	Map ranges in worklets

⚠ COMMON MISTAKES

- Reading `x.value` during render; it doesn't trigger re-renders, so use it in worklets only
- Calling a normal JS function (setState, navigation) inside a worklet without `scheduleOnRN`
- Forgetting the Babel plugin in a bare app; worklets crash at runtime

✓ TIPS & BEST PRACTICES

- Use CSS transitions for simple state-driven changes; keep shared values for gestures
- Combine with Gesture Handler for drag, swipe, and pinch interactions
- Profile memory on Android with Hermes V1; an SDK 56 regression was fixed in SDK 57

QUICK SUMMARY

- `useSharedValue` — UI-thread state
- `useAnimatedStyle` — derived styles
- `entering` / `layout` — layout animations
- CSS transitions — v4 declarative motion

Gesture Handler

GestureHandlerRootView · GestureDetector · usePanGesture · useTapGesture · usePinchGesture · useSimultaneousGestures · Reanimated integration

NATIVE GESTURES

`react-native-gesture-handler` recognizes touches on the native side, so pans, pinches, and swipes stay responsive even when JS is busy. **Version 3** replaces the `Gesture.Pan()` builder with **hooks** (`usePanGesture`, `useTapGesture`…) that take a config object. Attach a gesture with `GestureDetector`; with Reanimated installed, callbacks run on the UI thread and can update shared values directly.

VERSIONS

- v3 (latest, 3.3): hook API; `Gesture.*` builders deprecated
- v2 (`legacy` tag, 2.33) still ships with Expo SDK 57
- React Navigation 8 requires v3
- Expo Router / React Navigation set up the root view

DRAGGABLE, PINCHABLE CARD (V3)

```tsx
import { GestureDetector, usePanGesture, usePinchGesture,
  useSimultaneousGestures } from 'react-native-gesture-handler'
import Animated, { useSharedValue, useAnimatedStyle, withSpring }
  from 'react-native-reanimated'

function DraggableCard() {
  const x = useSharedValue(0), y = useSharedValue(0)
  const scale = useSharedValue(1)

  const pan = usePanGesture({
    onUpdate: e => { x.value += e.changeX; y.value += e.changeY },
    onDeactivate: () => {
      x.value = withSpring(0); y.value = withSpring(0) },
  })
  const pinch = usePinchGesture({
    onUpdate: e => { scale.value = e.scale },
    onDeactivate: () => { scale.value = withSpring(1) },
  })
  const both = useSimultaneousGestures(pan, pinch)

  const style = useAnimatedStyle(() => ({ transform: [
    { translateX: x.value }, { translateY: y.value },
    { scale: scale.value }] }))

  return (
    <GestureDetector gesture={both}>
      <Animated.View style={[styles.card, style]} />
    </GestureDetector>
  )
}
// Root: wrap the app in <GestureHandlerRootView style={{ flex: 1 }}>
// v2 equivalent: Gesture.Pan().onChange(…).onEnd(…)
```

NOTES

- → Config object — callbacks + options in one place
- → `changeX` — delta since the last event
- → `onDeactivate` — gesture ended or was cancelled
- → `useSimultaneousGestures` — both run together
- → `runOnJS: true` — run callbacks on the JS thread instead

IOS

- Native-stack edge swipe can steal horizontal pans
- Gestures map to native UIGestureRecognizers
- Pair gesture ends with haptics for native feel

ANDROID

- Wrap Modal content in `GestureHandlerRootView`
- System back gesture claims swipes near the screen edges
- Keep draggable areas away from the edges

GESTURE HOOKS

HOOK	RECOGNIZES
`useTapGesture`	Taps (`numberOfTaps`)
`useLongPressGesture`	Hold (`minDuration`)
`usePanGesture`	Drag (`activeOffsetX`, `failOffsetY`)
`usePinchGesture`	Two-finger scale
`useRotationGesture`	Two-finger rotate
`useFlingGesture`	Quick swipe (`direction`)

COMPOSITION & CALLBACKS

API	BEHAVIOR
`useSimultaneousGestures(a, b)`	Both active together
`useExclusiveGestures(a, b)`	First in order wins (double vs single tap)
`useCompetingGestures(a, b)`	First to activate cancels others
`simultaneousWith / requireToFail`	Relations in config
`onBegin → onActivate → onUpdate`	Lifecycle start
`onDeactivate → onFinalize`	Lifecycle end

⚠ COMMON MISTAKES

- Missing `GestureHandlerRootView`; gestures silently never fire (especially inside RN `Modal`)
- Mixing v2 tutorials (`Gesture.Pan().onEnd`) into a v3 codebase; the builder is deprecated
- Setting React state on every pan event; update shared values and call `scheduleOnRN` only when done

✓ TIPS & BEST PRACTICES

- Use `activeOffsetX` / `failOffsetY` so horizontal swipes don't fight a vertical `ScrollView`
- Use `useExclusiveGestures(doubleTap, singleTap)` for like-on-double-tap
- Use `ReanimatedSwipeable` for swipe-to-delete rows and the v3 `Touchable` for buttons

QUICK SUMMARY

- `use*Gesture` — v3 hook API
- `GestureDetector` — attach a gesture
- `useSimultaneousGestures` — combine
- Shared values — smooth UI-thread updates

Turbo Native Modules

CALLING NATIVE CODE

When no library exists, write a **Turbo Native Module**. You describe its API in a TypeScript **spec**; **Codegen** generates the C++/Objective-C++/Java glue at build time; then you implement the methods natively. TurboModules load lazily and are called through **JSI**, so synchronous methods are possible. Since 0.87 TurboModules are always enabled.

WORKFLOW

SPEC AND USAGE

```ts
// specs/NativeLocalStorage.ts
import type { TurboModule } from 'react-native'
import { TurboModuleRegistry } from 'react-native'

export interface Spec extends TurboModule {
  setItem(value: string, key: string): void
  getItem(key: string): string | null           // synchronous
  clear(): Promise<void>                         // async
}

export default TurboModuleRegistry.getEnforcing<Spec>('NativeLocalStorage')

// package.json
"codegenConfig": {
  "name": "AppSpecs", "type": "modules", "jsSrcsDir": "specs",
  "android": { "javaPackageName": "com.myapp.specs" }
}

// App code
import NativeLocalStorage from './specs/NativeLocalStorage'
NativeLocalStorage.setItem('hello', 'greeting')
NativeLocalStorage.getItem('greeting')          // 'hello'
```

NOTES

- → File must be named `Native*.ts`
- → `getEnforcing` — throws if the module isn't linked
- → Return types define sync vs `Promise`
- → `codegenConfig.name` — prefix for generated classes
- → Kotlin class extends generated `NativeLocalStorageSpec`

IOS

- › Implement in Objective-C++ (`.mm`) against the generated protocol
- › Swift needs an Objective-C++ wrapper
- › Codegen runs during `pod install`

ANDROID

- › Implement in Kotlin / Java extending the generated `Native…Spec`
- › Register it in a `BaseReactPackage`
- › Codegen runs during the Gradle build

SPEC TYPE MAPPING

TYPESCRIPT	NATIVE
`string`	NSString / String
`number`	double / Double
`boolean`	BOOL / Boolean
`Object / typed object`	NSDictionary / ReadableMap
`Array<T>`	NSArray / ReadableArray
`Promise<T>`	resolve / reject blocks / Promise

IMPLEMENTATION FILES

PLATFORM	YOU WRITE
Android	`NativeLocalStorageModule.kt` + `Package` (TurboReactPackage)
Android	Register package in `MainApplication`
iOS	`RCTNativeLocalStorage.h/.mm` (or Swift + ObjC++ shim)
iOS	Map name in `package.json` `ios.modulesProvider`
Both	Run Codegen: `pod install` / `./gradlew generateCodegenArtifactsFromSchema`

⚠ COMMON MISTAKES

- › Changing the spec without re-running Codegen; native signatures and JS no longer match
- › Doing slow work in a synchronous method; it blocks the JS thread
- › Naming the spec file without the `Native` prefix, so Codegen skips it

✓ TIPS & BEST PRACTICES

- › Use the Expo Modules API (sheet 36) for Swift/Kotlin modules with less boilerplate
- › Scaffold libraries with `npx create-react-native-library@latest`
- › Keep the spec small and stable; it's your contract between JS and native

QUICK SUMMARY

- › Spec — TypeScript interface
- › Codegen — generates glue
- › `TurboModuleRegistry` — get the module
- › Kotlin + ObjC++/Swift — implementations

Fabric Native Components

WRAPPING NATIVE VIEWS

A **Fabric Native Component** exposes a native view (a map, a video player, a custom chart) as a React component. Like modules, you write a TypeScript spec of its **props**, **events**, and **commands**, run Codegen, and implement the view in Kotlin and Objective-C++/Swift. The **interop layer** lets many old (Paper) components work without rewriting, but new code should use Fabric directly.

SPEC TYPES

- `ViewProps` — base props (style, layout)
- `DirectEventHandler<T>` — event to one component
- `BubblingEventHandler<T>` — bubbles up the tree
- `WithDefault<T, v>` — default prop values

COMPONENT SPEC

```ts
// specs/WebViewNativeComponent.ts
import type { CodegenTypes, HostComponent, ViewProps } from 'react-native'
import { codegenNativeComponent, codegenNativeCommands } from 'react-native'

type LoadEvent = Readonly<{ result: 'success' | 'error' }>

export interface NativeProps extends ViewProps {
  sourceURL?: string
  zoomEnabled?: CodegenTypes.WithDefault<boolean, true>
  onScriptLoaded?: CodegenTypes.BubblingEventHandler<LoadEvent> | null
}

export interface NativeCommands {
  reload: (viewRef: React.ElementRef<HostComponent<NativeProps>>) => void
}
export const Commands = codegenNativeCommands<NativeCommands>({
  supportedCommands: ['reload'] })

export default codegenNativeComponent<NativeProps>('CustomWebView')

// Usage
<CustomWebView ref={ref} style={{ flex: 1 }}
  sourceURL="https://example.com"
  onScriptLoaded={e => log(e.nativeEvent)} />
Commands.reload(ref.current!)
```

NOTES

- → `ViewProps` — gives `style`, accessibility, layout
- → `WithDefault` — native default if prop omitted
- → `BubblingEventHandler` — read data on `e.nativeEvent`
- → Commands — imperative calls like `reload()`
- → `codegenConfig.type` — `"components"` or `"all"`

IOS

- Subclass `RCTViewComponentView` in Objective-C++
- Props arrive in `updateProps:oldProps:`
- Wrap Swift views from the Objective-C++ class

ANDROID

- `SimpleViewManager<T>` + the generated `…ManagerInterface`
- Props are set via `@ReactProp` setters
- Register in the package's `createViewManagers`

NATIVE SIDE

PLATFORM	IMPLEMENT
Android	`ViewManager` (`SimpleViewManager<T>`) + `@ReactProp`
Android	Events via `UIManagerHelper` dispatcher
iOS	`RCTViewComponentView` subclass (`.mm`)
iOS	`updateProps:oldProps:` to apply props
Both	Register in `codegenConfig` / component provider

APPROACHES

APPROACH	WHEN
`Fabric component`	Full control, best performance
`Expo Modules View`	Swift/Kotlin view with less glue
`Interop layer`	Use an old Paper component as-is
`react-native-webview etc.`	Library already exists

⚠ COMMON MISTAKES

- Passing large data through props on every render; send it via a command or module instead
- Expecting `setNativeProps`-style mutations; Fabric views update through props
- Forgetting to mark optional event props `| null`, which breaks Codegen

✓ TIPS & BEST PRACTICES

- Read existing open-source Fabric components (e.g. `react-native-screens`) as references
- Use Expo Modules' `View` when you'd rather write Swift and Kotlin only
- Keep native views dumb; put logic in JS where it's testable

QUICK SUMMARY

- `codegenNativeComponent` — declare view
- Props / events / commands — the spec
- ViewManager / ComponentView — native impl
- Interop layer — legacy views still run

Expo Modules API

NATIVE CODE IN SWIFT AND KOTLIN

The **Expo Modules API** lets you write native modules and views in modern **Swift** and **Kotlin** with a small declarative DSL. It handles type conversion, threading, and registration, and works in any React Native app (Expo or bare). It's built on JSI and the New Architecture, and most `expo-*` packages are written with it.

GETTING STARTED

- `npx create-expo-module@latest my-module` (library)
- `npx create-expo-module@latest --local` (inside an app)
- Autolinked; rebuild the dev client after changes
- Expo Modules 2.0 is in beta with SDK 58

ONE MODULE, BOTH PLATFORMS

```swift
// ios/BatteryModule.swift
import ExpoModulesCore

public class BatteryModule: Module {
  public func definition() -> ModuleDefinition {
    Name("Battery")
    Function("getLevel") { () -> Float in
      UIDevice.current.isBatteryMonitoringEnabled = true
      return UIDevice.current.batteryLevel
    }
    AsyncFunction("fetchReport") {
      (days: Int) async throws -> [String: Any] in
      try await BatteryReport.load(days: days)
    }
    Events("onLowBattery")
  }
}
```

IOS

- Swift `Module` with `definition()`
- `ExpoAppDelegateSubscriber` hooks AppDelegate events
- Run `pod install` (or prebuild) after adding

ANDROID

- Kotlin `Module` with the same DSL
- `ReactActivityLifecycleListener` hooks Activity events
- Rebuild with `npx expo run:android`

KOTLIN + JS

```kotlin
// android/.../BatteryModule.kt
class BatteryModule : Module() {
  override fun definition() = ModuleDefinition {
    Name("Battery")
    Function("getLevel") {
      val bm = appContext.reactContext!!
        .getSystemService(Context.BATTERY_SERVICE) as BatteryManager
      bm.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPACITY) / 100f
    }
  }
}

// JS
import { requireNativeModule } from 'expo'
const Battery = requireNativeModule('Battery')
Battery.getLevel()                        // 0.82
```

DEFINITION DSL

COMPONENT	PURPOSE
`Name("X")`	JS-visible module name
`Function`	Synchronous call
`AsyncFunction`	Returns a Promise
`Constant / Property`	Values / getters
`Events("a") + sendEvent`	Native → JS events
`View(MyView.self) + Prop`	Native view component

LIFECYCLE & EXTRAS

API	USE
`OnCreate / OnDestroy`	Module setup / teardown
`OnStartObserving`	First JS listener added
`Record`	Typed struct arguments
`SharedObject`	Native object with JS handle

⚠ COMMON MISTAKES

- Blocking the JS thread with heavy work in `Function`; use `AsyncFunction`
- Forgetting to rebuild after editing Swift/Kotlin; Metro reloads only JS
- Mismatched module names between platforms; `requireNativeModule` fails on one OS

✓ TIPS & BEST PRACTICES

- Use `Record` classes instead of loose dictionaries for typed arguments
- Add a TypeScript wrapper exporting typed functions over `requireNativeModule`
- Prefer Expo Modules over raw TurboModules unless you need C++ or sync JSI tricks

QUICK SUMMARY

- `create-expo-module` — scaffold
- `ModuleDefinition` — declarative DSL
- `Function` / `AsyncFunction` / `Events`
- `requireNativeModule` — use from JS

Unit & Component Testing

Jest · jest-expo preset · React Native Testing Library · render / screen · fireEvent / userEvent · mocking native modules

TESTING COMPONENTS IN NODE

Unit and component tests run in **Jest** on Node, with no simulator. **React Native Testing Library** (RNTL) renders components to a test tree and queries them the way users perceive them: by text, role, and label. Native modules don't exist in Node, so you mock them; the `jest-expo` preset mocks the Expo SDK for you.

SETUP

- Expo: `npx expo install jest-expo jest @testing-library/react-native`
- `"jest": { "preset": "jest-expo" }` in package.json
- Bare: `preset: 'react-native'` (ships with the template)
- RNTL 13+ includes the Jest matchers

A COMPONENT TEST

```
import { render, screen, userEvent }
  from '@testing-library/react-native'
import { LoginForm } from './LoginForm'

jest.mock('expo-secure-store', () => ({ setItemAsync: jest.fn() }))

test('shows an error for a bad password', async () => {
  const onLogin = jest.fn().mockRejectedValue(new Error('Invalid'))
  const user = userEvent.setup()
  render(<LoginForm onLogin={onLogin} />)

  await user.type(screen.getByLabelText('Email'), 'a@b.com')
  await user.type(screen.getByLabelText('Password'), 'nope')
  await user.press(screen.getByRole('button', { name: 'Sign in' }))

  expect(onLogin).toHaveBeenCalledWith('a@b.com', 'nope')
  expect(await screen.findByText('Invalid')).toBeOnTheScreen()
  expect(screen.queryByText('Welcome')).not.toBeOnTheScreen()
})
```

NOTES

- → `userEvent` — realistic press/type sequences (async)
- → `getByLabelText` — uses `accessibilityLabel`
- → `getByRole('button', { name })` — role + accessible name
- → `findBy*` — waits for async UI
- → `queryBy*` — returns null; use for "not present"

IOS

- `jest-expo/ios` preset: `Platform.OS` is `'ios'`
- It resolves `.ios.tsx` files
- Tests run in Node — no Simulator needed

ANDROID

- `jest-expo/android` preset: `Platform.OS` is `'android'`
- `jest-expo/universal` runs iOS, Android and web together
- Mock native modules jest-expo doesn't cover

QUERY VARIANTS

PREFIX	BEHAVIOR
`getBy`	Throws if 0 or >1
`queryBy`	null if missing
`findBy`	Promise, waits (async)
`getAllBy / queryAllBy / findAllBy`	Arrays
`Priority`	Role → LabelText → Text → TestId

USEFUL MATCHERS

MATCHER	CHECKS
`toBeOnTheScreen()`	Element rendered
`toHaveTextContent()`	Text inside
`toBeDisabled()`	Disabled state
`toHaveStyle({…})`	Style values
`toHaveProp(name, v)`	Any prop
`toBeVisible()`	Not hidden

⚠ COMMON MISTAKES

- Querying by `testID` first; it doesn't test what users see, so prefer role and text
- Not awaiting `userEvent` calls; assertions run before state updates
- Mocking React Native itself wholesale; mock only the native module you need

✓ TIPS & BEST PRACTICES

- Wrap renders in a helper that provides your theme, query client, and navigation
- Use `jest.useFakeTimers()` for debounces and animations
- Test hooks with `renderHook` from RNTL

QUICK SUMMARY

- Jest + RNTL — component tests in Node
- `screen.getByRole` — user-centric queries
- `userEvent` — realistic interaction
- `jest.mock` — fake native modules

End-to-End Testing

TESTING THE REAL APP

End-to-end tests drive a built app on a simulator or device. **Maestro** uses simple YAML flows and works with any app (it reads the accessibility tree). **Detox** is a JavaScript gray-box framework that syncs with React Native's internals to reduce flakiness. Both find elements by `testID`, text, or accessibility label.

CHOOSING

- Maestro — fastest to start, YAML, Maestro Cloud
- Detox — JS tests, deep RN sync, more setup
- Appium — cross-tech, WebDriver ecosystem
- EAS Workflows can run Maestro on every build

MAESTRO FLOW

```yaml
# .maestro/login.yaml
appId: com.acme.myapp
---
- launchApp:
    clearState: true
- tapOn: "Email"
- inputText: "test@example.com"
- tapOn:
    id: "password-input"          # testID
- inputText: "hunter2"
- tapOn: "Sign in"
- assertVisible: "Welcome back"
- scrollUntilVisible:
    element: "Settings"
- takeScreenshot: home
```

IOS

- Tests run against a Simulator build
- `testID` becomes the accessibility identifier
- Pre-grant permissions in the test config

ANDROID

- `testID` becomes the view's tag / resource-id
- Detox needs the `androidTest` APK as well
- Disable emulator animations for stable runs

DETOX TEST

```ts
// e2e/login.test.ts
describe('Login', () => {
  beforeAll(async () => { await device.launchApp({ newInstance: true }) })

  it('signs in', async () => {
    await element(by.id('email-input')).typeText('test@example.com')
    await element(by.id('password-input')).typeText('hunter2')
    await element(by.text('Sign in')).tap()
    await expect(element(by.text('Welcome back'))).toBeVisible()
  })
})

// Run:
// maestro test .maestro/
// detox build -c ios.sim.release && detox test -c ios.sim.release
```

MAESTRO VS DETOX

TOPIC	MAESTRO	DETOX
Language	YAML	JS / TS
Sync with app	Polling + waits	Idling resources
Setup	CLI install	Native build config
Cloud	Maestro Cloud, EAS	DIY CI
Platforms	iOS, Android, web	iOS, Android

MAKING APPS TESTABLE

PRACTICE	WHY
`testID="password-input"`	Stable selector
`accessibilityLabel`	Readable + testable
Seed test data	Deterministic runs
Disable animations	Faster, less flaky
Release builds	Matches what users run

⚠ COMMON MISTAKES

- Testing against a debug build with Metro; timing differs from release and adds flakiness
- Using visible copy as the only selector; text changes break tests, add `testID` s
- Sharing state between tests; reset with `clearState` / `newInstance`

✓ TIPS & BEST PRACTICES

- Start with 3–5 critical flows (sign in, purchase, onboarding) before broad coverage
- Run E2E on PRs with EAS Workflows or a device farm
- Use `maestro studio` to inspect the element tree interactively

QUICK SUMMARY

- Maestro — YAML flows, quick start
- Detox — JS gray-box testing
- `testID` — stable selectors
- Release builds — realistic runs

Debugging & Performance

FINDING BUGS AND JANK

React Native DevTools (press `j` in Metro) is the official debugger: Console, Sources with breakpoints, Memory, Network, the **Performance** panel, and React's Components and Profiler tabs. **LogBox** shows warnings and errors in the app. For native crashes use Xcode and Android Studio (Logcat). Performance work focuses on dropped frames, slow startup, and extra re-renders.

RECENT CHANGES

- Remote JS debugging (Chrome) is gone; use RN DevTools
- `InteractionManager` removed in 0.87 → `requestIdleCallback`
- DevTools emulates light/dark mode (0.86)
- Network panel uses CDP; old interceptors deprecated

DEBUGGING TOOLKIT

```
console.log('user', user)          // DevTools Console
console.warn('slow path')          // yellow LogBox
console.error('bad state')         // red LogBox
debugger                           // breakpoint when DevTools open

LogBox.ignoreLogs(['Require cycle:'])   // silence a known warning
LogBox.ignoreAllLogs()                  // demos only

// Measure work
performance.mark('load-start')
await loadData()
performance.measure('load', 'load-start')

// Defer non-urgent work (replaces InteractionManager)
requestIdleCallback(() => prefetchImages())

// Find re-renders: React DevTools Profiler → "Highlight updates"
// Hermes CPU profile: DevTools → Performance → Record

// Release-like JS speed on Android without a full release build
// npx react-native run-android --mode debugOptimized
```

NOTES

- → `debugger` — pauses when DevTools is attached
- → `LogBox.ignoreLogs` — match by substring or regex
- → `performance.mark/measure` — Web Performance API
- → `requestIdleCallback` — run when JS is idle
- → `debugOptimized` — faster JS, still debuggable

IOS

- Xcode Instruments: Time Profiler, Allocations
- Native logs: Console.app or `xcrun simctl spawn booted log stream`
- Release test: `npx expo run:ios --configuration Release`

ANDROID

- Logs: `adb logcat *:S ReactNative:V ReactNativeJS:V`
- Android Studio Profiler or Perfetto for traces
- Release test: `npx expo run:android --variant release`

TOOLS

TOOL	USE
React Native DevTools	JS debugging, profiling, network
Xcode Instruments	iOS CPU, memory, leaks
Android Studio Profiler	CPU, memory, Logcat
Flashlight	Android FPS / perf score
Reactotron	State & API inspection
Sentry / Crashlytics	Production crashes

COMMON FIXES

SYMPTOM	FIX
Laggy lists	FlashList, memoized rows, fixed heights
Janky animations	Native driver / Reanimated
Slow startup	Lazy-load screens, fewer startup requires
Big bundle	Remove heavy deps, check with Expo Atlas
Frequent re-renders	React Compiler, store selectors
Large images	Resize, use `expo-image`

⚠ COMMON MISTAKES

- Judging performance in a debug build; dev mode is much slower, so profile release or `debugOptimized`
- Leaving `console.log` in hot paths in production; strip with a Babel plugin
- Calling `LogBox.ignoreAllLogs()` permanently and hiding real problems

✓ TIPS & BEST PRACTICES

- Use Expo Atlas (`EXPO_ATLAS=true npx expo start`) to see what's in your bundle
- Add Sentry or Crashlytics before your first release
- Check the React Profiler "why did this render" info before reaching for `memo`

QUICK SUMMARY

- React Native DevTools — `j` in Metro
- LogBox — in-app warnings
- Profile release-like builds
- `requestIdleCallback` — defer work

EAS Build

CLOUD BUILDS FOR BOTH STORES

EAS Build compiles your app on Expo's cloud machines (macOS for iOS, Linux for Android), manages signing credentials, and produces installable builds or store binaries. It works for Expo and bare React Native apps. **Build profiles** in `eas.json` define variants: a dev client for daily work, an internal preview, and production for the stores.

OUTPUT TYPES

- iOS simulator `.app`, device `.ipa` (ad hoc / store)
- Android `.apk` (sideload) or `.aab` (Play Store)
- Internal distribution — install links / QR codes
- `--local` builds on your own machine

EAS.JSON + COMMANDS

```
{
  "cli": { "version": ">= 16.0.0", "appVersionSource": "remote" },
  "build": {
    "development": {
      "developmentClient": true, "distribution": "internal",
      "env": { "APP_VARIANT": "development" },
      "ios": { "simulator": true }
    },
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" },
      "env": { "APP_VARIANT": "preview" }
    },
    "production": { "autoIncrement": true, "environment": "production" }
  },
  "submit": { "production": {} }
}
```

IOS

- Simulator build: `"ios": { "simulator": true }`
- Device builds need a provisioning profile — EAS manages it
- Register ad hoc devices with `eas device:create`

ANDROID

- `"android": { "buildType": "apk" }` for sideloading
- EAS creates and stores the keystore on the first build
- `eas build:run -p android` installs the latest build

CLI

```
npm i -g eas-cli && eas login
eas build:configure                    # creates eas.json

eas build --profile development --platform ios
eas build -p android --profile preview
eas build -p all --profile production   # both stores
eas build -p ios --local                # on your Mac

eas env:create --name API_URL --value https://api.example.com \
  --environment production
eas build:list                         # history & artifacts
eas credentials                        # view/manage signing
```

PROFILE KEYS

KEY	EFFECT
`developmentClient`	Include `expo-dev-client`
`distribution`	`internal` or `store`
`environment`	Which EAS env vars to use
`autoIncrement`	Bump build number each build
`channel`	EAS Update channel for OTA
`extends`	Inherit another profile

ENVIRONMENT VARIABLES

KIND	NOTES
`EXPO_PUBLIC_*`	Inlined into JS bundle (not secret)
`EAS env secret`	Build-time only, hidden in logs
`env in profile`	Plain values per profile
`.env files`	Local dev; EAS ignores unless uploaded

⚠ COMMON MISTAKES

- Putting API keys in `EXPO_PUBLIC_` variables thinking they're secret; they're in the bundle
- Building production with `distribution: internal`; the stores reject ad hoc builds
- Forgetting a native rebuild after adding a native library; OTA updates can't add native code

✓ TIPS & BEST PRACTICES

- Use `APP_VARIANT` plus `app.config.ts` so dev and prod apps install side by side
- Run `eas build --local` to debug failing cloud builds faster
- Cache-friendly: pin Node and package manager versions in `eas.json`

QUICK SUMMARY

- `eas.json` — build profiles
- `eas build -p` — iOS / Android / all
- `development` / `preview` / `production`
- EAS env vars — config per environment

EAS Submit & Versioning

SHIPPING BUILDS TO THE STORES

`eas submit` uploads a finished build to **App Store Connect** (TestFlight) or **Google Play**. Each store tracks two numbers: a user-facing **version** (`1.4.0`) and an internal **build number** that must increase with every upload. With `appVersionSource: "remote"`, EAS stores and auto-increments build numbers for you.

VERSION NUMBERS

- iOS: `CFBundleShortVersionString` + `CFBundleVersion`
- Android: `versionName` + `versionCode` (integer)
- Expo config: `version`, `ios.buildNumber`, `android.versionCode`
- Runtime check: `expo-application` `nativeApplicationVersion`

SUBMIT COMMANDS

```
# Build and submit in one go
eas build -p ios --profile production --auto-submit

# Submit an existing build
eas submit -p ios --latest
eas submit -p android --id <build-id>
eas submit -p android --path ./app-release.aab

# Version management (appVersionSource: remote)
eas build:version:get -p android
eas build:version:set -p ios          # set starting value
eas build:version:sync                # write back to local config

# eas.json submit profile
# "submit": { "production": {
#   "ios": { "ascAppId": "1234567890" },
#   "android": { "track": "internal", "releaseStatus": "draft" } } }
```

NOTES

- → `--auto-submit` — submit as soon as the build finishes
- → `--latest` — most recent build for that platform
- → `build:version:*` — manage remote build numbers
- → `ascAppId` — App Store Connect app ID
- → `track` — `internal` · `alpha` · `beta` · `production`

IOS

- Use an App Store Connect API key
- Uploads go through TestFlight processing first
- `buildNumber` must rise for each upload of a version

ANDROID

- The first upload must be done by hand in Play Console
- Needs a Google service account JSON key
- `versionCode` must always increase

SUBMIT CREDENTIALS

STORE	NEEDS
App Store	App Store Connect API key (`.p8`)
App Store	App record created in ASC first
Google Play	Service account JSON key
Google Play	First upload done manually in Play Console
Both	`eas credentials` stores keys securely

WHEN TO BUMP

CHANGE	BUMP
Native code / SDK upgrade	`version` (and runtime)
JS-only fix via OTA	Nothing (EAS Update)
New store binary	Build number (always)
Big feature	Minor or major version
Hotfix after store release	Patch version + new build

⚠ COMMON MISTAKES

- Re-uploading with the same build number; both stores reject duplicates
- Expecting the first Android submit to work from CLI; Play needs one manual upload to create the app
- Mixing local and remote version sources so numbers go backwards

✓ TIPS & BEST PRACTICES

- Let EAS own build numbers (`autoIncrement` + `remote`) and only edit `version` by hand
- Use Play's `internal` track and TestFlight for every release candidate
- Show `version (build)` in your settings screen for bug reports

QUICK SUMMARY

- `eas submit` — upload to stores
- `version` — user-facing
- build number — always increments
- `appVersionSource: remote` — EAS tracks it

Over-the-Air Updates

SHIP JS FIXES WITHOUT THE STORE

EAS Update publishes a new JS bundle and assets that installed apps download on launch, so you can fix bugs without a store review. Updates can only change JS and assets; **native code changes still need a new build**. The `runtimeVersion` ties each update to compatible native builds, and **channels** route updates to the right builds.

STORE RULES

- Apple and Google allow OTA for bug fixes and small changes
- Don't change the app's primary purpose via OTA
- Updates must target the same native runtime
- Keep a rollback path for bad updates

CONFIG AND PUBLISHING

```
# app.json
# "runtimeVersion": { "policy": "fingerprint" },
# "updates": { "url": "https://u.expo.dev/<project-id>" }

npx expo install expo-updates
eas update:configure

# Publish to the branch mapped to the "production" channel
eas update --channel production --message "Fix checkout crash"
eas update --branch preview --message "Try new onboarding"

# Gradual rollout and rollback
eas update --channel production --rollout-percentage 10
eas update:rollback
eas channel:edit production --branch release-1.4
```

IOS

- The embedded bundle is the fallback if no update loads
- Suspended apps can run for days — `Updates.reloadAsync()` for urgent fixes
- Updates must stay within App Review rules

ANDROID

- Google Play allows JS updates that don't change native code
- Debug builds load from Metro and ignore updates
- Test updates on a release build

IN-APP CHECKS

```
import * as Updates from 'expo-updates'

async function checkForUpdate() {
  if (__DEV__) return
  const result = await Updates.checkForUpdateAsync()
  if (result.isAvailable) {
    await Updates.fetchUpdateAsync()
    await Updates.reloadAsync()          // apply now
  }
}

// Or reactively
const { isUpdateAvailable, isUpdatePending } = Updates.useUpdates()
Updates.channel                          // 'production'
Updates.runtimeVersion                   // native compatibility key
```

RUNTIMEVERSION POLICIES

POLICY	CHANGES WHEN
`fingerprint`	Any native change (recommended)
`appVersion`	`version` changes
`nativeVersion`	version + build number change
`Custom string`	You bump it manually

CONCEPTS

TERM	MEANING
`Update`	A JS bundle + assets
`Branch`	Ordered list of updates
`Channel`	Set in a build; points to a branch
`Rollout`	Percent of users get the update
`Embedded update`	Bundle shipped inside the binary

⚠ COMMON MISTAKES

- Publishing an update that calls a native module the installed build doesn't have; the app crashes
- Forgetting the update applies on next launch by default; users may not see it immediately
- Using OTA to bypass review for new features; it violates store guidelines

✓ TIPS & BEST PRACTICES

- Use the `fingerprint` policy so incompatible updates are never delivered
- Roll out to 10% first and watch crash reports before going to 100%
- Test updates on a `preview` channel build before production

QUICK SUMMARY

- EAS Update — OTA JS & assets
- `runtimeVersion` — native compatibility
- Channels / branches — routing
- Native changes — need a store build

Upgrading React Native & Expo

STAYING CURRENT

React Native ships a minor version about every two months and supports the latest three. Expo releases SDKs about three times a year, each pinned to one RN version. Expo apps upgrade by bumping the SDK and running `expo install --fix`; with CNG, native folders are regenerated. Bare apps use the **Upgrade Helper** to diff template changes into their native projects.

CURRENT LINE-UP (OCT 2026)

- RN `0.87` stable, `0.86` active, `0.85` end of cycle
- Expo SDK `57` → RN `0.86`; SDK `58` beta → RN `0.88`
- Node `22.13+` required
- Skipping versions works but read every changelog in between

UPGRADE COMMANDS

```
# Expo
npx expo install expo@^57.0.0       # bump the SDK
npx expo install --fix              # align all expo-* + RN versions
npx expo-doctor                     # check for problems
npx expo prebuild                   # regenerate native folders (CNG)

# Bare React Native
# 1. Open the Upgrade Helper and apply the template diff by hand:
#    react-native-community.github.io/upgrade-helper/?from=0.85.0&to=0.87.0
npm install react-native@0.87 @react-native/babel-preset@0.87
cd ios && bundle exec pod install --repo-update
cd android && ./gradlew clean

# After upgrading
npx tsc --noEmit
npm test
```

NOTES

- → `expo install` — picks versions compatible with your SDK
- → `--fix` — corrects mismatched dependency versions
- → `expo-doctor` — config, duplicate deps, version checks
- → Upgrade Helper — shows every template file change
- → `pod install --repo-update` — refresh CocoaPods specs

IOS

- After upgrading: `cd ios && pod install --repo-update`
- Clear DerivedData on stale build errors
- Check the minimum iOS deployment target

ANDROID

- Gradle wrapper and AGP bumps come from the template diff
- `cd android && ./gradlew clean` after upgrading
- Check `minSdk`, `compileSdk` and Kotlin versions

UPGRADE CHECKLIST

STEP	WHY
Read release blog(s)	Breaking changes listed there
Check libraries on reactnative.directory	New Arch & version support
Upgrade one minor at a time if far behind	Smaller diffs
Clear caches	Metro, Gradle, DerivedData
Test both platforms in release	Catches native issues

RECENT BREAKING CHANGES

VERSION	CHANGE
0.82	New Architecture only
0.84	Hermes V1 default; iOS legacy arch compiled out
0.87	Strict TS API; deep imports removed
0.87	`InteractionManager`, StatusBar color props removed
SDK 57	`prebuild` cleans native dirs by default
SDK 58	iOS scene lifecycle (`SceneDelegate`)

⚠ COMMON MISTAKES

- Upgrading React Native with plain `npm install react-native@latest` in an Expo app; versions drift from the SDK
- Ignoring `expo-doctor` warnings about duplicate native modules
- Upgrading right before a release instead of on a quiet branch with time to test

✓ TIPS & BEST PRACTICES

- Upgrade on a branch and keep it small: SDK bump first, then library bumps
- Clear Xcode DerivedData and `./gradlew clean` when native builds fail after upgrades
- Subscribe to the RN and Expo changelogs so upgrades are never a surprise

QUICK SUMMARY

- RN — new minor every ~2 months
- `expo install --fix` — align versions
- Upgrade Helper — bare template diff
- Read changelogs — breaking changes

iOS Project Anatomy

WHAT'S INSIDE IOS/

The `ios/` folder is a normal Xcode project that embeds React Native. Native dependencies come from **CocoaPods**: `pod install` reads the `Podfile`, autolinks native modules from `node_modules`, and creates a `.xcworkspace`, which is what you open in Xcode. Since 0.77 the entry point is **Swift** (`AppDelegate.swift`) using `RCTReactNativeFactory`. Core RN supports **iOS 15.1+**; Expo SDK 56+ requires **16.4+**.

KEY FACTS

- Open `.xcworkspace`, never `.xcodeproj`
- Prebuilt RN core `.xcframework`s download during `pod install` (0.84+)
- SwiftPM is experimental in 0.87; CocoaPods stays default
- Expo SDK 58 adds `SceneDelegate.swift` (UIKit scene lifecycle)

APPDELEGATE.SWIFT (0.87 TEMPLATE)

```swift
import UIKit
import React
import React_RCTAppDelegate
import ReactAppDependencyProvider

@main
class AppDelegate: UIResponder, UIApplicationDelegate {
  var window: UIWindow?
  var reactNativeFactory: RCTReactNativeFactory?

  func application(_ application: UIApplication,
      didFinishLaunchingWithOptions launchOptions:
        [UIApplication.LaunchOptionsKey: Any]? = nil) -> Bool {
    let delegate = ReactNativeDelegate()
    let factory = RCTReactNativeFactory(delegate: delegate)
    delegate.dependencyProvider = RCTAppDependencyProvider()
    reactNativeFactory = factory
    window = UIWindow(frame: UIScreen.main.bounds)
    factory.startReactNative(withModuleName: "MyApp",
      in: window, launchOptions: launchOptions)
    return true
  }
}
// ReactNativeDelegate.bundleURL(): Metro in DEBUG, main.jsbundle in release
```

NOTES

- → `@main` — Swift app entry point
- → `RCTReactNativeFactory` — boots the RN runtime
- → `withModuleName` — must match `AppRegistry` name
- → `bundleURL` — Metro URL vs bundled JS
- → Expo projects generate an equivalent `ExpoAppDelegate`

IOS

- *Signing & Capabilities* writes the `.entitlements` file
- Deployment target must match the Podfile `platform`
- Bundle ID lives in the target's *General* tab

ANDROID

- `AndroidManifest.xml` ↔ Info.plist
- `MainApplication.kt` ↔ `AppDelegate.swift`
- Gradle `dependencies` ↔ Podfile

FILES IN IOS/

FILE	PURPOSE
`Podfile / Podfile.lock`	CocoaPods deps (commit the lock)
`MyApp.xcworkspace`	Open this in Xcode
`MyApp/Info.plist`	Bundle config, permissions, URL types
`MyApp/*.entitlements`	Capabilities (push, App Groups…)
`PrivacyInfo.xcprivacy`	Privacy manifest
`Images.xcassets`	App icon, colors, images

COMMANDS

COMMAND	EFFECT
`bundle install`	Install the pinned CocoaPods gem
`bundle exec pod install`	Link native deps
`pod install --repo-update`	Refresh spec repos
`RCT_USE_PREBUILT_RNCORE=0 pod install`	Build RN core from source
`npx react-native spm --deintegrate`	Try SwiftPM (0.87, experimental)
`xed ios`	Open the workspace in Xcode

⚠ COMMON MISTAKES

- Opening `MyApp.xcodeproj`; pods aren't linked and the build fails with missing modules
- Running a system `pod` instead of `bundle exec pod`; CocoaPods version mismatches cause odd errors
- Forgetting `pod install` after adding a native library; autolinking happens there

✓ TIPS & BEST PRACTICES

- Commit `Podfile.lock` so every machine gets the same native versions
- When builds break mysteriously, delete `ios/Pods`, `ios/build`, and DerivedData, then reinstall
- In Expo, change iOS settings in `app.json` `ios` or a config plugin, not in Xcode

QUICK SUMMARY

- `AppDelegate.swift` — native entry
- `Podfile` + `pod install` — native deps
- `.xcworkspace` — what you open
- iOS 15.1+ core, 16.4+ Expo SDK 56+

Xcode & Simulators

BUILDING AND RUNNING ON IOS

iOS builds require **macOS and Xcode**. The CLI drives `xcodebuild` for you, but Xcode is where you set signing, inspect native crashes, and use Instruments. A **scheme** picks the target and build configuration (**Debug** loads JS from Metro; **Release** embeds an optimized bundle). Simulators are managed with `xcrun simctl`; physical devices need signing and Developer Mode.

VERSION REQUIREMENTS

- RN 0.87 minimum: Xcode 16.1
- App Store uploads need Xcode 26 / iOS 26 SDK (since Apr 2026)
- Expo SDK 56+: Xcode 26.4
- iOS 27 SDK (Xcode 27) requires the UIKit scene lifecycle

RUNNING AND SIMULATOR CONTROL

```
# Run
npx expo run:ios                              # default simulator
npx expo run:ios --device "My iPhone"         # physical device
npx expo run:ios --configuration Release
npx react-native run-ios --simulator "iPhone 17 Pro"
npx react-native run-ios --udid <UDID>

# Simulators
xcrun simctl list devices available
xcrun simctl boot "iPhone 17"
open -a Simulator
xcrun simctl openurl booted myapp://post/42
xcrun simctl push booted com.acme.myapp payload.apns
xcrun simctl io booted screenshot shot.png
xcrun simctl privacy booted grant photos com.acme.myapp
xcrun simctl erase all                        # factory reset

# Build without running
xcodebuild -workspace ios/MyApp.xcworkspace -scheme MyApp \
  -configuration Release -sdk iphonesimulator build
```

NOTES

- → `--device` — prompts if no name is given
- → `--configuration Release` — test real performance
- → `simctl push` — fake a push notification from a JSON file
- → `simctl privacy grant` — skip permission prompts in tests
- → `xcodebuild` — CI builds without Xcode UI

IOS

- `xcrun simctl list devices` lists simulators
- `npx expo run:ios --device` targets a connected iPhone
- Turn on Developer Mode on iOS 16+ devices

ANDROID

- `emulator -list-avds` / `adb devices`
- `npx expo run:android --device` picks a device
- Turn on USB debugging in Developer options

DEBUG VS RELEASE

TOPIC	DEBUG	RELEASE
JS source	Metro over network	Embedded `main.jsbundle`
Dev menu	✔	✖
Speed	Slower	Optimized Hermes bytecode
Signing	Development	Distribution
Use for	Daily work	Perf tests, TestFlight

COMMON BUILD ERRORS

ERROR	FIX
`No such module 'React'`	Open `.xcworkspace`; `pod install`
`Signing requires a team`	Pick a team in Signing & Capabilities
`Sandbox: rsync … deny`	Set `ENABLE_USER_SCRIPT_SANDBOXING` NO
`Stale build artifacts`	Delete DerivedData, clean build folder
`Device not trusted`	Enable Developer Mode on the iPhone

⚠ COMMON MISTAKES

- Judging speed or animations in Debug; JS runs unoptimized with dev checks
- Updating macOS/Xcode the week of a release; new Xcode versions often need pod or RN updates
- Running on a device without enabling **Developer Mode** (Settings → Privacy & Security)

✓ TIPS & BEST PRACTICES

- Use `xcrun simctl` in scripts to reset state before E2E tests
- Keep one older simulator runtime installed to test your minimum iOS version
- Use `Product → Profile` (Instruments) on a Release build to find native hot spots

QUICK SUMMARY

- Xcode — required for iOS builds
- Debug vs Release — Metro vs bundle
- `xcrun simctl` — script simulators
- Developer Mode — needed on devices

Sheet 46

iOS Permissions & Entitlements

NS*UsageDescription keys · permission prompts · entitlements · capabilities · Background Modes · App Transport Security

ASKING IOS FOR ACCESS

iOS requires a **usage description** string in `Info.plist` for every protected resource; if it's missing, the app **crashes** the moment you touch the API, and App Review rejects vague text. **Entitlements** turn on capabilities like push, App Groups, Sign in with Apple, and HealthKit; they must also be enabled on your App ID. You get **one** system prompt per permission; after a denial, only Settings can change it.

RULES

- Usage text must explain *why*, specifically
- Ask in context, right before you need it
- `ATTrackingManager` prompt required before tracking (ATT)
- Background work needs a declared Background Mode

EXPO CONFIG (GENERATES INFO.PLIST + ENTITLEMENTS)

```
{ "expo": { "ios": {
  "infoPlist": {
    "NSCameraUsageDescription": "Scan receipts to add expenses.",
    "NSPhotoLibraryUsageDescription": "Attach photos to your notes.",
    "NSLocationWhenInUseUsageDescription": "Show stores near you.",
    "NSMicrophoneUsageDescription": "Record voice memos.",
    "NSFaceIDUsageDescription": "Unlock your vault with Face ID.",
    "UIBackgroundModes": ["remote-notification", "audio"],
    "ITSAppUsesNonExemptEncryption": false
  },
  "entitlements": {
    "com.apple.security.application-groups": ["group.com.acme.myapp"]
  },
  "usesAppleSignIn": true,
  "associatedDomains": ["applinks:example.com"]
} } }
```

NOTES

→ `infoPlist` — merged into `ios/MyApp/Info.plist`

→ Many Expo modules add these via plugin options instead

→ `UIBackgroundModes` — audio, location, fetch, push

→ `ITSAppUsesNonExemptEncryption` — skips export compliance prompt

→ `entitlements` — must match capabilities on the App ID

IOS

- A missing usage string crashes on request
- Photos can be `'limited'` — handle partial access
- Ask "When In Use" before upgrading to "Always"

ANDROID

- Declare in the manifest, request at runtime
- No usage strings — explain in your own UI first
- The system photo picker needs no permission (13+)

USAGE DESCRIPTION KEYS

KEY	FOR
`NSCameraUsageDescription`	Camera
`NSPhotoLibraryUsageDescription`	Read photos
`NSPhotoLibraryAddUsageDescription`	Save photos only
`NSLocationWhenInUseUsageDescription`	Location in foreground
`NSLocationAlwaysAndWhenInUseUsageDescription`	Background location
`NSContactsUsageDescription`	Contacts
`NSUserTrackingUsageDescription`	ATT tracking prompt

CAPABILITIES

CAPABILITY	ENTITLEMENT / NOTE
`Push Notifications`	`aps-environment`
`App Groups`	Share data with widgets/extensions
`Associated Domains`	Universal links, webcredentials
`Sign in with Apple`	Satisfies 4.8 privacy-login rule
`Background Modes`	`UIBackgroundModes` in Info.plist
`HealthKit / iCloud`	Separate entitlements + review notes
`Keychain Sharing`	Share credentials between apps

⚠ COMMON MISTAKES

- Missing a usage string; the app crashes on first use and reviewers see it immediately
- Writing "This app needs camera access"; Apple rejects strings that don't say why
- Requesting "Always" location up front; ask When In Use first, upgrade later

✓ TIPS & BEST PRACTICES

- Use `xcrun simctl privacy booted reset all com.acme.myapp` to see prompts again
- Pre-explain with your own screen, then call the system prompt
- Keep entitlements in config so EAS syncs capabilities to your App ID

QUICK SUMMARY

- `NS*UsageDescription` — required strings
- One prompt per permission
- Entitlements — capabilities
- `UIBackgroundModes` — background work

iOS UI Conventions & APIs

FEELING AT HOME ON IOS

iOS users expect swipe-back navigation, large titles, action sheets, haptics, SF Symbols, and system colors that adapt to dark mode. Native stacks and tabs give you most of this for free. Since iOS 26, **Liquid Glass** is the system material; `expo-glass-effect` exposes it. **Expo UI** (`@expo/ui/swift-ui`) lets you render real SwiftUI controls from React (stable since SDK 56).

PLATFORM EXPECTATIONS

- Back = swipe from left edge, not a hardware button
- Tab bar at bottom; no floating action buttons
- 44 × 44 pt minimum touch targets
- Respect Dynamic Type and Reduce Motion

IOS-ONLY APIS

```
// Action sheet
ActionSheetIOS.showActionSheetWithOptions(
  { options: ['Cancel', 'Save', 'Delete'], cancelButtonIndex: 0,
    destructiveButtonIndex: 2, title: 'Photo' },
  (i) => { if (i === 1) save(); if (i === 2) remove() })

// Share sheet (cross-platform)
await Share.share({ message: 'Check this out', url: link })

// Adaptive colors
const bg = DynamicColorIOS({ light: '#ffffff', dark: '#000000' })
const label = PlatformColor('label')

// Haptics
Haptics.selectionAsync()
Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium)

// SF Symbols (expo-symbols)
<SymbolView name="heart.fill" tintColor="red" size={24} />

// Liquid Glass (iOS 26+)
import { GlassView } from 'expo-glass-effect'
<GlassView glassEffectStyle="regular" style={styles.toolbar} />

// Native SwiftUI control via Expo UI
import { Host, Slider, Button } from '@expo/ui/swift-ui'
<Host matchContents>
  <Slider value={vol} min={0} max={1} onValueChange={setVol} />
</Host>
<Host matchContents>
  <Button label="Share" systemImage="square.and.arrow.up" onPress={share} />
</Host>
```

NOTES

- → `ActionSheetIOS` — native bottom action sheet
- → `DynamicColorIOS` — light/dark pair, auto switch
- → `PlatformColor('label')` — iOS semantic text color
- → `SymbolView` — SF Symbols, scales with text
- → `GlassView` — falls back to a plain view on older iOS

IOS

- `ActionSheetIOS` for native action sheets
- `DynamicColorIOS` for light / dark pairs
- Large titles and sheets match platform style

ANDROID

- Back button and gesture instead of edge swipe
- Floating action buttons and snackbars are normal
- 48 × 48 dp targets; `ToastAndroid` for short messages

NATIVE STACK IOS OPTIONS

OPTION	EFFECT
`headerLargeTitle`	Large collapsing title
`headerTransparent + headerBlurEffect`	Blurred header
`headerSearchBarOptions`	Native search bar
`presentation: 'formSheet'`	Sheet with detents
`gestureEnabled`	Swipe back
`Stack.Toolbar (Expo Router)`	Native toolbar items

IOS ACCESSIBILITY

API	USE
`accessibilityLabel`	VoiceOver text
`accessibilityRole`	Button, header, link…
`AccessibilityInfo.isReduceMotionEnabled()`	Tone down animation
`allowFontScaling / maxFontSizeMultiplier`	Dynamic Type
`accessibilityLanguage`	Pronunciation (iOS)

⚠ COMMON MISTAKES

- Building a custom back button and disabling swipe-back; it feels broken on iOS
- Using Android-style FABs and toasts on iOS; prefer toolbar buttons and inline banners
- Hardcoding white/black colors that ignore dark mode

✓ TIPS & BEST PRACTICES

- Use native stack/tabs (or `NativeTabs`) to get system transitions and Liquid Glass bars automatically
- Test with the largest Dynamic Type size in Settings → Accessibility
- Try Expo UI pickers, menus, and sheets before building custom controls

QUICK SUMMARY

- `ActionSheetIOS` / `Share` — native sheets
- `DynamicColorIOS` / `PlatformColor` — adaptive colors
- `expo-symbols` / haptics — iOS polish
- `GlassView` / Expo UI — Liquid Glass & SwiftUI

iOS Push Notifications

APNs · push key (.p8) · expo-notifications · device tokens · provisional authorization · notification service extension

PUSH ON IOS

iOS pushes go through **Apple Push Notification service (APNs)**. Your app asks permission, receives a **device token**, and your server sends payloads to APNs using a **.p8 auth key** from your Apple Developer account. `expo-notifications` handles permission, tokens, and listeners; with the **Expo Push Service** you send one API call and Expo forwards to APNs and FCM. Pushes need a real device or a simulator `simctl push`.

REQUIREMENTS

- Push Notifications capability + `aps-environment` entitlement
- APNs key (.p8) uploaded to EAS or your push provider
- Paid Apple Developer Program membership
- Silent pushes need `remote-notification` background mode

REGISTER AND LISTEN

```
import * as Notifications from 'expo-notifications'
import Constants from 'expo-constants'

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowBanner: true, shouldShowList: true,
    shouldPlaySound: true, shouldSetBadge: false }),
})

async function registerForPush() {
  const { status } = await Notifications.requestPermissionsAsync({
    ios: { allowAlert: true, allowBadge: true, allowSound: true,
           allowProvisional: false } })
  if (status !== 'granted') return null
  const projectId = Constants.expoConfig?.extra?.eas?.projectId
  const { data: expoToken } =
    await Notifications.getExpoPushTokenAsync({ projectId })
  const { data: apnsToken } = await Notifications.getDevicePushTokenAsync()
  return { expoToken, apnsToken }        // send to your server
}

useEffect(() => {
  const sub = Notifications.addNotificationResponseReceivedListener(r => {
    router.push(r.notification.request.content.data.url as string)
  })
  return () => sub.remove()
}, [])
```

NOTES

- → `setNotificationHandler` — show banners while app is open
- → `allowProvisional` — quiet delivery without a prompt
- → Expo token — use with Expo Push Service
- → Device token — raw APNs token for your own server
- → Response listener — user tapped a notification

IOS

- Test payloads: `xcrun simctl push booted <bundleId> file.apns`
- Provisional authorization delivers quietly without a prompt
- Foreground banners need `setNotificationHandler`

ANDROID

- FCM tokens instead of APNs
- Android 13+: `POST_NOTIFICATIONS` runtime permission
- Channels required; small icon must be white-on-transparent

APNS PAYLOAD KEYS

KEY	EFFECT
`aps.alert`	Title / subtitle / body
`aps.badge`	App icon badge number
`aps.sound`	`default` or file name
`aps.content-available: 1`	Silent / background push
`aps.mutable-content: 1`	Lets a service extension modify it
`aps.interruption-level`	`passive` · `active` · `time-sensitive` · `critical`

GOING FURTHER

FEATURE	HOW
`Images in notifications`	Notification Service Extension
`Custom UI`	Notification Content Extension
`Action buttons`	`setNotificationCategoryAsync`
`Live updates`	Live Activities (ActivityKit push)
`Local notifications`	`scheduleNotificationAsync`

⚠ COMMON MISTAKES

- Testing remote push on an old simulator setup and assuming it's broken; use a device or `simctl push`
- Mixing sandbox and production APNs environments; dev builds use sandbox tokens
- Asking for push permission on first launch before the user sees any value

✓ TIPS & BEST PRACTICES

- Use provisional authorization to deliver quietly and earn a later full opt-in
- Store tokens server-side per device and refresh them on every launch
- Upload your .p8 key once with `eas credentials`; it works for all your apps

QUICK SUMMARY

- APNs — Apple's push service
- `.p8` key — server auth
- `expo-notifications` — permissions & tokens
- Service extension — rich notifications

Sheet 49

iOS Code Signing & Provisioning

Apple Developer account · bundle identifier · certificates · provisioning profiles · automatic signing · EAS credentials

PROVING WHO BUILT THE APP

Every iOS build that runs on a device is **code signed**. Signing ties together your **team**, a **certificate** (your identity), an **App ID** (bundle identifier + capabilities), and a **provisioning profile** (which devices or store the build is allowed for). Xcode's automatic signing or **EAS credentials** can create and manage all of these for you.

ACCOUNTS

- Free Apple ID — run on your own devices, 7-day profiles
- Developer Program ($99/yr) — TestFlight, App Store, push
- Organization accounts need a D-U-N-S number
- Team ID — 10 characters, used in AASA files and entitlements

MANAGING CREDENTIALS

```
# EAS manages certs + profiles for you
eas credentials                          # interactive manager
eas build -p ios --profile production    # creates or reuses credentials
eas device:create                        # register a test device (ad hoc)

# See what's installed locally
security find-identity -v -p codesigning
ls ~/Library/Developer/Xcode/UserData/Provisioning\ Profiles

# Inspect a profile
security cms -D -i embedded.mobileprovision

# fastlane match (team sharing alternative)
fastlane match appstore
fastlane match development
```

NOTES

- → `eas credentials` — view, download, or revoke
- → `device:create` — adds UDIDs for internal builds
- → `find-identity` — certificates in your keychain
- → `cms -D` — decode a profile to check devices/entitlements
- → `match` — store certs in a private git repo

IOS

- Certificate + provisioning profile + bundle ID must match
- Manage them with `eas credentials`
- Distribution certificates are limited per account

ANDROID

- One upload keystore — no profiles or device lists
- Play App Signing holds the real signing key
- Lost upload key → request a reset in Play Console

SIGNING PIECES

PIECE	WHAT IT IS
Certificate	Development or Distribution identity (.p12)
App ID	Bundle ID + enabled capabilities
Provisioning profile	Cert + App ID + devices / store
Entitlements	Capabilities baked into the build
Team ID	Your developer team identifier
Push key (.p8)	APNs auth, not tied to one app
ASC API key (.p8)	CI uploads & EAS Submit

PROFILE TYPES

PROFILE	USED FOR
Development	Debug on registered devices
Ad Hoc	Internal installs (≤100 devices per type)
App Store	TestFlight and App Store
Enterprise	In-house distribution (Enterprise program)
Expiry	Profiles ~1 year; certs 1 year (dist.)
Renewal	EAS / Xcode regenerate automatically

⚠ COMMON MISTAKES

- Changing the bundle identifier after setup; profiles, push keys, and App Store records all point to the old one
- Enabling a capability in code but not on the App ID; signing fails with entitlement mismatches
- Letting several machines create distribution certs; you hit the cert limit and revoke each other's

✓ TIPS & BEST PRACTICES

- Let one system own credentials (EAS or fastlane match), never both
- Pick reverse-DNS bundle IDs you'll keep forever: `com.company.app`
- Use separate bundle IDs for dev/preview builds so they install side by side

QUICK SUMMARY

- Certificate — who signed it
- App ID — bundle ID + capabilities
- Profile — where it can run
- EAS / match — manage it for you

Sheet 50

App Store & TestFlight Release

archive · App Store Connect · TestFlight · PrivacyInfo.xcprivacy · App Review guidelines · phased release

GETTING INTO THE APP STORE

Release builds are uploaded to **App Store Connect**, processed, and then available in **TestFlight** for testing. When ready, you attach a build to a version, fill in metadata, screenshots, privacy details, and age rating, then submit for **App Review**. Approved versions can release immediately, on a date, or as a **phased release** over seven days.

REQUIRED BEFORE SUBMITTING

- Privacy manifest (`PrivacyInfo.xcprivacy`) with required-reason APIs
- App Privacy "nutrition label" in App Store Connect
- Built with the current required Xcode/SDK (Xcode 26+)
- Account deletion in-app if users can create accounts

UPLOAD PATHS

```
# EAS (build + upload)
eas build -p ios --profile production --auto-submit

# Xcode: Product → Archive → Distribute App → App Store Connect

# Command line
xcodebuild -workspace ios/MyApp.xcworkspace -scheme MyApp \
  -configuration Release -archivePath build/MyApp.xcarchive archive
xcodebuild -exportArchive -archivePath build/MyApp.xcarchive \
  -exportOptionsPlist ExportOptions.plist -exportPath build/
xcrun altool --upload-app -f build/MyApp.ipa -t ios \
  --apiKey $KEY_ID --apiIssuer $ISSUER_ID

# Expo privacy manifest (app.json)
# "ios": { "privacyManifests": { "NSPrivacyAccessedAPITypes": [
#   { "NSPrivacyAccessedAPIType":
#       "NSPrivacyAccessedAPICategoryUserDefaults",
#     "NSPrivacyAccessedAPITypeReasons": ["CA92.1"] } ] } }
```

NOTES

- → `--auto-submit` — straight to TestFlight after build
- → Archive — Release build packaged for distribution
- → `ExportOptions.plist` — method `app-store-connect`
- → `altool` / Transporter — upload an `.ipa`
- → `privacyManifests` — Expo merges into the manifest

IOS

- Review usually takes 24–48 h
- External TestFlight testers need beta review
- Third-party login requires a privacy-focused login option

ANDROID

- Play review: hours to days
- The internal testing track skips review
- Data safety form ↔ App Privacy label

TESTFLIGHT

GROUP	NOTES
Internal testers	Up to 100 team members, no review
External testers	Up to 10,000, beta review on first build
Public link	Anyone with the link joins
Build expiry	90 days
Feedback	Screenshots + crash reports from testers
Groups	Separate builds per tester group

COMMON REJECTIONS

GUIDELINE	CAUSE
2.1 App Completeness	Crashes, placeholder content, broken login
4.8 Login Services	Third-party login without a privacy-focused option
5.1.1 Data Collection	No account deletion, vague permission text
4.2 Minimum Functionality	Thin wrapper around a website
3.1.1 In-App Purchase	Digital goods sold outside IAP (rules vary by region)

⚠ COMMON MISTAKES

- Shipping without a privacy manifest entry for `UserDefaults` (AsyncStorage uses it); uploads get flagged
- Giving reviewers no demo account; login-gated apps get rejected for incompleteness
- Releasing to 100% at once; phased release limits damage from a bad build

✓ TIPS & BEST PRACTICES

- Put demo credentials and special instructions in App Review Information
- Upload early to TestFlight; processing and beta review take time
- Use `eas metadata` to keep store listing text in version control

QUICK SUMMARY

- App Store Connect — uploads & listings
- TestFlight — internal / external betas
- Privacy manifest + labels — required
- Phased release — gradual rollout

iOS Extensions, Widgets & Live Activities

BEYOND THE MAIN APP

Widgets, Live Activities, share extensions, and App Clips are separate **targets** written in Swift/SwiftUI that ship inside your app. They can't run React Native directly, so they share data with the app through an **App Group** container. Expo's `expo-widgets` (iOS widgets stable since SDK 56) and the `@bacons/apple-targets` plugin generate these targets from config so CNG doesn't wipe them.

HOW DATA FLOWS

- App writes to shared `UserDefaults(suiteName:)` / files
- Widget reads on its timeline and renders SwiftUI
- `WidgetCenter.shared.reloadAllTimelines()` refreshes
- Live Activities update locally or via ActivityKit push

WIDGET + SHARED DATA

```swift
// targets/widget/Widget.swift (SwiftUI, WidgetKit)
struct Entry: TimelineEntry { let date: Date; let steps: Int }

struct Provider: TimelineProvider {
  func placeholder(in: Context) -> Entry { Entry(date: .now, steps: 0) }
  func getSnapshot(in: Context, completion: @escaping (Entry) -> Void) {
    completion(load()) }
  func getTimeline(in: Context,
      completion: @escaping (Timeline<Entry>) -> Void) {
    completion(Timeline(entries: [load()], policy: .after(.now + 900))) }
  func load() -> Entry {
    let d = UserDefaults(suiteName: "group.com.acme.myapp")
    return Entry(date: .now, steps: d?.integer(forKey: "steps") ?? 0)
  }
}

@main struct StepsWidget: Widget {
  var body: some WidgetConfiguration {
    StaticConfiguration(kind: "Steps", provider: Provider()) { e in
      Text("\(e.steps) steps").font(.title)
    }.supportedFamilies([.systemSmall, .accessoryCircular])
  }
}
```

FROM REACT NATIVE

- → Write shared data to the App Group from JS with `expo-widgets` or a small native module
- → Then call `WidgetCenter.shared.reloadAllTimelines()` natively
- → `@bacons/apple-targets` — targets live in `targets/*/` with `expo-target.config.js`
- → Set `appleTeamId` in the plugin options
- → Both app and widget list `group.com.acme.myapp` in entitlements

IOS

- Extensions are separate targets — add via a config plugin
- Share data through an App Group (`group.com.example`)
- Widgets have tight memory limits

ANDROID

- Home-screen widgets use AppWidgetProvider / Glance
- `react-native-android-widget` renders widgets from JS
- `updatePeriodMillis` minimum is 30 minutes

EXTENSION TYPES

TARGET	PURPOSE
`Widget (WidgetKit)`	Home / Lock Screen glanceable info
`Live Activity`	Lock Screen + Dynamic Island live status
`Share extension`	Receive content from other apps
`Notification Service`	Modify pushes (images)
`App Clip`	Small, size-limited instant experience
`App Intents`	Siri, Shortcuts, Spotlight actions

CONSTRAINTS

RULE	DETAIL
`No RN runtime`	Write SwiftUI / Swift
`Shared data`	App Group on app and extension
`Memory limits`	Widgets ~30 MB
`Refresh budget`	Timelines, not live polling
`Bundle ID`	`com.acme.myapp.widget` (child of app)

⚠ COMMON MISTAKES

- Adding a target in Xcode inside a CNG project; `prebuild` deletes it, so generate it from config
- Mismatched App Group IDs between app and extension; the widget reads nothing
- Expecting a widget to refresh every second; WidgetKit budgets updates

✓ TIPS & BEST PRACTICES

- Keep widget data small and pre-formatted so the extension stays lightweight
- Use Live Activities for deliveries, timers, and scores instead of frequent pushes
- Try `expo-widgets` first; drop to `@bacons/apple-targets` for other extension types

QUICK SUMMARY

- Extensions — separate Swift targets
- App Group — shared storage
- WidgetKit / ActivityKit — widgets & live status
- Config plugins — keep targets in CNG

iPadOS

SAME APP, BIGGER CANVAS

iPad runs your iOS build. If you don't opt in, it shows an iPhone-sized app scaled up. Opt in with `ios.supportsTablet` (Expo) or by targeting iPad in Xcode, then design **adaptive layouts**: the window can be full screen, Split View, Slide Over, or a freely resized Stage Manager window. Users also expect pointer hover, keyboard shortcuts, and all orientations.

KEY BEHAVIORS

- Window size ≠ screen size; use `useWindowDimensions`
- All four orientations expected on iPad
- Pointer hover via `onHoverIn` / `onHoverOut`
- Mac (Apple silicon) can run iPad apps too

ADAPTIVE IPAD LAYOUT

```
// app.json
// "ios": { "supportsTablet": true },
// "orientation": "default"          // allow rotation

function Inbox() {
  const { width } = useWindowDimensions()
  const split = Platform.isPad && width >= 700
  return split ? (
    <View style={{ flex: 1, flexDirection: 'row' }}>
      <View style={{ width: 320, borderRightWidth: 1 }}>
        <MessageList onSelect={setSelected} />
      </View>
      <View style={{ flex: 1 }}>
        <MessageDetail id={selected} />
      </View>
    </View>
  ) : (
    <MessageList onSelect={id => router.push(`/message/${id}`)} />
  )
}

<Pressable onHoverIn={() => setHover(true)}
  onHoverOut={() => setHover(false)}
  style={{ backgroundColor: hover ? '#2e3348' : 'transparent' }} />
```

NOTES

- → `supportsTablet` — run natively instead of scaled
- → `split` — base on window width, not `isPad` alone
- → Two-pane — list + detail when wide
- → Phone-width windows fall back to stack navigation
- → `onHoverIn` — pointer and Apple Pencil hover

IOS

- `ios.supportsTablet: true` in Expo config
- Stage Manager resizes windows freely
- Hardware keyboard shortcuts need native `UIKeyCommand`

ANDROID

- Tablets and foldables resize windows at runtime
- Targeting API 36 ignores orientation locks on large screens
- Use window size classes for breakpoints

WINDOW MODES

MODE	TYPICAL WIDTH (PT)
Full screen (11")	~834 portrait / ~1194 landscape
Split View ½	~507–678
Split View ⅓	~320 (phone-like)
Slide Over	~320
Stage Manager	Freely resizable

INPUT ON IPAD

INPUT	API
Pointer hover	`onHoverIn` / `onHoverOut`
Keyboard shortcuts	`UIKeyCommand` via a native module
Context menu	Expo UI `ContextMenu` / `react-native-ios-context-menu`
Drag and drop	`UIDragInteraction` via a native module
Apple Pencil	Hover + pressure via native libs

⚠ COMMON MISTAKES

- Branching on `Platform.isPad` alone; a ⅓ Split View window is phone-sized
- Locking iPad to portrait; reviewers and users expect rotation
- Fixed-width layouts that leave huge empty margins in landscape

✓ TIPS & BEST PRACTICES

- Test Split View and Stage Manager sizes, not just full screen
- Reuse the same breakpoints for foldables and Android tablets
- Add a max content width (e.g. 700) for reading screens on big windows

QUICK SUMMARY

- `supportsTablet` — native iPad app
- Window width — drives layout
- Two-pane — list + detail
- Hover & keys — pointer input

macOS

NATIVE MAC APPS

React Native macOS is Microsoft's fork that renders real **AppKit** views (`NSView`), giving you a true Mac app. It tracks core RN with a lag (latest **0.83** as of Oct 2026), so pin your RN minor to match. Alternatives: run your iPad app on Apple silicon Macs ("Designed for iPad"), or enable **Mac Catalyst** for a UIKit-based Mac build of your iOS target.

SETUP RULES

- RN and RN macOS must share the same minor version
- Adds a `macos/` folder with its own Podfile and workspace
- `Platform.OS === 'macos'`
- Desktop UX: menus, windows, keyboard, hover

ADD MACOS TO A PROJECT

```
# New project at a matching RN minor
npx @react-native-community/cli init MyApp --version 0.83.0
cd MyApp
npx react-native-macos-init          # adds react-native-macos + macos/

npx react-native run-macos            # build + launch
npx react-native build-macos          # build only
xed -b macos                          # open the workspace in Xcode
```

IOS

- macOS uses AppKit, not UIKit — iOS-only libraries need a port
- Mac Catalyst / "Designed for iPad" is the zero-effort route
- `Platform.OS` is `'macos'`, not `'ios'`

ANDROID

- No Android desktop target — ChromeOS runs Android builds
- Keep Android-only code in `.android.tsx`
- Hover props (`onHoverIn` / `onHoverOut`) work on ChromeOS

DESKTOP-AWARE COMPONENTS

```
<Pressable
  onHoverIn={() => setHover(true)}
  onHoverOut={() => setHover(false)}
  focusable                           // tab-key focus
  tooltip="Open a file"               // macOS-only prop
  keyDownEvents={[{ key: 'Enter' }]}  // keys to receive
  onKeyDown={e => { if (e.nativeEvent.key === 'Enter') open() }}
  style={{ cursor: 'pointer' }}>
  <Text>Open file</Text>
</Pressable>

const title = Platform.select({
  macos: 'Preferences…', default: 'Settings' })
```

MAC OPTIONS COMPARED

OPTION	PROS	CONS
RN macOS	True AppKit app, desktop UX	Fork lags core; fewer libs
Mac Catalyst	Same iOS target, UIKit	iPad-like feel; some APIs missing
iPad app on Mac	Zero work	Feels like a tablet app
Electron / Tauri + RNW	Web stack	Not native views

MAC DIFFERENCES

AREA	NOTE
Navigation	Sidebars & windows, not stacks
Menus	Menu bar; native menus via native code
Input	Keyboard first; hover; right-click
Window size	Freely resizable, often huge
Libraries	Check for `macos` support on reactnative.directory

⚠ COMMON MISTAKES

- Mixing RN 0.87 with RN macOS 0.83; versions must match at the minor level
- Porting a phone UI unchanged; Mac users expect sidebars, menus, and shortcuts
- Assuming every native library supports macOS; many are iOS/Android only

✓ TIPS & BEST PRACTICES

- Share business logic and components; keep navigation and shell per platform
- Use `.macos.tsx` files for desktop-specific screens
- Try Mac Catalyst first if you only need "the iPad app, but on Mac"

QUICK SUMMARY

- RN macOS — AppKit fork by Microsoft
- `react-native-macos-init` — add target
- Match RN minor versions
- Catalyst / iPad-on-Mac — alternatives

tvOS (Apple TV)

BUILDING FOR THE LIVING ROOM

Apple TV support comes from **react-native-tvos**, a community fork that tracks core closely (`0.87.1`–1 as of Oct 2026) and also supports **Android TV**. You alias it as `react-native` in `package.json`. TV apps have no touch: users move a **focus** highlight with the Siri Remote, so every interactive element must be focusable and the layout must make focus movement predictable.

SETUP

- `"react-native": "npm:react-native-tvos@latest"`
- Don't install core `react-native` alongside it
- Expo: TV builds via CNG (Expo dev menu works on TV since SDK 54)
- `Platform.isTV` is true on tvOS and Android TV

FOCUS AND REMOTE EVENTS

```
import { TVFocusGuideView, useTVEventHandler, Platform } from 'react-native'

function Row({ items }: { items: Movie[] }) {
  return (
    <TVFocusGuideView autoFocus trapFocusLeft trapFocusRight
      style={{ flexDirection: 'row', gap: 24 }}>
      {items.map((m, i) => (
        <Pressable key={m.id} hasTVPreferredFocus={i === 0}
          onFocus={() => setActive(m.id)}
          style={({ focused }) => [styles.poster,
            focused && { transform: [{ scale: 1.1 }] }]}>
          <Image source={{ uri: m.poster }} style={styles.img} />
        </Pressable>
      ))}
    </TVFocusGuideView>
  )
}

useTVEventHandler(evt => {
  if (evt.eventType === 'playPause') togglePlayback()
})

// Menu / back button on Apple TV
TVEventControl.enableTVMenuKey()
BackHandler.addEventListener('hardwareBackPress',
  () => { goBack(); return true })
```

NOTES

- → `TVFocusGuideView` — groups focus and remembers last item
- → `autoFocus` — focus first child on first visit
- → `trapFocus*` — keep focus inside the row
- → `hasTVPreferredFocus` — initial focus target
- → `focused` style state — show the highlight

IOS

- One `react-native-tvos` package builds iOS and tvOS
- Expo: prebuild with `EXPO_TV=1` for TV targets
- tvOS has no WebKit — no WebView

ANDROID

- Needs the `LEANBACK_LAUNCHER` intent and a banner image
- Mark touchscreen as not required in the manifest
- Navigation is D-pad focus, not touch

FOCUS APIS

API	PURPOSE
`TVFocusGuideView destinations`	Redirect focus to specific views
`onFocusEnter / onFocusLeave`	Focus entered / left the guide
`nextFocusUp/Down/Left/Right`	Explicit neighbors (iOS + Android)
`focusable={false}`	Exclude from focus
`snapToAlignment="item"`	Scroll lists to focused items

REMOTE EVENTS

EVENTTYPE	FROM
`up / down / left / right`	Swipes or D-pad
`select`	Click / center button
`playPause`	Play/Pause button
`longSelect`	Press and hold
`Menu / Back`	`BackHandler` + `enableTVMenuKey`

⚠ COMMON MISTAKES

- Using touch-only components with no focus styles; users can't tell what's selected
- Installing both `react-native` and `react-native-tvos`; native linking breaks
- Small text and dense layouts; design for viewing from 3 meters away

✓ TIPS & BEST PRACTICES

- Keep a 90 pt safe margin on the edges for TV overscan
- Test with the Apple TV simulator's remote window and arrow keys
- Share code with the phone app, with `Platform.isTV` branches for layout

QUICK SUMMARY

- react-native-tvos — tvOS + Android TV
- Focus, not touch
- `TVFocusGuideView` — focus groups
- `useTVEventHandler` — remote buttons

visionOS

APPS IN SPATIAL COMPUTING

React Native visionOS is Callstack's fork that builds native visionOS apps: translucent glass windows, multiple windows, and **immersive spaces** you define in SwiftUI and open from JS. It lags core (latest `0.79.x` in Oct 2026). Without it, your iPad app can still run on Vision Pro in compatibility mode, but it looks like a flat iPad window.

SETUP

- `npx @callstack/react-native-visionos@latest init MyApp`
- Entry point is `App.swift` (SwiftUI), not `AppDelegate`
- CLI: `run-visionos`, `build-visionos`, `log-visionos`
- `UIApplicationSupportsMultipleScenes` = true for spaces

SWIFTUI ENTRY + IMMERSIVE SPACE

```
// visionos/MyApp/App.swift
@main
struct MyApp: App {
  @UIApplicationDelegateAdaptor var delegate: AppDelegate
  @State private var style: ImmersionStyle = .mixed

  var body: some Scene {
    RCTMainWindow(moduleName: "MyApp")          // your RN UI
    ImmersiveSpace(id: "Solar") {
      SolarSystemView()                          // RealityKit content
    }
    .immersionStyle(selection: $style, in: .mixed, .progressive, .full)
  }
}
```

IOS

- Most iPad apps run unmodified on visionOS
- RN visionOS forks iOS — UIKit code mostly ports
- `visionos_hoverEffect` adds gaze hover effects

ANDROID

- Android XR runs Android apps in floating panels
- Meta Quest (Horizon OS) runs Android apps too
- Spatial features need native Jetpack XR code

OPENING IT FROM REACT NATIVE

```
import { XR } from '@callstack/react-native-visionos'

async function enterSpace() {
  if (!XR.supportsMultipleScenes) return
  try {
    await XR.requestSession('Solar', { planet: 'earth' })
  } catch (e) {
    Alert.alert('Could not open space', String(e))
  }
}
const exitSpace = () => XR.endSession()

// Hover effect (eye / pointer highlight)
<Pressable style={{ cursor: 'pointer' }} onPress={enterSpace}>
  <Text>Enter</Text>
</Pressable>
```

VISIONOS CONCEPTS

CONCEPT	MEANING
`Window`	2D glass panel (`RCTMainWindow`)
`Ornament`	Toolbar attached outside a window
`Volume`	3D bounded content
`Immersive space`	Content around the user (`mixed` → `full`)
`Hover effect`	Highlight where the user looks

PLATFORM NOTES

RULE	DETAIL
`One immersive space at a time`	`requestSession` fails otherwise
`Interaction`	Look + pinch, not touch
`Targets`	60 pt minimum
`3D content`	RealityKit / SwiftUI, not RN views
`Compatibility mode`	iPad build runs without the fork

⚠ COMMON MISTAKES

- Forgetting `UIApplicationSupportsMultipleScenes`; `requestSession` throws
- Mismatched IDs between `ImmersiveSpace(id:)` and `requestSession('…')`
- Expecting RN components to render in 3D; immersive content is SwiftUI/RealityKit

✓ TIPS & BEST PRACTICES

- Ship the iPad build first; move to the fork only if you need spaces or the glass look
- Use `cursor: 'pointer'` on interactive elements so gaze highlighting works
- Keep 3D scenes in Swift and drive them with data passed from `requestSession`

QUICK SUMMARY

- react-native-visionos — Callstack fork
- `App.swift` + `RCTMainWindow` — entry
- `XR.requestSession` — open immersive space
- `cursor: 'pointer'` — hover effects

Android Project Anatomy

WHAT'S INSIDE ANDROID/

The `android/` folder is a standard **Gradle** project. `MainApplication.kt` creates the `ReactHost` and loads React Native; `MainActivity.kt` hosts your root component. Native modules from `node_modules` are **autolinked** by the React Native Gradle Plugin. The 0.87 template targets **minSdk 24** (Android 7.0), **targetSdk 36**, **compileSdk 37**, with AGP 9 and Kotlin 2.2.

KEY FACTS

- Kotlin entry files since 0.73; Java still works
- `newArchEnabled` is ignored (always on since 0.82)
- `edgeToEdgeEnabled=true` in the template
- Hermes V1 is the default engine (0.84+)

MAINAPPLICATION.KT + MAINACTIVITY.KT

```
class MainApplication : Application(), ReactApplication {
  override val reactHost: ReactHost by lazy {
    getDefaultReactHost(
      context = applicationContext,
      packageList = PackageList(this).packages.apply {
        // add(MyReactNativePackage())   // manual (non-autolinked)
      },
    )
  }
  override fun onCreate() {
    super.onCreate()
    loadReactNative(this)
  }
}

class MainActivity : ReactActivity() {
  override fun getMainComponentName(): String = "MyApp"
  override fun createReactActivityDelegate(): ReactActivityDelegate =
    DefaultReactActivityDelegate(this, mainComponentName, fabricEnabled)
}
```

NOTES

- → `PackageList` — autolinked native packages
- → `loadReactNative` — initializes RN and Hermes
- → `getMainComponentName` — must match `AppRegistry`
- → `DefaultReactActivityDelegate` — New Arch delegate
- → Expo generates equivalents with `ReactActivityDelegateWrapper`

IOS

- Info.plist ↔ `AndroidManifest.xml`
- `AppDelegate.swift` ↔ `MainApplication.kt`
- Podfile ↔ Gradle `dependencies`

ANDROID

- `applicationId` in `app/build.gradle` is the package ID
- `res/values/strings.xml` holds `app_name`
- Launcher icons live in `res/mipmap-*`

KEY FILES

FILE	PURPOSE
`app/build.gradle`	applicationId, versions, signing, R8
`build.gradle`	SDK levels, Kotlin, NDK versions
`gradle.properties`	Flags: Hermes, edge-to-edge, ABIs
`settings.gradle`	Includes RN Gradle plugin, autolinking
`app/src/main/AndroidManifest.xml`	Activities, permissions, intents
`app/src/main/res/`	Icons, splash, strings, styles

GRADLE.PROPERTIES FLAGS

PROPERTY	EFFECT
`hermesEnabled=true`	Use Hermes (default)
`edgeToEdgeEnabled=true`	Draw under system bars
`reactNativeArchitectures`	ABIs to build (`arm64-v8a`…)
`android.builtInKotlin=false`	AGP 9 opt-out (until AGP 10)
`android.newDsl=false`	AGP 9 DSL opt-out
`org.gradle.jvmargs`	Gradle memory

⚠ COMMON MISTAKES

- Changing `applicationId` after publishing; Play treats it as a different app
- Editing `android/` by hand in an Expo CNG project; `prebuild` regenerates it, so use config plugins
- Building only `arm64-v8a` for local speed and shipping that; older and x86 devices break

✓ TIPS & BEST PRACTICES

- Build one ABI locally for speed: `-PreactNativeArchitectures=arm64-v8a`
- Keep `gradle.properties` changes in config (`expo-build-properties`) for Expo apps
- Use Android Studio's "Open" on `android/` to get Gradle sync and Kotlin tooling

QUICK SUMMARY

- `MainApplication.kt` — RN host
- `MainActivity.kt` — root component
- Gradle files — SDKs, signing, flags
- minSdk 24 · target 36 · compile 37

Sheet 57

Android Studio, Emulators & adb

SDK Manager · AVD emulators · adb devices · adb reverse · logcat · run:android --variant · Gradle sync errors

BUILDING AND RUNNING ON ANDROID

Android builds work on macOS, Windows, and Linux. **Android Studio** installs the SDK, build tools, and emulators (AVDs). The RN CLI runs Gradle for you; **adb** (Android Debug Bridge) installs apps, forwards ports, and streams logs. You need **JDK 17**, `ANDROID_HOME` set, and the platform tools on your PATH.

ENVIRONMENT

- `ANDROID_HOME=$HOME/Library/Android/sdk` (macOS)
- Add `$ANDROID_HOME/platform-tools` and `emulator` to PATH
- Install the SDK Platform for compileSdk (API 37)
- Physical device: enable Developer options + USB debugging

RUNNING, ADB, GRADLE

```
npx expo run:android                          # build + install + Metro
npx expo run:android --variant release
npx react-native run-android --mode debugOptimized
npx react-native run-android --deviceId emulator-5554

emulator -list-avds && emulator -avd Pixel_9_API_36
adb devices                                   # list devices
adb reverse tcp:8081 tcp:8081                 # Metro over USB
adb install app-release.apk
adb shell am start -n com.acme.myapp/.MainActivity
adb logcat *:S ReactNative:V ReactNativeJS:V
adb shell input keyevent 82                   # open dev menu
adb shell pm clear com.acme.myapp             # wipe app data

cd android
./gradlew assembleDebug                       # APK
./gradlew bundleRelease                       # AAB for Play
./gradlew clean
./gradlew :app:dependencies
```

NOTES

- → `--variant` / `--mode` — Gradle build type
- → `adb reverse` — device `localhost:8081` → your Metro
- → `logcat` filter — only React Native logs
- → `keyevent 82` — menu key = dev menu
- → `bundleRelease` — Play Store artifact

IOS

- `xcodebuild` ↔ `./gradlew`
- `simctl` ↔ `adb` / `emulator`
- Debug / Release configurations ↔ build types

ANDROID

- `./gradlew assembleRelease` (APK) or `bundleRelease` (AAB)
- `adb install -r app-release.apk`
- Raise Gradle memory via `org.gradle.jvmargs`

BUILD TYPES

TYPE	USE
`debug`	Daily dev, Metro, debuggable
`debugOptimized`	Faster JS, still debuggable (0.82+)
`release`	Store builds, minified, signed
`assembleRelease`	Signed APK for sideloading
`Product flavors`	Variants like `free` / `pro`

COMMON BUILD ERRORS

ERROR	FIX
`SDK location not found`	Set `ANDROID_HOME` or `local.properties`
`Unsupported class file / JDK`	Use JDK 17 (`JAVA_HOME`)
`INSTALL_FAILED_UPDATE_INCOMPATIBLE`	Uninstall: signature changed
`Duplicate class / version clash`	`./gradlew clean`, align deps
`Out of memory`	Raise `org.gradle.jvmargs`
`CMake / NDK missing`	Install via SDK Manager

⚠ COMMON MISTAKES

- Testing performance in `debug`; use `debugOptimized` or `release`
- Forgetting `adb reverse` on USB devices, so the app can't reach Metro
- Mixing JDK versions between Android Studio and the terminal

✓ TIPS & BEST PRACTICES

- Use Android Studio's Logcat and Layout Inspector for native crashes and view hierarchies
- Create an emulator with the newest API **and** one with your minSdk
- Turn on Gradle configuration cache and build cache for faster local builds

QUICK SUMMARY

- Android Studio — SDK + emulators
- `adb` — install, logs, port forwarding
- `./gradlew bundleRelease` — AAB
- JDK 17 + `ANDROID_HOME` — required

Android Permissions

THE ANDROID PERMISSION MODEL

Every permission must be declared in `AndroidManifest.xml`. **Normal** permissions (internet, vibrate) are granted at install. **Dangerous** ones (camera, location, contacts, notifications on Android 13+) also need a **runtime request**. Users can deny, deny permanently ("Don't ask again"), or grant once. `PermissionsAndroid` is the built-in API; Expo modules wrap the same flow.

VERSION CHANGES

- Android 13 (API 33): `POST_NOTIFICATIONS` runtime permission
- Android 13: granular `READ_MEDIA_IMAGES` / `VIDEO`
- Android 14: partial photo access (`READ_MEDIA_VISUAL_USER_SELECTED`)
- Background location must be requested separately

MANIFEST + RUNTIME REQUEST

```
// AndroidManifest.xml (or app.json "android.permissions")
// <uses-permission android:name="android.permission.CAMERA" />
// <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

import { PermissionsAndroid, Platform, Linking } from 'react-native'

async function requestCamera(): Promise<boolean> {
  if (Platform.OS !== 'android') return true
  const result = await PermissionsAndroid.request(
    PermissionsAndroid.PERMISSIONS.CAMERA,
    { title: 'Camera access', message: 'Scan receipts to add expenses.',
      buttonPositive: 'OK', buttonNegative: 'Not now' })
  if (result === PermissionsAndroid.RESULTS.GRANTED) return true
  if (result === PermissionsAndroid.RESULTS.NEVER_ASK_AGAIN) {
    Linking.openSettings()
  }
  return false
}

const already = await PermissionsAndroid.check(
  PermissionsAndroid.PERMISSIONS.ACCESS_FINE_LOCATION)

const results = await PermissionsAndroid.requestMultiple([
  PermissionsAndroid.PERMISSIONS.ACCESS_FINE_LOCATION,
  PermissionsAndroid.PERMISSIONS.ACCESS_COARSE_LOCATION,
])
```

NOTES

- → Rationale object — shown if the user denied once before
- → `NEVER_ASK_AGAIN` — only Settings can change it now
- → `check` — current status, no dialog
- → `requestMultiple` — returns a map of results
- → Expo: `android.permissions` and `blockedPermissions` in app.json

IOS

- No manifest list — Info.plist usage strings instead
- One prompt; after that, only Settings
- No rationale API — explain before asking

ANDROID

- `PermissionsAndroid.request(perm, rationale)`
- `'never_ask_again'` → `Linking.openSettings()`
- One-time grants for location, camera and mic (11+)

COMMON DANGEROUS PERMISSIONS

PERMISSION	FOR
`CAMERA`	Camera
`RECORD_AUDIO`	Microphone
`ACCESS_FINE_LOCATION / COARSE`	Precise / approximate location
`ACCESS_BACKGROUND_LOCATION`	Location while in background
`POST_NOTIFICATIONS`	Show notifications (13+)
`READ_MEDIA_IMAGES`	Photos (13+)

RESULTS VALUES

VALUE	MEANING
`granted`	Allowed
`denied`	Denied; can ask again
`never_ask_again`	Permanently denied
`Play policy`	Some perms need a declaration form

⚠ COMMON MISTAKES

- Requesting a permission that isn't in the manifest; the request returns denied instantly
- Asking for background location in the same request as foreground; Android 11+ requires separate steps
- Leaving unused permissions merged in from libraries; Play flags them, so remove with `tools:node="remove"`

✓ TIPS & BEST PRACTICES

- Use Expo's `blockedPermissions` to strip permissions added by dependencies
- Prefer the Photo Picker (no permission) over `READ_MEDIA_*` for picking images
- Check `aapt dump permissions app.apk` to see the final merged list

QUICK SUMMARY

- Manifest — declare every permission
- `PermissionsAndroid.request` — runtime prompt
- `never_ask_again` — send to Settings
- `POST_NOTIFICATIONS` — Android 13+

Android UI Conventions & APIs

FEELING AT HOME ON ANDROID

Android users expect the system **back** gesture to work everywhere, ripple feedback on touch, Material components, and apps that draw **edge-to-edge** under transparent system bars (enforced on Android 15+, no opt-out on 16). Handle back with `BackHandler`, show quick messages with `ToastAndroid`, and use **Material You** dynamic colors. Expo UI's `@expo/ui/jetpack-compose` renders real Compose components.

PLATFORM EXPECTATIONS

- Back gesture / button must close modals, then go back
- Predictive back animation default when targeting API 36
- Status and nav bars transparent; pad with safe-area insets
- 48 × 48 dp minimum touch targets

ANDROID-ONLY APIS

```
// Back handling (also handles Android TV back)
useEffect(() => {
  const sub = BackHandler.addEventListener('hardwareBackPress', () => {
    if (menuOpen) { setMenuOpen(false); return true }   // handled
    return false                                        // default: back / exit
  })
  return () => sub.remove()
}, [menuOpen])

ToastAndroid.show('Saved', ToastAndroid.SHORT)

<Pressable android_ripple={{ color: '#ffffff33', borderless: false }}>
  <Text>Ripple</Text>
</Pressable>

// Material You (Android 12+) dynamic colors
const primary = PlatformColor('@android:color/system_accent1_500')

// Navigation bar style (expo-navigation-bar, SDK 56 component)
<NavigationBar style="light" hidden={false} />

// Native Jetpack Compose control via Expo UI
import { Host, Switch } from '@expo/ui/jetpack-compose'
<Host matchContents><Switch value={on} onCheckedChange={setOn} /></Host>
```

NOTES

- → Return `true` — you handled back; `false` — default behavior
- → `ToastAndroid` — short system message
- → `android_ripple` — native ripple feedback
- → `system_accent1_*` — wallpaper-based palette
- → `NavigationBar` — bar icon style / visibility

IOS

- Edge swipe back ↔ system back
- 44 pt ↔ 48 dp touch targets
- Dynamic Type ↔ font scale

ANDROID

- Font scale goes up to 200% — test it
- Cap growth with `maxFontSizeMultiplier`
- Use `BackHandler` for custom back behavior

EDGE-TO-EDGE CHECKLIST

ITEM	HOW
Enable	`edgeToEdgeEnabled=true` (template default)
Top / bottom padding	`useSafeAreaInsets()`
Keyboard	`KeyboardAvoidingView` (fixed for 15+ in 0.86)
Status bar	`barStyle` only; color props removed 0.87
Nav bar contrast	`enforceNavigationBarContrast` theme attr

ANDROID-ONLY APIS

API	STATUS
`BackHandler`	Current; also tvOS menu key
`ToastAndroid`	Current
`PermissionsAndroid`	Current
`DrawerLayoutAndroid`	Deprecated 0.87 → `react-native-drawer-layout`
`TouchableNativeFeedback`	Legacy → `Pressable android_ripple`
`UIManager.setLayoutAnimationEnabledExperimental`	No-op on New Arch

⚠ COMMON MISTAKES

- Returning `true` from every back handler; users get trapped and can't exit the app
- Setting a solid status bar color and assuming content starts below it; content now draws underneath
- iOS-style navigation without a back affordance; Android users rely on the system gesture

✓ TIPS & BEST PRACTICES

- Register back handlers inside `useFocusEffect` so only the visible screen intercepts back
- Test 3-button and gesture navigation modes
- Use `expo-system-ui` / app theme background to avoid white flashes behind bars

QUICK SUMMARY

- `BackHandler` — system back
- Edge-to-edge — pad with insets
- `android_ripple` / Material You — native feel
- Expo UI Compose — real Material components

Android Push Notifications

PUSH ON ANDROID

Android pushes go through **Firebase Cloud Messaging (FCM)**. Add your app to a Firebase project, download `google-services.json`, and give your server (or EAS) an **FCM v1 service account key**. On Android 13+ you must request `POST_NOTIFICATIONS`. Every notification belongs to a **channel** (Android 8+) that controls sound and importance, and that users can mute individually.

REQUIREMENTS

- `google-services.json` in config (`android.googleServicesFile`)
- FCM v1 service account JSON uploaded to EAS for Expo Push
- Channel created before the first notification
- Device must have Google Play services (not some Chinese OEMs)

CHANNEL, PERMISSION, TOKEN

```
import * as Notifications from 'expo-notifications'

async function setupAndroidPush() {
  await Notifications.setNotificationChannelAsync('orders', {
    name: 'Order updates',
    importance: Notifications.AndroidImportance.HIGH,
    sound: 'default',
    vibrationPattern: [0, 250, 250, 250],
    lightColor: '#7e57c2',
  })
  const { status } = await Notifications.requestPermissionsAsync() // 13+
  if (status !== 'granted') return null
  const { data: fcmToken } = await Notifications.getDevicePushTokenAsync()
  return fcmToken
}

// app.json
// "android": { "googleServicesFile": "./google-services.json" },
// "plugins": [["expo-notifications", {
//    "icon": "./assets/notification-icon.png", "color": "#7e57c2",
//    "defaultChannel": "orders" }]]

// Server → FCM v1 (HTTP)
// POST https://fcm.googleapis.com/v1/projects/<id>/messages:send
// { "message": { "token": "<fcmToken>",
//   "notification": { "title": "Shipped", "body": "Order #42" },
//   "android": { "notification": { "channel_id": "orders" } },
//   "data": { "url": "/orders/42" } } }
```

NOTES

- → `setNotificationChannelAsync` — idempotent; call on startup
- → `importance: HIGH` — heads-up banner
- → Device token — raw FCM token for your server
- → Notification icon — white silhouette PNG
- → `channel_id` — must match an existing channel

IOS

- APNs key instead of FCM — no channels
- A permission prompt is always required
- Silent pushes are throttled by iOS

ANDROID

- Heads-up banners need a high-importance channel
- Data-only messages need a background handler
- Small icon: white on transparent

MESSAGE TYPES

TYPE	BEHAVIOR
`notification`	System shows it when app is backgrounded
`data only`	Delivered to app code; you display it
`Both`	Shown by system; data on tap
`High priority`	Wakes device from Doze (use sparingly)

CHANNEL IMPORTANCE

LEVEL	EFFECT
`MAX / HIGH`	Sound + heads-up banner
`DEFAULT`	Sound, no banner
`LOW`	No sound
`MIN`	Silent, shade only
`Note`	Users can override; you can't raise it later

⚠ COMMON MISTAKES

- Not creating a channel; on Android 8+ the notification is silently dropped
- Using a full-color icon; Android renders it as a white blob, use a transparent silhouette
- Changing a channel's importance in code after creation; Android ignores it, create a new channel ID

✓ TIPS & BEST PRACTICES

- Create channels per category (orders, chat, marketing) so users mute only what they want
- Use data messages + local notifications when you need custom display logic
- Test background and killed states; delivery differs from foreground

QUICK SUMMARY

- FCM — Android push transport
- `google-services.json` — Firebase config
- Channels — required, user-controlled
- `POST_NOTIFICATIONS` — Android 13+

Sheet 61

Build Types, Flavors & SDK Levels

minSdk / targetSdk / compileSdk · buildTypes · productFlavors · R8 / ProGuard · Kotlin & AGP versions · ABI splits

SHAPING THE ANDROID BUILD

Three SDK numbers matter: **minSdk** (oldest Android you support), **targetSdk** (the behavior level you've tested; Google Play requires a recent one), and **compileSdk** (APIs you compile against). **Build types** (debug, release) control signing and minification; **product flavors** create variants like `free` and `pro`. **R8** shrinks and obfuscates release builds.

CURRENT NUMBERS

- RN 0.87 template: min 24, target 36, compile 37
- Play: new apps & updates must target API 36 (since Aug 31, 2026)
- Libraries need compileSdk ≥ 34 (`minCompileSdk` , 0.87)
- Expo SDK 58 enables R8 by default in release

APP/BUILD.GRADLE EXCERPTS

```
android {
  namespace "com.acme.myapp"
  defaultConfig {
    applicationId "com.acme.myapp"
    minSdkVersion rootProject.ext.minSdkVersion        // 24
    targetSdkVersion rootProject.ext.targetSdkVersion  // 36
    versionCode 42
    versionName "1.4.0"
  }
  buildTypes {
    debug { applicationIdSuffix ".debug" }
    release {
      signingConfig signingConfigs.release
      minifyEnabled true                    // R8
      shrinkResources true
      proguardFiles getDefaultProguardFile("proguard-android-optimize.txt"),
        "proguard-rules.pro"
    }
  }
  flavorDimensions "tier"
  productFlavors {
    free { dimension "tier"; applicationIdSuffix ".free" }
    pro  { dimension "tier" }
  }
}
// Expo: expo-build-properties → android.enableMinifyInReleaseBuilds
```

NOTES

- → `namespace` — Kotlin/R class package (AGP 8+)
- → `applicationIdSuffix` — install debug next to release
- → `minifyEnabled` — runs R8 shrinking
- → `proguard-rules.pro` — keep rules for reflection-based libs
- → Flavors × build types — `freeRelease` , `proDebug` …

IOS

- `minSdk` ↔ deployment target
- R8 shrinking ↔ Xcode dead-code stripping
- AAB per-device splits ↔ App Thinning

ANDROID

- Keep rules in `proguard-rules.pro` for reflection libraries
- AAB lets Play deliver per-ABI APKs
- Test minified release builds before uploading

API LEVELS

API	ANDROID
24	7.0 (RN minimum)
33	13 (notification permission)
35	15 (edge-to-edge enforced)
36	16 (Play target requirement 2026)
37	Next (compileSdk in 0.87)

SHRINKING & SIZE

TECHNIQUE	EFFECT
`R8 (minifyEnabled)`	Removes unused code, obfuscates
`shrinkResources`	Removes unused resources
`AAB`	Play serves per-device APKs
`ABI filters / splits`	Fewer native libs per APK
`Hermes bytecode`	Smaller, faster-loading JS

⚠ COMMON MISTAKES

- Turning on R8 without testing a release build; reflection-based libraries crash until you add keep rules
- Bumping `targetSdk` without reading behavior changes (edge-to-edge, predictive back, large-screen rules)
- Hardcoding SDK numbers in each module instead of `rootProject.ext`

✓ TIPS & BEST PRACTICES

- Use `./gradlew :app:assembleRelease` locally before every store upload
- Keep `mapping.txt` from R8 builds and upload it for readable crash stack traces
- Native libs must support 16 KB memory pages for Play; RN 0.77+ does, check third-party `.so` files

QUICK SUMMARY

- min / target / compile — support, behavior, APIs
- Build types — debug vs release
- Flavors — app variants
- R8 — shrink & obfuscate

Android App Signing

SIGNING FOR RELEASE

Every APK/AAB is signed. With **Play App Signing** (required for new apps), Google holds the final **app signing key** and you sign uploads with your own **upload key**. If you lose the upload key, Google can reset it; if you lost the old self-managed key, you couldn't update the app. Debug builds use a shared debug keystore. EAS can generate and store the upload keystore for you.

TWO KEYS

- Upload key — yours, signs what you upload
- App signing key — Google's, signs what users install
- Fingerprints of **both** go into Firebase, Maps, App Links
- Never commit keystores or passwords

CREATE AND CONFIGURE AN UPLOAD KEY

```
keytool -genkeypair -v -storetype PKCS12 \
  -keystore my-upload-key.keystore -alias my-key-alias \
  -keyalg RSA -keysize 2048 -validity 10000
mv my-upload-key.keystore android/app/

# ~/.gradle/gradle.properties (outside the repo)
MYAPP_UPLOAD_STORE_FILE=my-upload-key.keystore
MYAPP_UPLOAD_KEY_ALIAS=my-key-alias
MYAPP_UPLOAD_STORE_PASSWORD=*****
MYAPP_UPLOAD_KEY_PASSWORD=*****

# android/app/build.gradle
# signingConfigs { release {
#   storeFile file(MYAPP_UPLOAD_STORE_FILE)
#   storePassword MYAPP_UPLOAD_STORE_PASSWORD
#   keyAlias MYAPP_UPLOAD_KEY_ALIAS
#   keyPassword MYAPP_UPLOAD_KEY_PASSWORD } }

cd android && ./gradlew signingReport      # SHA-1 / SHA-256 of all keys
keytool -list -v -keystore android/app/my-upload-key.keystore

eas credentials -p android                 # EAS-managed keystore
```

NOTES

- → `PKCS12` — modern keystore format
- → `-validity 10000` — ~27 years
- → Global `gradle.properties` — keeps secrets out of git
- → `signingReport` — fingerprints for debug and release
- → `eas credentials` — download a backup of the keystore

IOS

- Certificate + provisioning profile per Apple team
- No fingerprints — the Team ID goes in the AASA file
- EAS can generate and store everything

ANDROID

- `keytool -genkeypair -v -storetype PKCS12 -keystore upload.keystore -alias upload -keyalg RSA -keysize 2048 -validity 10000`
- Keep passwords in `~/.gradle/gradle.properties`
- `./gradlew signingReport` prints SHA-1 / SHA-256

WHERE FINGERPRINTS GO

SERVICE	NEEDS
Firebase / Google Sign-In	SHA-1 + SHA-256 (debug, upload, app signing)
App Links assetlinks.json	SHA-256 of app signing key
Google Maps API key	SHA-1 restriction
Play Console	Shows app signing + upload certs

IF SOMETHING GOES WRONG

SITUATION	FIX
Lost upload key	Request upload key reset in Play Console
Leaked upload key	Reset upload key
Wrong key on upload	Play rejects; sign with registered upload key
Signature mismatch installing	Uninstall old build first

⚠ COMMON MISTAKES

- Committing the keystore and passwords to git; anyone with them can sign updates as you
- Adding only the debug SHA-1 to Firebase; Google Sign-In fails in Play builds
- Not backing up the upload keystore outside your laptop or CI

✓ TIPS & BEST PRACTICES

- Store keystores in a password manager or let EAS manage them
- Use separate debug keystores per developer only if a service requires it; otherwise share the default
- Copy fingerprints from Play Console → App integrity for the app signing key

QUICK SUMMARY

- Upload key — you sign uploads
- Play App Signing — Google signs installs
- `keytool` / `signingReport` — create & inspect
- Keep secrets out of git

Google Play Release

.aab bundle · Play Console · internal / closed / open testing tracks · target SDK requirements · Data safety form · staged rollout

SHIPPING ON GOOGLE PLAY

Play takes an **Android App Bundle (.aab)** and generates optimized APKs per device. You upload to a **track**: internal testing (fast, up to 100 testers), closed testing, open testing, then production. Before your first release you complete the store listing, content rating, **Data safety** form, target audience, and, for new personal developer accounts, a **closed test with at least 12 testers for 14 days**.

REQUIREMENTS (2026)

- Target API 36 for new apps and updates (since Aug 31, 2026)
- AAB required for new apps
- 16 KB page size support for native libraries
- Privacy policy URL; account deletion if accounts exist

BUILD AND UPLOAD

```
# Bare: build a signed AAB
cd android && ./gradlew bundleRelease
# → android/app/build/outputs/bundle/release/app-release.aab

# Test the AAB locally with bundletool
bundletool build-apks --bundle=app-release.aab --output=app.apks \
  --ks=my-upload-key.keystore --ks-key-alias=my-key-alias
bundletool install-apks --apks=app.apks

# EAS: build + submit to the internal track
eas build -p android --profile production
eas submit -p android --latest
# eas.json → "submit": { "production": { "android": {
#   "track": "internal", "releaseStatus": "draft",
#   "serviceAccountKeyPath": "./play-service-account.json" } } }

# Promote internal → production in Play Console,
# start a staged rollout (e.g. 10%) and watch Android vitals
```

NOTES

- → `bundleRelease` — signed with your upload key
- → `bundletool` — reproduce what Play will install
- → `track` — `internal` · `alpha` · `beta` · `production`
- → `releaseStatus: draft` — review before rollout
- → Service account — needs Play Console API access

IOS

- TestFlight ↔ internal / closed testing tracks
- App Privacy label ↔ Data safety form
- Phased release ↔ staged rollout

ANDROID

- New personal accounts: 12 testers for 14 days before production
- Staged rollouts by percentage
- The pre-launch report tests on real devices

TESTING TRACKS

TRACK	NOTES
Internal testing	≤ 100 testers, available in minutes
Closed testing	Invite lists / Google Groups
Open testing	Anyone can join from the listing
Production	Staged rollout by percentage
Pre-launch report	Automated tests on real devices
Halt rollout	Stop a staged release in Play Console

COMMON POLICY ISSUES

ISSUE	FIX
Target API too low	Bump `targetSdk`, test behavior changes
Undeclared sensitive permission	Remove it or file a declaration
Data safety mismatch	Match SDK data collection (analytics, ads)
Background location	Prominent disclosure + declaration
Missing account deletion	In-app + web deletion path
Ads SDK undeclared	Mark "contains ads" in App content

⚠ COMMON MISTAKES

- Uploading an APK for a new app; Play requires AAB
- Filling Data safety without checking what third-party SDKs collect; mismatches lead to removal
- Rolling out 100% immediately; halt is only possible during a staged rollout

✓ TIPS & BEST PRACTICES

- Ship every build to the internal track first; it's the fastest way to test on real devices
- Watch Android vitals (crash and ANR rates); bad numbers reduce store visibility
- Keep release notes per language in `eas metadata` or fastlane supply

QUICK SUMMARY

- `.aab` — required upload format
- Tracks — internal → production
- Data safety + target API — policy gates
- Staged rollout — limit risk

Android TV, Large Screens & Other Devices

ANDROID BEYOND PHONES

The same Android build can run on tablets, foldables, Chromebooks, and TVs. For **Android TV**, use **react-native-tvos** (same fork as Apple TV) for D-pad focus handling and declare the leanback launcher. For **large screens**, Android 16 ignores orientation and resizability restrictions on displays ≥ 600 dp for apps targeting API 36, so layouts must adapt.

LARGE-SCREEN FACTS

- Window size changes on fold/unfold and in multi-window
- `useWindowDimensions` updates; design with breakpoints
- ChromeOS adds mouse, keyboard, and free-form windows
- Meta Quest (Horizon OS) runs Android apps, incl. RN

TV MANIFEST + ADAPTIVE LAYOUT

```
<!-- AndroidManifest.xml additions for Android TV -->
<uses-feature android:name="android.software.leanback"
  android:required="false" />
<uses-feature android:name="android.hardware.touchscreen"
  android:required="false" />

<application android:banner="@drawable/tv_banner">
  <activity android:name=".MainActivity"
    android:resizeableActivity="true"
    android:configChanges="..."> <!-- keep RN's default list -->
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
      <category android:name="android.intent.category.LEANBACK_LAUNCHER" />
    </intent-filter>
  </activity>
</application>
```

IOS

- iPad multitasking ↔ multi-window
- Apple TV ↔ Android TV
- visionOS ↔ Android XR

ANDROID

- Foldable posture needs Jetpack WindowManager (native)
- Android Auto / Automotive need the Car App Library
- Test resizing in the resizable emulator

BRANCHING IN JS

```
const { width } = useWindowDimensions()
const layout = Platform.isTV ? 'tv'
  : width >= 840 ? 'expanded'      // tablets, unfolded
  : width >= 600 ? 'medium'        // small tablets, folded-out
  : 'compact'                      // phones

// Android TV emulator
// npx react-native run-android --device tv_api_31

// Detect a hinge / fold posture
// Jetpack WindowManager via a small native module
```

WINDOW SIZE CLASSES

CLASS	WIDTH (DP)
Compact	< 600 (phones)
Medium	600–839 (foldables, small tablets)
Expanded	840–1199 (tablets)
Large / Extra-large	≥ 1200 (desktop windows)

DEVICE NOTES

DEVICE	WATCH OUT FOR
Android TV	Focus/D-pad, 10-foot UI, banner image
Foldables	Resize on fold; keep state across config changes
Tablets	Two-pane layouts, all orientations
ChromeOS	Mouse hover, keyboard, window resizing
Wear OS / Auto	Not supported by RN; use native modules/apps

⚠ COMMON MISTAKES

- Locking orientation and expecting it to hold on tablets; Android 16 ignores the lock on large screens
- Using core `react-native` for Android TV and fighting focus issues; react-native-tvos handles D-pad focus
- Shipping a TV build without a `banner`; it won't appear properly on the TV launcher

✓ TIPS & BEST PRACTICES

- Use the Resizable emulator and Pixel Fold / Tablet AVDs to test window changes
- Share breakpoints with your iPad layouts
- Keep state in stores, not in components that remount on configuration changes

QUICK SUMMARY

- react-native-tvos — Android TV focus
- Leanback launcher — TV entry point
- Window size classes — adaptive layout
- Android 16 — no orientation locks on large screens

React Native Windows Setup

NATIVE WINDOWS APPS

React Native for Windows (RNW) is Microsoft's out-of-tree platform for building native Windows desktop apps. It runs on the **New Architecture** (Fabric) only since 0.82, rendering through the Windows App SDK. RNW tracks core with a short lag (latest **0.84** as of Oct 2026); your RN version must match its minor. Build on a Windows machine with **Visual Studio**.

REQUIREMENTS

- Windows 10/11 dev machine; Visual Studio 2026 for RNW 0.84
- VS workloads: Desktop development with C++, WinUI app development
- Node and Yarn (Yarn recommended)
- Developer Mode enabled in Windows settings

ADD WINDOWS TO A PROJECT

```
# Create a project at the matching RN version
npx --yes @react-native-community/cli@latest init MyApp --version 0.84.0
cd MyApp
yarn add react-native-windows@^0.84.0
npx react-native init-windows --overwrite     # creates windows/

npx react-native run-windows                  # build, deploy, launch
npx react-native run-windows --release --arch x64
npx react-native autolink-windows             # before opening VS

# Visual Studio: open windows/MyApp.sln,
# Debug | x64, run `yarn start`, press Run

# Check / install prerequisites
powershell -ExecutionPolicy Unrestricted -File `
  node_modules\react-native-windows\scripts\rnw-dependencies.ps1
```

NOTES

- → RN and RNW must share the same minor version
- → `init-windows --overwrite` — also rewrites `metro.config.js`
- → `run-windows` — builds the `.sln` and deploys
- → `autolink-windows` — links native modules into the solution
- → `rnw-dependencies.ps1` — installs/checks prerequisites

IOS

- iOS builds need a Mac; Windows builds need Windows
- UIKit-only libraries have no Windows port
- `.ios.tsx` is ignored — add `.windows.tsx`

ANDROID

- One Windows PC can build Android and Windows
- RN Windows also resolves `.native.tsx`
- Android-only libraries need Windows fallbacks too

WINDOWS/ FOLDER

ITEM	PURPOSE
`MyApp.sln`	Visual Studio solution
`MyApp/`	C++ app project (default template)
`MyApp.Package/`	Packaging project (MSIX)
`AutolinkedNativeModules.g.*`	Generated by autolinking
`ExperimentalFeatures.props`	Feature flags
`NuGet packages`	Native deps (Windows App SDK)

JS DIFFERENCES ON WINDOWS

API	NOTE
`Platform.OS`	`'windows'`
`.windows.tsx`	Platform files
`Keyboard & focus`	`onKeyDown`, `focusable`, tab order
`Mouse`	`onMouseEnter` / `onMouseLeave` (Pressable hover)
`Libraries`	Check "Windows" support on reactnative.directory
`Debug JS`	Web debugger default in Debug builds

⚠ COMMON MISTAKES

- Installing RNW 0.84 into an RN 0.87 app; the minors must match
- Running `init-windows --overwrite` without backing up a customized `metro.config.js`
- Missing VS workloads; builds fail with cryptic MSBuild errors

✓ TIPS & BEST PRACTICES

- Run `rnw-dependencies.ps1` on new machines to set up everything in one go
- Keep Windows-specific UI in `.windows.tsx` files and share the rest
- Pin RN to the newest version RNW supports when Windows is a target

QUICK SUMMARY

- RNW — Microsoft's Windows platform
- `init-windows` / `run-windows` — setup & run
- Visual Studio + C++ workloads — required
- Match RN minor versions

Sheet 66

Windows Native Modules & APIs

C++ / C# modules · WinRT APIs · WinAppSDK · Fabric on Windows · platform checks · Windows-only components

CALLING WINDOWS APIS

On Windows, Turbo Native Modules are written in **C++/WinRT** (or C#) using attribute macros, and share the same **TypeScript spec + Codegen** flow as iOS and Android. `codegen-windows` generates a spec struct; referencing it as `ModuleSpec` makes the compiler check your implementation against the spec. From native code you can call any **WinRT** or Windows App SDK API.

WORKFLOW

C++ TURBO MODULE

```
// windows/testlib/testlib.h
#include "codegen/NativeTestlibSpec.g.h"
#include "NativeModules.h"

namespace winrt::testlib {
REACT_MODULE(Testlib)
struct Testlib {
  using ModuleSpec = testlibCodegen::TestlibSpec;   // compile-time check

  REACT_INIT(Initialize)
  void Initialize(React::ReactContext const &reactContext) noexcept {
    m_context = reactContext;
  }

  REACT_SYNC_METHOD(multiply)
  double multiply(double a, double b) noexcept { return a * b; }

  REACT_METHOD(getDeviceName)
  void getDeviceName(React::ReactPromise<std::string> promise) noexcept {
    promise.Resolve(winrt::to_string(
      winrt::Windows::Security::ExchangeActiveSyncProvisioning::
        EasClientDeviceInformation().FriendlyName()));
  }
 private:
  React::ReactContext m_context;
};
}
// ReactPackageProvider.cpp:  AddAttributedModules(packageBuilder, true);
```

NOTES

- → `REACT_MODULE` — exposes the struct as a module
- → `ModuleSpec` — fails to compile if it drifts from the TS spec
- → `REACT_INIT` — runs once when created
- → `REACT_SYNC_METHOD` / `REACT_METHOD` — sync / async
- → `ReactPromise<T>` — resolve or reject async calls

IOS

- One TS spec generates iOS, Android and Windows bindings
- Pure C++ Turbo Modules can be shared with iOS
- iOS adds C++ through Objective-C++

ANDROID

- Pure C++ modules need CMake + JNI registration on Android
- Or implement the same spec in Kotlin
- Share C++ logic, keep platform UI separate

ATTRIBUTE MACROS

MACRO	PURPOSE
`REACT_MODULE(Name)`	Declare a module
`REACT_INIT(fn)`	Initialization hook
`REACT_METHOD(fn)`	Async method (callback / promise)
`REACT_SYNC_METHOD(fn)`	Synchronous method
`REACT_CONSTANT / REACT_GET_CONSTANTS`	Constants
`REACT_EVENT(name)`	Native → JS events

WINDOWS NOTES

TOPIC	DETAIL
`New .h/.cpp files`	Add to `.vcxproj` + `.filters` manually
`Threading`	UI work on the UI dispatcher
`C# modules`	Attributes like `[ReactModule]`
`Native components`	Fabric ComponentView via Composition
`Sample`	`react-native-windows-samples` repo

⚠ COMMON MISTAKES

- Removing the `ModuleSpec` line to "fix" compile errors; the JS and native APIs silently drift
- Forgetting to add new source files to the `.vcxproj` ; they're not compiled
- Calling UI APIs from a background thread; dispatch to the UI thread

✓ TIPS & BEST PRACTICES

- Start from the native module sample library and copy its structure
- Keep the TS spec shared across iOS, Android, and Windows for one JS API
- Use C++/WinRT projections to call Windows APIs without COM boilerplate

QUICK SUMMARY

- TS spec + `codegen-windows` — same contract
- `REACT_MODULE` / `REACT_METHOD` — C++ macros
- `ModuleSpec` — compile-time checking
- WinRT — full Windows API access

Packaging & Distributing on Windows

SHIPPING A WINDOWS APP

Windows apps built with RNW are packaged as **MSIX**, the modern Windows installer format with clean install/uninstall and automatic updates. You produce a **Release** build with the JS bundle embedded, sign the package with a certificate, and then publish to the **Microsoft Store** (which re-signs it) or distribute it directly (sideloading, App Installer, or enterprise tools like Intune).

SIGNING RULES

- Store submissions: Microsoft signs for you
- Direct distribution: sign with a trusted code-signing cert
- Self-signed certs need manual trust on each PC
- Publisher in the manifest must match the certificate subject

RELEASE BUILDS AND PACKAGING

```
# Release build with bundled JS
npx react-native run-windows --release --arch x64
npx react-native run-windows --release --no-launch --no-deploy

# Visual Studio → right-click MyApp.Package → Publish →
#   Create App Packages → Microsoft Store or Sideloading
# Output: AppPackages\MyApp_1.0.0.0_x64.msixbundle

# Sign for sideloading (Windows SDK signtool)
signtool sign /fd SHA256 /a /f MyCert.pfx /p <password> MyApp.msix

# Install locally
Add-AppxPackage .\MyApp_1.0.0.0_x64.msix

# Package.appxmanifest essentials:
#   <Identity Name="Acme.MyApp" Publisher="CN=Acme" Version="1.0.0.0" />
#   <Capabilities> <Capability Name="internetClient" /> </Capabilities>
```

NOTES

- → `--release` — embeds bundle, disables dev tools
- → `--no-launch --no-deploy` — build only (CI)
- → `.msixbundle` — multiple architectures in one file
- → `signtool` — part of the Windows SDK
- → `Add-AppxPackage` — PowerShell install

IOS

- Microsoft Store ↔ App Store: the store signs for you
- MSIX ↔ `.ipa`
- Package identity ↔ bundle ID

ANDROID

- Sideloading MSIX ↔ sideloading APK
- Store listing ↔ Play listing
- Version must increase on each submission

DISTRIBUTION OPTIONS

CHANNEL	NOTES
Microsoft Store	Partner Center account; Store signs & updates
App Installer (.appinstaller)	Web-hosted auto-updates
Sideload MSIX	Signed package; manual install
Intune / Configuration Manager	Enterprise deployment
winget	Community package manager manifests
Unpackaged	Plain .exe (no MSIX), you handle updates

MANIFEST BASICS

ELEMENT	MEANING
Identity Version	Four-part version `1.2.0.0`
Publisher	Must equal cert subject
Capabilities	Permissions (internet, webcam…)
VisualElements	Tiles, logos, display name
TargetDeviceFamily	Min/max Windows versions
Applications	Entry point & app ID

⚠ COMMON MISTAKES

- Shipping a Debug build that expects Metro; it shows a red screen on users' PCs
- Mismatched `Publisher` and certificate subject; package installation fails
- Forgetting capabilities (e.g. `webcam`); APIs fail with access denied

✓ TIPS & BEST PRACTICES

- Use the Store for consumer apps to get signing and updates for free
- Bump the four-part version on every package so updates install
- Automate packaging in CI with `msbuild` and the `.Package` project

QUICK SUMMARY

- MSIX — Windows package format
- `--release` — bundled JS build
- Sign — Store or your certificate
- Store / App Installer / Intune — channels

React Native for Web

THE SAME COMPONENTS IN A BROWSER

react-native-web implements RN components and APIs on top of **React DOM**, so `View` becomes a `div` and `Text` a `span` with styles turned into atomic CSS. **Expo** wires it up with Metro: `npx expo start --web` runs it, and Expo Router gives every screen a real URL with static, server, or single-page output. Use `.web.tsx` files where the browser needs something different.

GOOD TO KNOW

- Latest react-native-web: `0.21.x`
- Most core components work; some APIs are no-ops on web
- Native-only libraries need web fallbacks
- Accessibility props map to ARIA

RUNNING AND EXPORTING

```
npx expo install react-dom react-native-web @expo/metro-runtime
npx expo start --web                  # dev server
npx expo export --platform web        # → dist/
npx serve dist                        # preview locally

# app.json
# "web": { "output": "static", "favicon": "./assets/favicon.png" }
#   static → pre-rendered HTML per route (SEO)
#   server → server rendering, loaders, API routes
#   single → one index.html SPA
```

IOS

- On iPhone the web build runs in Safari (WebKit)
- Use `env(safe-area-inset-*)` for the notch
- No native modules — check `Platform.OS === 'web'`

ANDROID

- On Android the web build runs in Chrome
- PWAs can be installed to the home screen
- Hardware back follows browser history

PLATFORM-AWARE CODE

```
// Button.web.tsx overrides Button.tsx on web
import { Platform } from 'react-native'

const isWeb = Platform.OS === 'web'

<Pressable
  style={({ hovered, pressed }) => [styles.btn, hovered && styles.hover]}
  accessibilityRole="link"
  href="/pricing"                  // RNW renders an <a>
/>

<Text selectable style={{ userSelect: 'text' }}>Copy me</Text>

// Web-only APIs behind a guard
if (isWeb) document.title = 'Pricing · MyApp'

// Expo Router metadata
import Head from 'expo-router/head'
<Head><title>Pricing</title><meta name="description" content="…" /></Head>
```

SUPPORT ON WEB

API	WEB BEHAVIOR
`View` / `Text` / `Image` / `ScrollView`	✔ Full
`FlatList`	✔ (virtualized)
`Modal`	✔ portal
`Alert.alert`	Basic (`window.alert` / `confirm`)
`Vibration` / `PermissionsAndroid`	No-op
`StatusBar` / `BackHandler`	No-op

EXPO WEB OUTPUT

MODE	USE
`static`	Marketing, docs, SEO pages
`server`	Data loaders, server rendering, API routes
`single`	Client-only app behind auth
`Hosting`	EAS Hosting, Vercel, Netlify, any static host

⚠ COMMON MISTAKES

- Calling `document` or `window` at module scope in shared code; native crashes, and static rendering breaks
- Assuming every npm RN library works on web; check for web support or add a `.web.tsx` fallback
- Using `Dimensions` for responsive web layouts; use `useWindowDimensions` or media queries via a library

✓ TIPS & BEST PRACTICES

- Use Expo Router `static` output for public pages and `+api.ts` routes for backend endpoints
- Test keyboard navigation and screen readers; RNW maps roles to ARIA
- Consider separate web layouts (sidebars) instead of stretching phone UIs

QUICK SUMMARY

- react-native-web — RN on React DOM
- `expo start --web` / `export` — run & build
- `.web.tsx` — browser overrides
- `web.output` — static, server, single

Web-Specific Patterns & DOM Components

'use dom' DOM components · responsive breakpoints · hover & cursor · SEO / metadata · Expo Router web hosting

WHEN NATIVE MEETS THE WEB

DOM components (`'use dom'`, Expo SDK 52+) let you drop real web code (HTML, CSS, web-only libraries like charts or rich-text editors) into a native app; on native it renders inside a WebView, on web it renders directly. Combine that with responsive breakpoints, hover states, and per-route metadata to make one codebase feel right in both places.

DOM COMPONENT RULES

- First line of the file: `'use dom'`
- Props must be serializable; functions become async bridges
- Uses `@expo/dom-webview` by default (SDK 56)
- Heavier than native views; use for islands, not whole screens

A DOM COMPONENT

```tsx
// components/RichEditor.tsx
'use dom'
import { useState } from 'react'
import 'quill/dist/quill.snow.css'

export default function RichEditor({ initial, onSave }:
  { initial: string; onSave: (html: string) => Promise<void>;
    dom?: import('expo/dom').DOMProps }) {
  const [html, setHtml] = useState(initial)
  return (
    <div style={{ padding: 12 }}>
      <div contentEditable
        onInput={e => setHtml(e.currentTarget.innerHTML)}
        dangerouslySetInnerHTML={{ __html: initial }} />
      <button onClick={() => onSave(html)}>Save</button>
    </div>
  )
}

// Used from a native screen
<RichEditor initial={note.html}
  onSave={async html => { await api.save(note.id, html) }}
  dom={{ scrollEnabled: false, matchContents: true }} />
```

NOTES

- → `'use dom'` — whole file runs in web context
- → CSS imports — allowed inside DOM components
- → `onSave` — native function called from the web side
- → `dom` prop — WebView options
- → On web it's just a normal React component

IOS

- DOM components render in WKWebView
- Debug with Safari's Develop menu
- Enable inspection with `webviewDebuggingEnabled`

ANDROID

- Renders in Android System WebView (Chromium)
- Debug with `chrome://inspect`
- Old devices may have old WebViews — avoid new CSS

WEB-FRIENDLY PATTERNS

NEED	APPROACH
Breakpoints	`useWindowDimensions` / NativeWind `md:`
Hover	`Pressable` `hovered` state, `cursor: 'pointer'`
Links	`<Link href>` renders `<a>` on web
Metadata	`expo-router/head` or `generateMetadata` (SDK 56)
Server data	Expo Router loaders (stable SDK 58)

DOM COMPONENT OR NATIVE?

CONTENT	PICK
Charts / rich text / maps from web libs	DOM component
Lists, forms, navigation	Native components
Marketing HTML/CSS	DOM component
Performance-critical UI	Native components

⚠ COMMON MISTAKES

- Building entire screens as DOM components; scrolling and gestures feel non-native
- Passing non-serializable props (class instances, refs) into a DOM component
- Forgetting the file-level `'use dom'` directive, so the component tries to render HTML natively and crashes

✓ TIPS & BEST PRACTICES

- Use DOM components to adopt a web library today, then replace with native later if needed
- Share design tokens between web CSS and RN styles
- Deploy web builds with EAS Hosting alongside your native releases

QUICK SUMMARY

- `'use dom'` — web code in native apps
- Serializable props — async function bridges
- Hover / Link / Head — web polish
- Islands, not screens

Other Out-of-Tree Platforms

BEYOND THE BIG THREE

React Native's core targets iOS and Android. Everything else is **out-of-tree**: a fork or add-on package that implements the renderer and native modules for another OS, tracked against core releases. The React Native docs list partner platforms (macOS, Windows, visionOS, OpenHarmony) and community ones (tvOS, Web, Skia). Each has its own setup, release cadence, and library compatibility.

EVALUATING A PLATFORM

- Which RN version does it support today?
- New Architecture support?
- Do your key libraries support it?
- Who maintains it, and how active is it?

HOW OUT-OF-TREE PLATFORMS PLUG IN

```
// 1. Install the platform package (version matched to RN)
//    yarn add react-native-windows@^0.84.0
// 2. Metro resolves platform-specific files
//    Screen.windows.tsx · Screen.macos.tsx · Screen.harmony.tsx
// 3. Platform.OS reports the platform
switch (Platform.OS) {
  case 'ios': case 'android': break
  case 'windows': case 'macos': /* desktop UI */ break
  case 'web': /* browser */ break
  default: /* other out-of-tree platforms */
}
// 4. Native modules need an implementation per platform;
//    unsupported ones should fail gracefully:
const Haptics = NativeModules.Haptics ?? { impact: () => {} }

// react-native.config.js can declare extra platforms for autolinking
module.exports = { project: { windows: { sourceDir: 'windows' } } }
```

NOTES

- → Version match — platform packages track RN minors
- → Platform files — Metro picks by extension
- → `Platform.OS` — string per platform
- → Graceful fallbacks — keep shared code safe
- → `react-native.config.js` — per-platform project settings

IOS

- iOS and Android are in-tree — maintained in core
- macOS, tvOS and visionOS are forks that track iOS
- Apple targets reuse most iOS native code

ANDROID

- Android-based devices need no extra platform
- TV, Quest, Android XR and ChromeOS use the Android build
- HarmonyOS NEXT isn't Android — needs RNOH

OUT-OF-TREE PLATFORMS

PLATFORM	MAINTAINER · LATEST (OCT 2026)
macOS	Microsoft · 0.83
Windows	Microsoft · 0.84
visionOS	Callstack · 0.79
OpenHarmony	Huawei partners (CPF-RN)
tvOS / Android TV	Community · 0.87
Web (react-native-web)	Community · 0.21
Skia (Linux, macOS)	Community, experimental

CHOOSING AN APPROACH

GOAL	OPTION
Desktop app, native feel	RN Windows / macOS
Desktop app, web stack OK	Electron or Tauri + react-native-web
Spatial (Vision Pro)	RN visionOS or iPad compatibility mode
TV	react-native-tvos
Huawei HarmonyOS NEXT	React Native OpenHarmony
Embedded / kiosk Linux	Skia renderer (experimental)

⚠ COMMON MISTAKES

- Picking a platform without checking its RN version; you may be stuck several releases behind core
- Assuming a library's iOS support covers macOS or visionOS; each platform needs its own native code
- Mixing a fork package and core `react-native` in one project

✓ TIPS & BEST PRACTICES

- Keep business logic platform-agnostic and push differences to the edges
- Use reactnative.directory filters to check platform support before adopting a library
- Follow each platform's release notes alongside core RN's

QUICK SUMMARY

- Out-of-tree — forks/add-ons per OS
- Match RN versions — critical
- Platform files + `Platform.OS` — branch code
- Partners vs community — maintenance levels

Sheet 71

Quick Reference — Components, Styling & Navigation

CLI · core components · lists · input & touch · hooks · StyleSheet & flexbox · safe areas & theming · Expo Router · React Navigation

CLI & EXPO

COMMAND	DOES	#
`npx create-expo-app@latest`	new Expo app	02
`npx expo start`	dev server	04
`npx expo run:ios / run:android`	native build	04
`npx expo prebuild`	generate native dirs	25
`npx expo install pkg`	compatible version	26
`npx @react-native-community/cli init`	bare RN app	02

CORE COMPONENTS

COMPONENT	DOES	#
`View`	layout container	10
`Text`	all text must be inside	10
`Image`	local / remote image	10
`ScrollView`	scroll small content	13
`Modal`	overlay screen	15
`ActivityIndicator`	spinner	15

LISTS

PROP	DOES	#
`data + renderItem`	FlatList basics	14
`keyExtractor`	stable keys	14
`ItemSeparatorComponent`	dividers	14
`onEndReached`	infinite scroll	14
`refreshing / onRefresh`	pull to refresh	14
`SectionList sections`	grouped list	14

INPUT & TOUCH

COMPONENT / PROP	DOES	#
`Pressable onPress`	touch target	12
`({ pressed }) => style`	pressed state	12
`hitSlop`	bigger touch area	12
`TextInput value / onChangeText`	controlled input	11
`keyboardType / secureTextEntry`	input type	11
`KeyboardAvoidingView`	lift above keyboard	11

HOOKS

HOOK	DOES	#
`useState / useEffect`	state / effects	07
`useReducer / useContext`	complex / shared state	08
`useMemo / useCallback`	memoize	07
`use(promise) / useActionState`	React 19	09
`useWindowDimensions()`	screen size	18
`useColorScheme()`	light / dark	19

STYLESHEET & FLEXBOX

ITEM	DOES	#
`StyleSheet.create({})`	define styles	16
`style={[a, b && c]}`	combine styles	16
`flexDirection: "column"`	default direction	17
`justifyContent / alignItems`	align	17
`flex: 1`	fill space	17
`gap / rowGap`	spacing	17

PLATFORM & LAYOUT

API	DOES	#
`Platform.OS`	"ios" / "android" / "web"	24
`Platform.select({ ios, android })`	per-platform value	24
`file.ios.tsx / file.android.tsx`	per-platform files	24
`SafeAreaView / useSafeAreaInsets`	notches	18
`Dimensions / PixelRatio`	screen metrics	18

EXPO ROUTER

FILE / API	DOES	#
`app/index.tsx`	/ route	20
`app/[id].tsx`	dynamic route	20
`_layout.tsx + <Stack> / <Tabs>`	layouts	21
`<Link href="/x">`	navigate	20
`router.push("/x")`	imperative	20
`useLocalSearchParams()`	route params	20

REACT NAVIGATION

API	DOES	#
`NavigationContainer`	root (bare RN)	22
`createNativeStackNavigator`	stack	22
`createBottomTabNavigator`	tabs	22
`navigation.navigate("X")`	go to screen	22
`route.params`	screen params	22
`linking config`	deep links	23

⚠ COMMON MISTAKES

- Text outside a `<Text>` component — it crashes on native
- `ScrollView` for long lists — use `FlatList` so rows are virtualized
- Unstable or index keys in lists causing re-render bugs

✓ TIPS & BEST PRACTICES

- Use `npx expo install` so native packages match your SDK
- Pass `({ pressed })` style functions for touch feedback
- Test on a real device early — simulators hide performance issues

QUICK SUMMARY

- View / Text / Image / Pressable / FlatList cover most screens
- StyleSheet + flexbox (column by default) for layout
- `Platform.select` and `.ios` / `.android` files for differences
- Expo Router: files in `app/` are screens

Sheet 72

Quick Reference — Data, Native & Release

config · device APIs · networking & state · storage · animation & gestures · native modules · testing & debugging · EAS · platform builds

APP CONFIG

ITEM	DOES	#
app.json / app.config.ts	Expo config	25
name / slug / version	app identity	25
ios.bundleIdentifier	iOS id	25
android.package	Android id	25
plugins: [...]	config plugins	25
scheme	deep link scheme	23

DEVICE APIS

PACKAGE	DOES	#
expo-camera	camera	26
expo-location	GPS	26
expo-notifications	push notifications	48
expo-image-picker	photo library	26
expo-haptics	vibration feedback	26
Linking.openURL(url)	open links	23

DATA & STATE

TOOL	DOES	#
fetch / TanStack Query	networking + cache	27
Zustand / Redux Toolkit	global state	28
AsyncStorage	key-value	29
expo-secure-store	secrets / tokens	29
expo-sqlite	local SQL DB	30
NetInfo	online / offline	30

ANIMATION & GESTURES

API	DOES	#
Animated.timing(v, {…})	built-in animation	31
useNativeDriver: true	run on UI thread	31
useSharedValue()	Reanimated value	32
useAnimatedStyle()	animated style	32
withSpring / withTiming	animations	32
Gesture.Pan() + GestureDetector	gestures	33

NATIVE CODE

ITEM	DOES	#
Turbo Native Module	native API via JSI	34
Codegen spec (NativeX.ts)	typed interface	34
Fabric component	native view	35
Expo Modules API	Swift / Kotlin modules	36
New Architecture	default since 0.76	01

TEST & DEBUG

TOOL	DOES	#
Jest + RNTL	unit / component	37
render / screen.getByText	RNTL queries	37
Maestro / Detox	end-to-end	38
React Native DevTools	debugger (j in Metro)	39
Hermes	JS engine	39
Flashlight / perf monitor	performance	39

EAS

COMMAND	DOES	#
eas build -p ios	cloud build	40
eas build --profile preview	internal build	40
eas submit	upload to stores	41
eas update	OTA JS update	42
eas.json	build profiles	40
runtimeVersion	OTA compatibility	42

IOS & ANDROID

ITEM	DOES	#
Info.plist usage strings	iOS permissions	46
Signing & provisioning	iOS release	49
TestFlight	iOS beta	50
AndroidManifest permissions	Android permissions	58
Upload key / Play App Signing	Android signing	62
.aab + Play Console tracks	Android release	63

⚠ COMMON MISTAKES

- Tokens in AsyncStorage — use `expo-secure-store`
- Shipping an OTA update that needs new native code — bump `runtimeVersion` and rebuild
- Animating layout on the JS thread — use the native driver or Reanimated

✓ TIPS & BEST PRACTICES

- Keep native config in `app.config.ts` + config plugins (Continuous Native Generation)
- Use `eas build --profile preview` builds for testers
- Profile release builds; dev mode is much slower

QUICK SUMMARY

- `app.json` / `app.config.ts` configure the app
- TanStack Query + Zustand for data and state
- Reanimated + Gesture Handler for motion
- EAS builds, submits and ships OTA updates